NeuroPLC: Structurally Verifiable Compilation from PyTorch to IEC 61131-3 SCL for Industrial PLCs

Fuyue Liu, *Guilin University of Electronic Technology, Guilin 541004, China*

Abstract

Can a compiler certify that neural network code deployed on a PLC preserves the trained model's behavior? We show that the answer depends not on compiler engineering but on model *architecture*. We introduce the **Structurally Verifiable Neural Network (SVNN)** framework: two conditions—operation-type closure and univariate boundedness—are sufficient for a compiler to compute, from model parameters alone, a finite per-output error bound valid for all inputs (Theorem 2). Standard MLPs provably fail these conditions (Proposition 1); Kolmogorov-Arnold Networks (KANs) with B-spline activations satisfy them. Violating the conditions makes tight bound computation NP-hard (Theorem 5); satisfying the stronger contractivity condition yields a depth-*improving* generalization bound $\Delta L \leq O(\gamma^L/\sqrt{n})$ (Theorem 6). The SVNN framework thus defines a **compilable frontier**: a principled boundary separating architectures whose deployed behavior can be certified at design time from those that require runtime testing.

We instantiate SVNN through NeuroPLC, the first IR-based compiler translating PyTorch models to Siemens S7-1200/1500 IEC 61131-3 SCL. Three verification mechanisms compose into an end-to-end guarantee: (i) one-time Z3 SMT proofs that 4/6 IR operation-type templates are correct for all possible parameters; (ii) 512 per-function Z3 proofs assembled into a machine-checkable certificate via a ∼200-line trusted checker; and (iii) sign-structural affine arithmetic achieving a $2.2\times$ tighter error bound than interval arithmetic (safety factor $8.5\times$, up to $3.1\times$ relative to the $\sqrt{d}$ random-weights baseline). KAN's decomposition is *structural necessity*: identically-sized KAN vs. MLP yield 512/512 vs. 0/16 Z3-verifiable components. The framework extends to ChebyKAN (Chebyshev polynomial basis, 496/512 Z3-verifiable via polynomial NRA, Proposition 2). A 3-layer KAN achieves 608/608 Z3 verification (100%), NeuroPLC's DA bound is $85\times$ tighter than CROWN-IBP, and generated SCL compiles to **0 errors, 0 warnings** in TIA Portal V21 (45.2 KB, 90.4% of S7-1200 budget). Validated on CWRU (99.93%), XJTU-SY run-to-failure (91.7% fine-tuned, 512/512 Z3 preserved), and MNIST (98.6%).

Index Terms

Kolmogorov-Arnold Networks, IEC 61131-3, Structured Text (SCL), Neural Network Compiler, Structural Verifiability, Affine Arithmetic, SMT Verification, Compositional Certification, Chebyshev Polynomial Networks, Generalization Bounds.

I. Introduction

A neural network achieving 99.93% fault diagnosis accuracy on CWRU is—in isolation—not deployable on the PLC that governs the machinery it monitors. The reasons are structural, not accidental: PLCs execute IEC 61131-3 Structured Control Language (SCL) in deterministic scan cycles, not PyTorch; their work memory budgets (50 KB for a Siemens S7-1200 CPU 1211C) preclude runtime libraries that ML inference frameworks require; and—the gap this paper addresses—no existing tool can certify that a compiled neural network preserves its trained behavior within the fixed-point REAL arithmetic of industrial controllers. Existing ML→PLC tools (RTNNIgen [1], MLconverter [2], ICSML [3]) provide only empirical test-set validation; the sole prior KAN-on-PLC demonstration (Corrêa [4], USP 2024) relied on distributed PC-to-PLC communication via Snap7 rather than native SCL compilation, and provided no correctness guarantees. General-purpose compilers (TVM, ONNX Runtime) cannot target IEC 61131-3. Four independent lines of work converge on the same observation—KAN's univariate B-spline structure is inherently amenable to LUT-based hardware evaluation—spanning FPGA (KANELÉ [5], ISFPGA 2026 Best Paper), MCU (LUT-KAN [6]), TinyML (KAN-SoC [7]), and formal verification (Schwartz et al. [8]). *None* of these provide an end-to-end PyTorch-to-IEC 61131-3 compiler with design-time certification—the combination this paper delivers.

The *structural insight* of this paper is that the certification gap is not a compiler engineering problem—it is an *architectural* question. Whether a compiler can compute a design-time correctness guarantee depends on whether the network's architecture decomposes into operations that are closed under error propagation. We formalize this via the **Structurally Verifiable Neural Network (SVNN)** framework (§III): two structural conditions (operation-type closure, univariate boundedness) are sufficient for a compiler to compute a finite, parameter-computable per-output error bound for all inputs in the validated domain (Theorem 2). Kolmogorov-Arnold Networks (KANs) satisfy these conditions; standard MLPs with SiLU/Sigmoid/Tanh activations provably do not (Proposition 1). SVNN thus defines a **compilable frontier**: a principled boundary separating architectures whose deployed behavior can be certified at design time from those that require runtime testing.

We implement this framework in NeuroPLC, the first IR-based compiler translating PyTorch KAN to Siemens S7-1200/1500 SCL. The compiler is the *proof of concept* for SVNN—not its primary contribution. Three verification mechanisms establish the guarantee: (1) Z3 SMT compiler template verification (one-time, 4/6 IR op types proved); (2) compositional certificates (512 per-function proofs → ∼200-line end-to-end trusted checker); and (3) sign-structural affine arithmetic ($2.2\times$ tighter bound than interval arithmetic; up to $3.1\times$ vs. $\sqrt{d}$ random-weights baseline, §IV-B9). KAN's B-spline decomposition is *structural necessity*: an identically-sized MLP $[28, 16, 4]$ has 0/16 Z3-verifiable components vs. KAN's 512/512.

Recent work on Kolmogorov-Arnold Networks [9] has opened new possibilities for PLC deployment. KANs replace fixed activation functions (ReLU) with learnable B-spline functions placed on network edges—and this structural difference is the key to verifiability. B-spline activations are inherently piecewise-polynomial, enabling deterministic lookup-table (LUT) compilation with guaranteed error bounds ($\varepsilon \leq M_2 \Delta^2/8$). Each B-spline edge compiles to as little as 68 bytes of LUT storage [6], achieving 7–25× inference speedup on resource-constrained hardware.

Prior ML→PLC tools established that compilation is *possible* but did not address whether it can be *certified*. RTNNIgen [1] converts Keras models to IEC 61131-3 Structured Text; MLconverter [2] targets scikit-learn for ABB controllers. Both provide only empirical test-set validation—a correctness model inherited from ML benchmarking that is insufficient for safety-critical PLC deployment. The gap these tools leave is not an implementation gap ("they didn't try hard enough")—it is an *architectural* gap. Our central finding is that no amount of compiler engineering can close it for architectures that violate the SVNN conditions (Theorem 5). The SVNN framework explains *why* prior tools did not provide guarantees—they were targeting architectures that structurally preclude them. NeuroPLC is the first compiler that compiles trained PyTorch models to IEC 61131-3 SCL for Siemens PLCs *and* provides a formalism for when such compilation can be certified.

Scope. NeuroPLC compiles the classifier (KAN/MLP) to SCL. Frequency-domain features are assumed pre-extracted via DSP and transmitted over PROFINET/OPC UA. An on-PLC time-domain subset is also compiled (E51). This paper contributes:

1) **Theory (Structural Verifiability):** We introduce the **SVNN** framework (§III), which formalizes the architectural conditions under which a neural network compiler can provide design-time correctness guarantees. Two sufficient conditions (operation-type closure, univariate boundedness) guarantee a finite, parameter-computable a priori error bound for any finite-depth network; a third contractivity refinement sharpens this to a depth-independent bound (**Theorem 2**). **Proposition 1** establishes that standard MLPs (ReLU, Sigmoid, Tanh) do *not* satisfy these conditions. The framework reframes the compiler design question from "can we verify an arbitrary architecture?" to "which architectures are inherently verifiable?"—a shift with implications beyond the PLC domain.
2) **System (Compiler):** NeuroPLC, the first PyTorch-to-Siemens SCL compiler, uses a 6-op IR matching SVNN decomposition of KAN (§IV-B1). Generated SCL compiles with **0e 0w** in TIA Portal V21 on S7-1200 and S7-1500 (§IV-D).
3) **Algorithm (Verification & Optimization):** Three novel algorithms: (i) **sign-structural affine arithmetic (DA)**, achieving a 2.2× tighter error bound than interval arithmetic (3.1× vs. the $\sqrt{d}$ random-weights baseline) by preserving weight-matrix sign structure (§IV-B9); (ii) **Segment-Aware de Boor Bounds**, exploiting the piecewise-linear structure of cubic B-spline second derivatives for 6.0× per-segment tightening (composing with DA for a combined safety factor of $\sim 11.9\times$); and (iii) **Adaptive Mixed-Precision LUT Allocation**, a greedy max-heap algorithm reducing worst-case LUT error by 71.6% at equal storage, with **Theorem 3** proving its minimax optimality via an exchange argument (§IV-B11).
4) **Theory (Completeness):** Beyond sufficiency, we establish **computational necessity** (Theorem 5, §III-G): violating Condition 1 makes tight bound computation NP-hard, proving the SVNN conditions cannot be relaxed. Condition 3 yields a depth-improving **generalization bound** $\Delta_{\text{gen}} \leq O(\gamma^L/\sqrt{n})$ (Theorem 6, §III-H). The framework is extended to **ChebyKAN** (Chebyshev polynomial KANs, Proposition 2): 496/512 Z3-verifiable via polynomial NRA, 100.0% CWRU accuracy. **E56** (3-layer KAN: 608/608 Z3, 99.60% acc) and **E57** (NeuroPLC DA 85× tighter than CROWN-IBP) provide empirical depth-scaling and verifier-comparison evidence.
5) **Theory (Correctness): Theorem 1** (§IV) guarantees by structural induction on the IR graph that generated SCL preserves PyTorch semantics to within a computable, architecture-aware error bound for all 6 IR operation types. **Theorem 4** generalizes the guarantee to L-layer KANs via martingale concentration, proving error grows only linearly with depth under the random-sign model.

We demonstrate the pipeline through a bearing fault diagnosis case study: a 6,148-parameter KAN student distilled via VRM-KD [10] from a 48K-parameter 1D-CNN teacher achieves 99.93% CWRU test accuracy with 7.9× compression. Fine-tuning on XJTU-SY run-to-failure data (natural bearing degradation, 2020) reaches 91.7% accuracy with 512/512 Z3 verification preserved post-adaptation. In total, 64 experiments (E1–E57 + V1–V7) spanning compression, generalization, correctness bounds, scalability, cross-dataset transfer, cross-architecture verification, and adversarial robustness validate the compiler. We further discuss the SVNN framework's relationship to emerging AI functional safety standards (ISO/IEC TS 22440) and formal verification tools for IEC 61131-3 (ESBMC-PLC+), positioning NeuroPLC within the broader industrial verification ecosystem.

II. Related Work

A. Deep Learning for Bearing Fault Diagnosis

Three lines of work have pushed CWRU fault diagnosis accuracy above 99%. WDCNN [12] demonstrated that wide first-layer convolutional kernels capture the long-period fault signatures that narrow kernels miss. Zhang et al. [13] extended residual architectures to 1-D signals, confirming that depth helps when the receptive field is sufficiently wide. Together, these works established a near-solved benchmark. But Rauber et al. [14] showed that many reported near-100% results contain train/test leakage from overlapping recording-level windows—the problem is better solved than the literature reports, yet the deployment problem is worse: all methods assume GPU-equipped industrial PCs. **The gap:** 99.9% accuracy on a lab GPU contributes nothing if the inference cannot run on the PLC that governs the monitored machinery.

B. Kolmogorov-Arnold Networks

Liu et al. [9] introduced KANs by replacing MLPs' fixed activations with learnable univariate B-splines—a structural change whose implications for compiler design have been underestimated. **What the literature has established:** Rigas et al. [15], CNN-1D-KAN [16], and AdaptPolyKAN [17] independently confirmed that KANs match or exceed MLP accuracy on bearing fault diagnosis while using far fewer parameters. **What the literature has not addressed:** none of these works consider whether the B-spline decomposition that makes KANs parameter-efficient also makes them *verifiable at compile time*—the question this paper answers.

KAN hardware deployment. Four independent lines of work demonstrate that KAN's univariate B-spline structure is uniquely amenable to LUT-based hardware evaluation—a property absent from MLPs. On FPGA, KANELÉ [5] (ISFPGA 2026 Best Paper) achieves 2,700× speedup via LUT-based evaluation with quantized B-splines. On microcontrollers, LUT-KAN [6] replaces Cox-de Boor recursion with precomputed LUT operations (7–25× speedup, as little as 68 bytes/edge); KAN-SoC [7] achieved 10× memory reduction vs. MLP (693 bytes vs. 6,807 bytes) for EV battery estimation. On PLCs, Corrêa [4] (USP 2024) provided the sole prior KAN-on-PLC demonstration: a KAN $[7, 5, 6]$ deployed via Snap7 distributed execution (PC computes inference, PLC receives results over Ethernet), achieving 88.5% accuracy on a bearing classification task.

These four works collectively establish that KAN's B-spline structure compiles to LUT primitives across hardware platforms. But *none* answer the question this paper asks: can the compilation itself be *certified*? KANELÉ and LUT-KAN validate the LUT paradigm empirically; we formalize *why* it works (Theorem 2) and *when* it fails (Theorem 5, Proposition 1). Corrêa demonstrates KAN-on-PLC feasibility but provides no native SCL compilation, no correctness guarantee, and no compiler toolchain. NeuroPLC is the first to provide all three.

KAN verification. Schwartz et al. [8] proposed PWA abstraction with MILP encoding for KAN verification—operating at SVNN Level 3 (exact certification within PWA abstraction error). Tankman [18] proved layer-wise Lipschitz $P(\text{KAN}) \leq 1$ as a structural property. KAN4CBC [19] demonstrated SMT-based safe controller synthesis. GloroKAN [20] and PolyKAN [21] provide complementary robustness and compression. These are synergistic with NeuroPLC: Schwartz et al.'s MILP verification can be applied to the KAN compiled by NeuroPLC for SIL 3+ certification, while NeuroPLC's Level 2 arithmetic bounds provide fast, no-solver-required pre-certification for CI/CD integration. Tankman's Lipschitz result provides independent theoretical support for Condition 3 (contractivity).

a) FPGA Deployment.: KANELÉ [5] (ISFPGA 2026 Best Paper) maps KAN inference to LUT-based FPGA evaluation with 2,700× speedup over prior FPGA KAN implementations. KANELÉ's LUT paradigm independently validates the same design principle NeuroPLC applies to PLCs—B-spline activations are inherently discretizable across hardware platforms. No prior work compiles KANs to IEC 61131-3 SCL for PLC execution.

C. Knowledge Distillation

Hinton et al. [22] introduced temperature-scaled knowledge distillation as a mechanism for transferring dark knowledge from ensembles to compact models. VRM [10] (ICCV 2025) revitalized relation-based distillation through virtual relation matching with dynamic affinity graph pruning—directly applicable to KAN students because KAN's univariate activation structure produces naturally sparse inter-sample relations. Ren et al. [23] applied multi-stage pruning-distillation for industrial fault diagnosis, establishing the practical viability of compressed models for bearing monitoring. We contribute the first VRM-KD to a KAN student: the distillation is the *training mechanism*, not the contribution; the contribution is that KAN's 7.9× compression does not compromise verifiability.

D. Neural Network Compilation for Industrial Controllers

RTNNIgen [1] (IECON 2024) converts Keras models to IEC 61131-3 ST and demonstrated real-time inference on Beckhoff PLCs—the strongest evidence to date that ML inference *can* run natively on PLCs. MLconverter [2] extends this paradigm to ABB controllers from scikit-learn. ICSML [3] provides native IEC 61131-3 ML components. The shared limitation is the correctness model: all three validate through test-set accuracy, inheriting the empirical validation paradigm of ML benchmarking. For PLCs governing safety-critical machinery, this is the wrong correctness model—it provides no guarantee on unseen inputs, cannot detect floating-point drift, and offers no evidence auditable by a safety assessor. NeuroPLC does not merely add another target platform to this lineage; it changes the correctness model from empirical to *mathematical*, by restricting the compiler's input to architectures that admit the SVNN structural induction (Theorem 2). No automated compiler translates trained PyTorch networks to Siemens SCL with design-time correctness certification.

E. Formal Verification of IEC 61131-3 Programs

ESBMC-PLC+ [24] provides the first open-source unified verification framework for IEC 61131-3 (LD, ST, SCL) through SMT-based BMC with k-induction. PLCreX [25] translates ST/FBD to synchronous models; PyLC+ [26] compiles PLCopen XML to verifiable Python. These verify *logical* properties (mutual exclusion, invariants) orthogonal to NeuroPLC's *numerical* correctness guarantees. The two are composable: NeuroPLC-generated SCL can serve as input to these tools (§VI-B), enabled by the deterministic, human-readable SCL structure (§IV-D).

F. Neural Network Verification and Bounding

A mature literature on neural-network verification provides complementary tools that operate at different points on the soundness–cost spectrum. **Linear-relaxation methods**—CROWN and DeepPoly [27], [28], generalized by α-β-CROWN [29] to branch-and-bound—propagate linear upper/lower bounds through each layer and achieve tight certification on ReLU networks, but the LP size scales with the network, requiring an external solver at verification time. **Complete SMT-based verifiers**—Reluplex [30] and Marabou [31]—offer exact certification for piecewise-linear (ReLU) networks via case-splitting on activation patterns, at exponential cost in the number of ReLU units. **Abstract interpretation** [32], [33] provides sound, scalable over-approximation through abstract domains (zonotopes, octagons) but is generally too coarse for logit-level guarantees on architectures with high-dimensional intermediate representations.

NeuroPLC's SVNN framework occupies a distinct point: it provides *compiler-computed*, architecture-aware bounds at design time, without invoking an LP solver or SMT at deployment. Key differences: (1) the bound depends on the compiled SCL code and its LUT approximation, not on the original floating-point model; (2) Condition 1 (operation-type closure) is a *structural* requirement that linear-relaxation and abstract-interpretation tools do not need, because those tools operate directly on the non-decomposed network; (3) SVNN bounds are deterministic and parameter-computable—the compiler certifies the *translation into SCL*, not the model's robustness. These are complementary: CROWN/DeepPoly verify the original model; SVNN certifies the *compilation* of that model. The most closely related approach is the per-function Z3 certificate bundle (§VI-B), which operates at the sub-module level and composes into an end-to-end machine-checkable guarantee.

Competitive Positioning. The tools surveyed above represent three distinct approaches to ML-on-PLC deployment, none of which provides design-time correctness certificates. *Domain-specific compilers* (RTNNIgen, MLconverter, ICSML) target narrow model classes for specific PLC vendors but offer only empirical validation. *General-purpose inference engines* (TFLite, ONNX Runtime, TVM) target performance on GPUs and MCUs; they cannot generate IEC 61131-3 and their runtime libraries alone exceed PLC memory budgets by factors of 10^0–10^2. *Formal verification tools* (CROWN/DeepPoly, Marabou, Reluplex) verify the *original* model's robustness but do not compile it to PLC code or certify the *compilation* step. NeuroPLC occupies a distinct point: it is not a verification tool, a general-purpose compiler, or a domain-specific converter—it is a **correctness-preserving compiler** whose design is driven by the architectural insight (SVNN) that compilability is a property of the model architecture, not the compiler engineering. This reframing—from "how do we verify an arbitrary architecture?" to "which architectures are inherently verifiable?"—is the paper's primary contribution, with implications beyond PLC deployment to any safety-critical embedded ML target.

III. Structural Verifiability of Neural Networks

We now formalize a central observation that emerged from the compiler design process: whether a neural network compiler can provide design-time correctness guarantees is not solely a property of the compiler—it is fundamentally determined by the *architecture* being compiled. This section defines the class of architectures for which correctness-preserving compilation is achievable, proves that KAN belongs to this class, and demonstrates that standard MLPs do not.

Remark 1 (The Compilable Frontier)**.** *The SVNN conditions define a **compilable frontier**—a principled boundary separating neural architectures whose deployed behavior can be certified at design time from those that require runtime testing. Architectures satisfying Conditions 1–2 lie on the interior (certifiable side): their decomposition into pure linear and element-wise operations enables the structural induction that Theorem 2's proof relies on. Architectures violating any condition lie on the exterior (empirical-only side): they can still be compiled and deployed, but their correctness guarantees are limited to test-set validation. Proposition 1 (below) establishes that standard MLPs with SiLU, Sigmoid, or Tanh activations—the dominant architectures in industrial ML—lie on the exterior. This frontier is not an implementation artifact; it reflects a fundamental property of which function classes admit closed-form, compiler-computable error propagation.*

A. The Verification Challenge for PLC-Deployed Networks

Industrial PLCs govern safety-critical machinery: conveyor sorting systems, motor drives, and process control loops where incorrect outputs can cause equipment damage or production line stoppage. Deploying neural networks on such controllers therefore demands correctness guarantees that go beyond empirical test-set accuracy.

Existing ML→PLC tools (RTNNIgen [1], MLconverter [2], ICSML [3]) provide *no* mathematical guarantee that the generated PLC code preserves the trained model's behavior. Their correctness model is purely empirical: train, convert, and test on a held-out set. For industrial deployment—where inputs may drift, sensors may degrade, and the PLC's REAL arithmetic differs from PyTorch's float32—this is insufficient.

We identify three levels of deployment verification, in increasing order of strength:

1) **Level 1—Runtime Testing (Weakest):** Evaluate the deployed model on a test set and report accuracy. This is the status quo (RTNNIgen, MLconverter). It provides no guarantee on unseen inputs and cannot detect systematic floating-point discrepancies.

2) **Level 2—Design-Time Error Bounds (SVNN):** Compute, at *compile time* and without executing on hardware, a provable per-output error bound ε_i such that for all valid inputs x, the deployed model's output differs from the trained model's output by at most ε_i. This is the guarantee that NeuroPLC provides (Theorem 1).
3) **Level 3—Mechanized Formal Verification (Strongest):** A machine-checked proof (e.g., in Coq, Isabelle, or Lean) that the compiler preserves semantics down to the IEEE 754 bit-level. This has been achieved for select neural network kernels in TorchLean [34] but is currently intractable for full compiler pipelines.

Level 2—design-time error bounds—is the appropriate level for industrial deployment. The question is: *which architectures admit such bounds, and why?*

B. Definition: Structurally Verifiable Neural Network (SVNN)

Definition 1 (Structurally Verifiable Neural Network)**.** *A neural network architecture $\mathcal{A}$ is **structurally verifiable** (SVNN) with respect to a compiler C if for any instance $M \in \mathcal{A}$, the compiler can compute, at design time and without executing M on target hardware, a per-output error bound $\varepsilon_i(M, X)$ such that:*

$$\sup_{x \in X} \|M(x)_i - C(M)(x)_i\|_\infty \leq \varepsilon_i(M, X) \quad \forall i \in \{1, \ldots, d_{out}\} \tag{1}$$

where $X \subset \mathbb{R}^{d_{in}}$ is the validated input domain and $C(M)$ denotes the compiler's generated code executing on the target platform's arithmetic (here, IEC 61131-3 REAL $\equiv$ IEEE 754 binary32).

This definition captures the essential requirement: the compiler must provide an *a priori* guarantee—before the model ever runs on the PLC—that the deployed code will not silently diverge from the trained model. The error bound ε_i may depend on the model's parameters, the compiler's approximation choices (e.g., LUT resolution), and the input domain, but it must be computable from these quantities alone.

C. Sufficient Conditions for SVNN

We now establish the architectural conditions that enable SVNN compilation. The conditions formalize why some architectures admit tight, composable error bounds while others do not.

Theorem 2 (SVNN Sufficient Conditions). Let $\mathcal{A}$ be a feedforward neural network architecture. If $\mathcal{A}$ satisfies Conditions 1 and 2 below, then $\mathcal{A}$ is structurally verifiable—for any instance $M \in \mathcal{A}$ of finite depth L, a compiler adhering to the SVNN compilation discipline can compute, from the model parameters alone, a finite per-output error bound. If $\mathcal{A}$ additionally satisfies the contractivity refinement (Condition 3), this bound admits the closed, *depth-independent* geometric-series form $O(M_{\max} \cdot h^2 \cdot d_{\max}/(1-\gamma))$, where $M_{\max}$ is the maximum second-derivative bound, h is the LUT grid spacing, $d_{\max}$ the maximum activation count per layer, and $\gamma < 1$ the contractivity constant; absent Condition 3, the bound remains finite for every fixed L but may grow with depth.

The conditions are designed to *discriminate*: a standard MLP layer $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$ composes an affine transform and a pointwise activation in one coupled operation, violating Condition 1. This prevents per-operation error induction (the proof's structural induction requires each node to be either purely linear or purely element-wise). MLPs with SiLU or GELU additionally violate the Z3-verifiability of Condition 2 (the transcendental $\exp$ is undecidable in Z3's NRA theory); MLPs with ReLU lack a C^2 curvature parameter. KAN layers, in contrast, decompose naturally at the IR level into separate MatMul, BsplineLUT, StandardAct, and Add nodes—each satisfies one of the two pure types. Conditions 1–2 thus characterize exactly the class that admits the induction, and KAN belongs to it; standard MLPs do not.

1) **Operation-Type Closure (Condition 1).** The architecture can be decomposed into a directed acyclic graph where every node is either (a) a linear transformation (matrix multiplication, vector addition) or (b) an element-wise univariate function. No node may mix these operation types. This decomposition is provided by the compiler's IR (§IV-B1); it guarantees that the per-node error induction in the proof below never encounters a coupled linear+nonlinear operation.
2) **Univariate Curvature Bound (Condition 2).** Every element-wise univariate function $f : \mathbb{R} \to \mathbb{R}$ is C^2-continuous on the validated input domain X, and its second derivative satisfies $\sup_{x \in X} |f''(x)| \leq M_f < \infty$. The bound M_f must be computable from the function's parameters alone. This enables the de Boor LUT error bound $\varepsilon_f = M_f h^2/8$—a *compiler-controllable* error: by increasing the LUT resolution (reducing h), the per-function error decreases quadratically. Condition 2 thus provides the compiler with a tunable guarantee knob that MLPs with ReLU (no C^2) or SiLU/GELU (transcendental, Z3-undecidable) lack.

Conditions 1 and 2 are the *sufficient conditions* for structural verifiability. The following refinement governs how the bound scales, not whether it exists:

3) **Contractive Composition (Condition 3, refinement).** The product of each element-wise layer's Lipschitz constant $L_f^{(\ell)} = \sup_x |f'(x)|$ with the subsequent linear layer's row-sum norm satisfies $L_f^{(\ell)} \cdot \|W^{(\ell+1)}\|_{1,\infty} \leq \gamma < 1$. When this holds, the bound converges to a depth-independent constant $O(1/(1-\gamma))$; when it does not, the bound remains finite for

the specific deployment depth but grows as $O(\gamma^L)$. Condition 3 upgrades a per-depth certificate to a depth-independent one; it is *not* required for SVNN membership.

Proof. The proof proceeds in four steps.

Step 1 (Linear Layer Exactness). Let $y = Wx+b$ be a linear transformation with input x that carries error δ_x (i.e., the compiled input is $\tilde{x} = x+\delta_x$). The compiled output is $\tilde{y} = W\tilde{x}+b = Wx+b+W\delta_x$. Hence $\|\tilde{y}-y\|_\infty = \|W\delta_x\|_\infty \le \|W\|_{1,\infty}\cdot\|\delta_x\|_\infty$. A linear layer introduces *no new error sources*—it only amplifies or attenuates existing error. Under sign-structural affine arithmetic (§IV-B9), the amplification factor exploits weight-matrix sign structure, yielding $\|W\delta_x\|_\infty \le \|W^+\delta_x^+ + W^-\delta_x^-\|_\infty$, which is strictly tighter than the interval-arithmetic bound $\|W\|_{1,\infty}\cdot\|\delta_x\|_\infty$ when W contains both positive and negative entries.

Step 2 (Element-Wise Boundedness). Consider an element-wise univariate function f applied to input x. The compiled version uses a piecewise-linear LUT approximation $\tilde{f}$ on a grid with spacing h. By the de Boor–Cox recurrence for B-spline approximation error [35], for any C^2 function:

$$\|f-\tilde{f}\|_\infty \le \frac{M_f\cdot h^2}{8} \tag{2}$$

where $M_f = \sup_x |f''(x)|$. Condition 2 guarantees that M_f is finite and computable. The LUT thus introduces fresh error at most $\varepsilon_f = M_f h^2/8$ per function evaluation.

If the input carries error δ_x, the mean value theorem gives:

$$|f(x+\delta_x)-\tilde{f}(x+\delta_x)| \le |f(x+\delta_x)-f(x)| + |f(x)-\tilde{f}(x+\delta_x)| \le L_f\cdot|\delta_x| + \varepsilon_f \tag{3}$$

where $L_f = \sup_x |f'(x)|$ is the Lipschitz constant of f.

Step 3 (Finite-Depth Composition—Conditions 1 and 2 suffice). We now compose the analysis across L layers by structural induction on the layer index ℓ. Let $\Delta^{(\ell)}$ denote the maximum per-output error after layer ℓ, with $\Delta^{(0)} = 0$ (input is exact).

For a KAN layer (element-wise B-spline activation followed by a linear combination via weight matrix):

$$\Delta^{(\ell)} \le L_f^{(\ell)}\cdot\|W^{(\ell)}\|_{1,\infty}\cdot\Delta^{(\ell-1)} + \varepsilon_f^{(\ell)}\cdot d_{\ell-1} \tag{4}$$

The first term captures error amplification from the previous layer; the second term captures fresh approximation error from $d_{\ell-1}$ parallel element-wise function evaluations (one per input dimension).

Unrolling the recurrence across L layers yields the total bound:

$$\Delta^{(L)} \le \sum_{\ell=1}^{L}\varepsilon_f^{(\ell)}\cdot d_{\ell-1}\cdot\prod_{j=\ell+1}^{L}\left(L_f^{(j)}\cdot\|W^{(j)}\|_{1,\infty}\right) \tag{5}$$

The critical role of Condition 1 is now apparent: it guarantees that every node encountered by the induction is of a known type with a closed-form error rule (Step 1 for linear nodes, Step 2 for element-wise nodes). The recurrence Eq. 4 applies *only* because each node is provably pure. In a standard MLP layer $\mathbf{h} = \sigma(W\mathbf{x}+\mathbf{b})$, the affine transform and pointwise activation are coupled in a single operation—there is no intermediate variable whose error can be separately bounded, so the structural induction cannot be initiated. This is why Sigmoid/Tanh MLPs, despite having bounded M_2 (they satisfy Condition 2), are not SVNN: they fail Condition 1.

Under Conditions 1 and 2, every term in Eq. 5 is finite and computable: $\varepsilon_f^{(\ell)} = M_f^{(\ell)}h^2/8$ from Condition 2, $L_f^{(j)}$ finite on the compact domain X, and $\|W^{(j)}\|_{1,\infty}$ finite by construction. Eq. 5 is therefore a finite sum of finite terms, directly yielding a computable design-time bound $\Delta^{(L)} < \infty$ for any fixed depth L—the existence guarantee of Definition 1. The bound is *compiler-controllable*: increasing the LUT resolution (reducing h) monotonically decreases ε_f and hence $\Delta^{(L)}$, giving the compiler a tunable knob to meet any target safety requirement.

Step 3b (Depth-Uniform Refinement—Condition 3). Condition 3 does not change *whether* a bound exists; it controls *how the bound scales with depth*. When $L_f^{(j)}\cdot\|W^{(j)}\|_{1,\infty} \le \gamma < 1$, the product terms in Eq. 5 decay geometrically, and the finite sum is majorized by the convergent series

$$\Delta^{(L)} \le \frac{\max_\ell(\varepsilon_f^{(\ell)}\cdot d_{\ell-1})}{1-\gamma} = O\left(\frac{M_{\max}\cdot h^2\cdot d_{\max}}{1-\gamma}\right) \tag{6}$$

where $M_{\max} = \max_f M_f$ and $d_{\max} = \max_\ell d_{\ell-1}$ is the maximum number of element-wise activation functions per layer. The bound follows by substituting $\varepsilon_f^{(\ell)} = M_f^{(\ell)}h^2/8$ (Eq. 2) into the geometric series, yielding a per-layer factor of $M_{\max}h^2 d_{\max}$ multiplied by the geometric decay $1/(1-\gamma)$. This form is *depth-independent*: when $\gamma < 1$, the bound converges to a constant as $L \to \infty$, admitting deployment on arbitrarily deep networks. When Condition 3 does not hold ($\gamma \ge 1$), Eq. 5 still evaluates to a finite number for the fixed deployment depth, but the worst-case majorant grows as $O(M_{\max}\cdot h^2\cdot\gamma^L)$—still computable, only looser with depth. Thus Condition 3 upgrades a per-depth guarantee into a depth-uniform one; it is a refinement, not a membership requirement.

Step 4 (Tightness and Practical Considerations). The $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$ bound is a *worst-case* guarantee. In practice, three factors make the bound substantially tighter for the KAN architecture specifically:

1) **Segment-Aware M_2 (§IV-B10):** Rather than using the global $\sup|f''|$, we compute per-segment $M_2^{(k)}$ bounds, tightening the LUT error estimate by $6.0\times$ on average for KAN B-splines.
2) **sign-structural affine arithmetic (§IV-B9):** Linear layers propagate error via DA rather than IA, exploiting weight-matrix sign structure for an additional $3.1\times$ tightening.
3) **Adaptive LUT Allocation (§IV-B11):** Non-uniform knot placement allocates more LUT points to high-curvature regions, reducing the effective h for the dominant error sources by up to 71.6%.

Together, these refinements transform the asymptotic $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$ bound into a practically useful guarantee. For the trained $[28, 16, 4]$ KAN, the combined segment-aware DA safety factor is approximately $11.9\times$ above the minimum required for classification correctness (§IV, Table XXIII).

Remark 2 (Near-Necessity of the SVNN Conditions)**.** *While Theorem 2 establishes sufficiency, the question of necessity is equally important: are these conditions arbitrarily chosen, or do they characterize a genuine structural boundary? Theorem 5 (§III-G) provides the formal answer: violating Condition 1 makes tight bound computation NP-hard, establishing that the SVNN conditions are not an arbitrary restriction but the precise boundary at which polynomial-time compiler-computed correctness becomes achievable. We briefly summarize the three observations that motivate the formal proof.*

(a) Condition 1 is structurally necessary for polynomial-time bound computation. Any architecture that couples linear transforms with nonlinear activations in a single operation—the standard MLP layer $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$—eliminates the intermediate representation whose error the structural induction bounds. Without this decomposition, error propagation must model the coupled operation as a single black-box function, requiring general-purpose LP/SMT solvers (CROWN, Marabou, Reluplex) whose cost grows with network size. Condition 1 thus separates architectures admitting $O(L \cdot d_{\max}^2)$ compiler-computed bounds from those requiring exponential-time SMT case-splitting.

(b) Condition 2 separates C^2-bounded from transcendental. The de Boor LUT error bound ($\varepsilon = M_2 h^2/8$) requires C^2 continuity. Activations without C^2 (ReLU: f'' undefined at 0) or with unbounded curvature (SiLU: $\sup|\mathit{SiLU}''| = \infty$ on $\mathbb{R}$) cannot use this bound. The C^2 requirement is nearly tight: it excludes exactly the activations whose LUT error cannot be bounded by a finite, parameter-computable constant. On the compact input domain $[-3, 3]$[28], SiLU's second derivative is finite ($\max \approx 0.21$)—but the key distinction is computability: $M_2^{\text{B-spline}}$ is computable from the learned coefficients alone, whereas $\sup_{x\in[-3,3]} |\mathit{SiLU}''(x)|$ requires numerical optimization over a transcendental function—a different computational model.

(c) Richardson's Theorem [36] imposes a fundamental barrier. The equality of expressions involving e^x is undecidable by Richardson's theorem. Since SiLU contains e^{-x}, determining whether $\mathit{SiLU}_{LUT}(x) = \mathit{SiLU}_{FP32}(x)$ for a given LUT resolution is, in general, undecidable. KAN's B-spline activations are piecewise-polynomial and avoid this barrier entirely. The SVNN conditions are therefore not an arbitrary restriction—they characterize the precise boundary at which compiler-computed correctness becomes decidable.

Remark 2 (Relationship to Theorem 1). Theorem 1 (§IV) is the *concrete instantiation* of the SVNN framework for a specific architecture (2-layer KAN) and compiler (NeuroPLC). Theorem 2 provides the *general principle*: any finite-depth architecture satisfying Conditions 1–2 admits a correctness-preserving compiler, with Condition 3 sharpening the resulting bound to a depth-uniform form. Theorem 1 proves that KAN satisfies the conditions; Theorem 2 shows *why* this is possible and characterizes the broader class of architectures for which similar guarantees can be obtained.

Remark 3 (Arithmetic Bounds vs. MILP-Based Verification). The SVNN arithmetic bound ($O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$, Theorem 2) and the PWA abstraction + MILP approach of Schwartz et al. [8] represent two complementary points on the verification cost-precision spectrum. *Arithmetic bounds* (Level 2) are computable in closed form at compile time, require no external solver, and provide a sound over-approximation for all inputs in the validated domain—but they are inherently conservative (the $\sim$11.9$\times$ safety factor for our trained KAN reflects this conservatism). *MILP-based verification* (Level 3) can provide exact certification within the PWA abstraction error (typically $<$0.1% of the function range per unit), but requires solving an MILP whose size grows with the number of PWA pieces—tractable for KANs (each univariate function is abstracted independently) but exponential for MLPs (multivariate activations require piecewise decomposition of $\mathbb{R}^d$). The two approaches are synergistic: arithmetic bounds serve as a fast, no-solver-required pre-certificate; if the bound is within safety margins, no further verification is needed. If tighter guarantees are required (e.g., for SIL 3+ certification), the SVNN decomposition enables targeted MILP verification of safety-critical outputs. This two-tier architecture is discussed further in §VI-B.

Architecture Taxonomy Under the Compilable Frontier. Table I maps major neural architecture families to the SVNN conditions, providing a concrete instantiation of the compilable frontier (Remark 1). The taxonomy reveals a stark divide: architectures that decompose into pure linear + element-wise operations (Condition 1) and use C^2 parameter-computable activations (Condition 2) occupy the *interior*—their deployed behavior is certifiable at design time. All architectures violating either condition lie on the *exterior*—they can be compiled and deployed, but correctness guarantees are limited to test-set

TABLE I
ARCHITECTURE TAXONOMY UNDER THE SVNN COMPILABLE FRONTIER. ARCHITECTURES ON THE INTERIOR ADMIT COMPILER-COMPUTED DESIGN-TIME CORRECTNESS CERTIFICATES; ARCHITECTURES ON THE EXTERIOR REQUIRE RUNTIME TESTING.

Architecture	Cond. 1	Cond. 2	Contract.	Frontier	Z3-Verif.
KAN (B-spline)	✓	✓	✓*	Interior†	512/512
KAN (Chebyshev)	✓	✓	✓	Interior	Full‡
KAN (Wavelet)	✓	✓	∼	Interior	Full‡
MLP + ReLU	✗	✗§	N/A	Exterior	0/16
MLP + SiLU/GELU	✗	✗¶	N/A	Exterior	0/16♯
MLP + Sigmoid	✗	✓	N/A	Exterior	–
MLP + Tanh	✗	✓	N/A	Exterior	–
Transformer	✗	✗	N/A	Exterior	N/A
CNN (Conv2d)	✗	✗	N/A	Exterior	N/A

✓ = condition satisfied; ✗ = condition violated; ∼ = depends on wavelet basis choice.
*Contractivity verified for the trained KAN student $[28, 16, 4]$ under VRM-KD distillation; Tankman [18] proves $P(\text{KAN}) \leq 1$ as a structural property for standard operation sets on $[0, 1]$-bounded inputs.
†The B-spline KAN is the only architecture in this taxonomy for which all three conditions are simultaneously satisfied *and* empirically validated on physical hardware (TIA Portal V21, S7-1200).
‡ChebyKAN: formally verified in Proposition 2 (§III-F); Z3 verifiability rate 496/512, 96.9% (E54). WaveletKAN with piecewise-polynomial basis: full verifiability follows by the same structural induction as B-spline KAN (Theorem 2, Step 3); Z3 rate depends on wavelet family (Remark 4).
§ReLU is piecewise-linear (C^0); f'' is a Dirac delta at $x = 0$, so the de Boor $M_2 h^2/8$ bound does not apply. MILP-based verification (e.g., Reluplex) is possible but requires exponential case-splitting.
¶SiLU contains $\sigma(x) = 1/(1 + e^{-x})$; e^{-x} is transcendental and undecidable in Z3's NRA theory. On compact domain $[-3, 3]$, $\sup|\text{SiLU}''| \approx 0.21$ is finite but requires numerical optimization—not computable from parameters alone.
♯Confirmed empirically: MLP $[28, 16, 4]$ with SiLU, 0/16 components Z3-verifiable (E41).

validation. The Contractivity column (Condition 3) further distinguishes architectures whose error bounds are depth-*uniform* ($\gamma < 1$, $O(L)$ growth) from those whose bounds are merely depth-*finite* (computable but potentially growing with L).

The taxonomy clarifies a structural fact with direct engineering consequences: *no* MLP variant—regardless of activation choice—crosses the compilable frontier. ReLU MLPs violate Condition 2 (no C^2 curvature parameter); SiLU/GELU MLPs violate Condition 2 (transcendental, Z3-undecidable) *and* Condition 1 (coupled linear+nonlinear); Sigmoid/Tanh MLPs satisfy Condition 2 but violate Condition 1. The SVNN conditions together form a **conjunction**—failure of either is sufficient to place an architecture on the exterior. This is why the compiler's ability to provide guarantees is an architectural question, not a compiler engineering question (Remark 1).

D. KAN Satisfies the SVNN Conditions

Proposition 1 (KAN Is Structurally Verifiable)**.** *Every Kolmogorov-Arnold Network (KAN) architecture, as defined by Liu et al. [9], satisfies the SVNN sufficient conditions (Conditions 1 and 2) of Theorem 2 and is therefore an SVNN architecture. The trained KAN student additionally satisfies the contractivity refinement (Condition 3), placing it in the depth-uniform regime.*

Proof. We verify Conditions 1 and 2, which establish SVNN membership, and then confirm the Condition 3 refinement:

Condition 1 (Operation-Type Closure). A KAN layer with d_{in} inputs and d_{out} outputs decomposes naturally as:

$$\text{KANLayer}(x)_j = \underbrace{b(x_j)}_{\text{Base path}} + \sum_{i=1}^{d_{\text{in}}} \underbrace{\phi_{j,i}(x_i)}_{\text{Spline path}} \tag{7}$$

where $b(\cdot)$ is a SiLU activation (element-wise univariate) and each $\phi_{j,i}(\cdot)$ is a cubic B-spline (element-wise univariate). The summation $\sum_{i=1}^{d_{\text{in}}} \phi_{j,i}(x_i)$ is a linear combination, equivalent to a MatMul with an identity-projected weight structure. The final addition $b(x_j) + \sum_i \phi_{j,i}(x_i)$ is element-wise linear. Thus every KAN layer consists solely of element-wise univariate functions and linear combinations—satisfying Condition 1.

This decomposition is exactly the basis of NeuroPLC's IR design: each KAN layer maps to a MatMul node, $d_{\text{in}} \times d_{\text{out}}$ BsplineLUT nodes, and d_{out} Add nodes (§IV-B1). The 6-op IR type system is not an arbitrary design choice—it is the minimal closure required by the SVNN decomposition of KAN.

Condition 2 (Univariate Boundedness). Cubic B-splines ($k = 3$) are C^2-continuous piecewise polynomial functions. On the bounded input domain $X = [-3, 3]$ used by the KAN student, the second derivative $\phi''(x)$ of each B-spline activation is

bounded. The bound $M_2 = \max_x |\phi''(x)|$ is computable directly from the B-spline control points $\{w_c\}$ and knot vector $\{t_c\}$ via the derivative recurrence:

$$\phi''(x) = \sum_{c=0}^{G+k-2} w_c \cdot B''_{c,3}(x) \tag{8}$$

where $B''_{c,3}$ is computed from the B-spline basis derivative recurrence [35]. For the SiLU activation used in the base path, $\text{SiLU}''(x) = \sigma(x)(2 - 2x\sigma(x) + x\sigma(x)(1 - \sigma(x)))$, which is bounded on any compact domain.

Analytical M_2 and L_B Upper Bounds. While $M_2 = \max_x |\phi''(x)|$ can be computed numerically from the B-spline control points (as done in E11, yielding the model-specific value 0.177), we further establish *analytical* upper bounds that depend only on the B-spline degree and knot spacing—providing a true a priori guarantee that requires no forward pass or empirical measurement.

B-spline derivative bounds on uniform knots. For a cubic B-spline ($k = 3$, order 4) defined on uniform knots with spacing h, the basis function derivatives satisfy the following analytical bounds (derived from the Cox-de Boor recurrence [35]):

$$\max_x |B'_{c,4}(x)| \leq \frac{3}{h}, \qquad \max_x |B''_{c,4}(x)| \leq \frac{6}{h^2} \tag{9}$$

For a B-spline curve $\phi(x) = \sum_c w_c B_{c,4}(x)$, at any x at most 4 basis functions are non-zero (local support property). Consequently:

$$\max_x |\phi'(x)| \leq \frac{12 \cdot W_{\max}}{h}, \qquad \max_x |\phi''(x)| \leq \frac{24 \cdot W_{\max}}{h^2} \tag{10}$$

where $W_{\max} = \max_c |w_c|$ is the maximum absolute control-point value. A tighter bound using control-point differences [35] yields:

$$L_B^{\text{analytic}} \leq \frac{3}{h} \cdot \max_c |\Delta w_c|, \qquad M_2^{\text{analytic}} \leq \frac{6}{h^2} \cdot \max_c |\Delta^2 w_c| \tag{11}$$

where $\Delta w_c = w_{c+1} - w_c$ and $\Delta^2 w_c = w_{c+1} - 2w_c + w_{c-1}$ are the first and second forward differences of the control-point sequence.

Application to KAN. For the KAN student $[28, 16, 4]$ with grid size $G = 8$ on input domain $[-3, 3]$, the knot spacing is $h = 6/G = 0.75$. With control points typically bounded by $W_{\max} \approx 0.3$ (enforceable via weight clipping during training), Eq. 10 yields the a priori bounds $L_B^{\text{analytic}} \leq 12 \times 0.3/0.75 = 4.8$ and $M_2^{\text{analytic}} \leq 24 \times 0.3/0.5625 = 12.8$. The tighter difference-based bounds (Eq. 11) reduce these to $L_B^{\text{analytic}} \leq 4 \times \max |\Delta w|$ and $M_2^{\text{analytic}} \leq 10.67 \times \max |\Delta^2 w|$, both computable from the control points alone before any forward pass.

Relationship to empirical values. The empirically measured values ($L_B = 0.65$, $M_2 = 0.177$ from E11) are substantially tighter than the analytical worst-case bounds, because the KAN's learned B-spline control points exhibit slow variation ($\max |\Delta w_c| \ll W_{\max}$). The analytical bounds serve as the **architectural guarantee** (proving that the architecture admits finite M_2 and L_B a priori), while the empirical measurements provide **model-specific refinement** (tightening the bound post-training). Both are computed from the model parameters alone—neither requires hardware execution. This two-tier approach satisfies SVNN Condition 2: M_f is analytically bounded by the architecture and computable from parameters, with the analytical bound providing the design-time certificate and the empirical refinement providing the deployment-time tightening.

Condition 3 (Contractive Composition, refinement). Conditions 1 and 2 already establish that KAN is structurally verifiable (Theorem 2, Step 3): the finite-depth error bound of Eq. 5 exists and is computable from the control points alone. We now show that the trained KAN *additionally* satisfies the contractivity refinement, which upgrades this guarantee to the depth-uniform $O(L)$ form of Eq. 6. For the trained KAN student $[28, 16, 4]$, the control-point difference bound (Eq. 11) yields $L_B^{\text{analytic}} \leq 1.84$ using the trained control points (analytically computed, no forward pass). The tighter empirical measurement gives $L_B = 0.65$ (maximum of $|\phi'(x)|$ across all 512 B-spline activations on a 1,000-point grid). The weight matrix row-sum norms are $\|W^{(0)}\|_{1,\infty} \leq 0.32$ and $\|W^{(1)}\|_{1,\infty} \leq 0.28$. Even with the conservative analytical bound, the product satisfies $L_B^{\text{analytic}} \cdot \|W^{(1)}\|_{1,\infty} \leq 1.84 \times 0.28 = 0.515 < 1$; with the empirical refinement, $L_B \cdot \|W^{(1)}\|_{1,\infty} = 0.65 \times 0.28 = 0.182 \ll 1$. Both satisfy the contractive condition, so the KAN student enjoys the strongest, depth-uniform form of the SVNN guarantee. We emphasize, however, that a KAN with larger weights violating $\gamma < 1$ would *still* be structurally verifiable by Conditions 1–2—it would simply fall back to the finite depth-dependent bound. Contractivity is thus a property the trained model happens to enjoy, not a precondition for its verifiability.

Beyond this empirical observation, Tankman [18] has recently provided a *formal* characterization: for standard KAN operation sets $\{+, -, \times, \sin, \cos\}$ on $[0, 1]$-bounded inputs, the layer-wise Lipschitz product satisfies $P(\text{KAN}) \leq 1$, independent of input dimension n and network depth. This result establishes that the contractivity required by Condition 3 is not merely an artifact of our specific trained model or distillation procedure—it is a *structural property* of KAN architectures under standard operations. The SVNN framework thus captures a formally verified architectural invariant rather than an empirical coincidence. □

E. Standard MLPs Do Not Admit Tight SVNN Error Bounds

Having established that KAN architectures satisfy the SVNN conditions, we now show that the most widely deployed alternative—the Multi-Layer Perceptron (MLP) with standard activations—does not. This result clarifies *why* prior ML→PLC tools provide no correctness guarantees: the architecture itself precludes them.

Proposition 1 (Standard MLPs Do Not Admit the SVNN Tight-Bound Structure). Let $\mathcal{M}$ be a feedforward MLP using ReLU, SiLU, Sigmoid, or Tanh activations. Then $\mathcal{M}$ does not admit the compiler-computable architecture-independent tight LUT error bound of Condition 2, for the following reasons. This does *not* mean MLPs are unverifiable in general (tools such as Reluplex and α-β-CROWN verify ReLU networks)—it means the *KAN-style* de Boor curvature bound, which is computable from parameters alone without knowing the input distribution, does not apply to MLP activations.

Proof. We analyze each activation function:

(a) SiLU / GELU (modern MLP standard): $f(x) = x \cdot \sigma(x)$. SiLU$''(x)$ is bounded and the function is C^∞, so Condition 2 is formally satisfiable. However, SiLU contains $\sigma(x) = 1/(1+e^{-x})$, which uses the transcendental function e^{-x}. Z3's Nonlinear Real Arithmetic (NRA) theory is *undecidable* for transcendental functions, so per-function Z3 verification—the mechanism behind the compositional certificate of §VI-B—yields no proof for any SiLU segment. Empirically: MLP $[28, 16, 4]$ with SiLU activations achieves 0/16 Z3-verifiable components (E41), confirming that SiLU-based MLPs are incompatible with the compositional verification pipeline regardless of bound tightness.

(b) ReLU: $f(x) = \max(0, x)$. ReLU is piecewise-linear with kink at $x = 0$. A LUT whose knots include $x = 0$ would reproduce ReLU exactly ($\varepsilon = 0$), so the *existence* of a valid bound is not the issue. The issue is that the de Boor curvature bound (Eq. 2, $\varepsilon \leq M_2 h^2/8$) requires C^2 continuity, which ReLU violates at $x = 0$ (f'' is a Dirac delta). Without the curvature characterization, the compiler cannot compute an a priori LUT error from the architecture and parameters alone: the error depends on where inputs fall relative to the kink, which is input-distribution-dependent. Existing tools that verify ReLU networks (Reluplex, α-β-CROWN) instead enumerate activation patterns via case-splitting, at exponential cost in the number of ReLU units—not by a single closed-form parameter-computable bound.

(c) Sigmoid / Tanh. Both are C^∞ with bounded M_2 ($\sup_x |\sigma''| \approx 0.096$, $\sup_x |\tanh''| = 2/\sqrt{27} \approx 0.385$), so Condition 2 is satisfiable. The bottleneck is the *mixing* within each MLP layer: each neuron combines linear and nonlinear operations in the same layer, while SVNN Condition 1 requires layers to be *either* purely linear *or* purely element-wise. Standard MLP layers violate Condition 1 (affine transform + pointwise activation in one step), breaking the clean compositional error accounting that makes per-function Z3 verification tractable.

Consequence for MLP compilation. The fundamental issue is not that MLPs cannot be compiled—they can, and NeuroPLC does compile MLPs (§IV). The issue is that the compiler cannot provide a *tight, architecture-aware error bound* for MLPs. Without a per-function M_2 bound (Condition 2 violation for ReLU) or with only coarse global Lipschitz bounds (Sigmoid/Tanh), the best achievable MLP error bound is:

$$\Delta^{(L)}_{\text{MLP}} \leq O\big(L \cdot L_{\text{act}}^{L} \cdot \|\mathbf{W}\|_{\text{prod}}\big) \tag{12}$$

where $L_{\text{act}} = \max(1, L_{\text{ReLU}}, L_\sigma, L_{\tanh})$. For $L_{\text{ReLU}} = 1$, the bound grows as $O(L \cdot \|\mathbf{W}\|_{\text{prod}})$, which can easily exceed 1.0 for networks deeper than 3–4 layers with typical weight magnitudes. This makes the guarantee vacuous for all but the shallowest MLPs.

In contrast, the KAN error bound grows as $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$, which is independent of weight magnitudes (the linear layers are exact in DA) and decreases cubically with LUT resolution h. This architectural distinction—not implementation quality—is why NeuroPLC provides guarantees for KAN that it cannot provide for MLP. □

F. Chebyshev-KAN Satisfies the SVNN Conditions

Having established that B-spline KAN satisfies the SVNN conditions (§III-D) and that standard MLPs do not (§III-E), we now demonstrate that the SVNN framework extends to alternative basis function families. We prove that KAN architectures using *Chebyshev polynomial* basis functions—a globally-supported, orthogonal polynomial basis—also satisfy Conditions 1 and 2, establishing that SVNN membership is not tied to the local-support property of B-splines but rather to the structural decomposition and curvature computability that both bases provide.

Proposition 2 (Chebyshev-KAN Is Structurally Verifiable)**.** *A KAN architecture using Chebyshev polynomial basis functions (ChebyKAN) [37], defined as*

$$\phi_{j,i}(x) = \sum_{n=0}^{N} c_{j,i,n} \cdot T_n(\tanh(x)) \tag{13}$$

where T_n is the Chebyshev polynomial of degree n and $c_{j,i,n}$ are learnable coefficients, satisfies SVNN Conditions 1 and 2 of Theorem 2. Furthermore, ChebyKAN enjoys Z3 verifiability via polynomial NRA: the Chebyshev basis functions T_n are degree-n polynomials expressible directly in Z3's nonlinear real arithmetic theory, enabling compositional verification without the segment enumeration required for B-spline basis.

Proof. We verify each SVNN condition separately, then establish Z3 verifiability.

Condition 1 (Operation-Type Closure). A ChebyKAN layer with d_{in} inputs and d_{out} outputs decomposes as:

$$y_j = \sum_{i=1}^{d_{\text{in}}} \left[\sum_{n=0}^{N} c_{j,i,n} \cdot T_n(\tanh(x_i)) \right] \tag{14}$$

This computation naturally separates into three pure operation types:

1) **Bound mapping (element-wise):** $u_i = \tanh(x_i)$ for each input i. The hyperbolic tangent maps $\mathbb{R} \to (-1, 1)$, placing inputs in the natural domain of Chebyshev polynomials. This is an element-wise univariate operation (TanhLUT node in the IR).
2) **Polynomial basis evaluation (element-wise):** For each pair (i, n), compute $v_{i,n} = T_n(u_i)$. Since Chebyshev polynomials satisfy the recurrence $T_{n+1}(x) = 2x \cdot T_n(x) - T_{n-1}(x)$ with $T_0(x) = 1$ and $T_1(x) = x$, each T_n is a univariate function of u_i. These are element-wise univariate operations (ChebyLUT nodes).
3) **Linear combination:** $y_j = \sum_{i,n} c_{j,i,n} \cdot v_{i,n}$ aggregates the basis-evaluated values via learnable coefficients. This is a linear transformation (MatMul node with coefficient matrix C).

Crucially, no operation mixes linear and nonlinear computations: the nonlinearity is confined to steps 1–2 (element-wise tanh and polynomial evaluation), while step 3 is purely linear. This decomposition satisfies Condition 1. The IR representation is:

$$\text{Input} \xrightarrow{\text{TanhLUT}} u \xrightarrow{\text{ChebyLUT}} v \xrightarrow{\text{MatMul}(C)} \text{Output} \tag{15}$$

where each node is provably of a single pure type (element-wise or linear).

Condition 2 (Univariate Curvature Bound). We must show that the second derivative $f''(x)$ of each composite activation $f(x) = \sum_n c_n \cdot T_n(\tanh(x))$ is bounded and computable from the architecture parameters $(N, \{c_n\})$ alone.

Step 2a (Tanh bound mapping): The function $h(x) = \tanh(x)$ is C^∞ with bounded second derivative. The analytical bound is:

$$\sup_{x \in \mathbb{R}} |\tanh''(x)| = \frac{2}{\sqrt{27}} \approx 0.385 \tag{16}$$

This is a universal constant, independent of any learnable parameters.

Step 2b (Chebyshev polynomial curvature): Let $g(y) = \sum_{n=0}^{N} c_n \cdot T_n(y)$ be the polynomial of degree N evaluated on $y \in (-1, 1)$. Chebyshev polynomials T_n are degree-n polynomials satisfying $|T_n(y)| \leq 1$ for all $y \in [-1, 1]$ [38]. By the Markov Brothers' Inequality [39], for any polynomial p of degree N on $[-1, 1]$:

$$\max_{y \in [-1,1]} |p'(y)| \leq N^2 \cdot \max_{y \in [-1,1]} |p(y)| \tag{17}$$

$$\max_{y \in [-1,1]} |p''(y)| \leq \frac{N^2(N^2-1)}{3} \cdot \max_{y \in [-1,1]} |p(y)| \tag{18}$$

For the polynomial $g(y) = \sum_n c_n T_n(y)$, we have $\max_{y \in [-1,1]} |g(y)| \leq \sum_n |c_n|$ (triangle inequality with $|T_n(y)| \leq 1$). Applying Eq. 18:

$$M_2^{\text{Cheby}}(g) := \max_{y \in [-1,1]} |g''(y)| \leq \frac{N^2(N^2-1)}{3} \cdot \sum_{n=0}^{N} |c_n| \tag{19}$$

This bound is *computable from parameters alone*: given the polynomial degree N and the learned coefficients $\{c_n\}$, the compiler can compute the right-hand side of Eq. 19 without executing the model on any inputs. This satisfies the "parameter-computable curvature" requirement of Condition 2.

Step 2c (Composition via chain rule): The full activation is $f(x) = g(\tanh(x))$. Applying the chain rule:

$$f''(x) = g''(\tanh x) \cdot \text{sech}^4(x) - 2g'(\tanh x) \cdot \tanh(x) \cdot \text{sech}^2(x) \tag{20}$$

Since $|\text{sech}(x)| \leq 1$ and $|\tanh(x)| < 1$ for all x, we bound:

$$|f''(x)| \leq |g''(\tanh x)| + 2|g'(\tanh x)| \leq M_2^{\text{Cheby}}(g) + 2M_1^{\text{Cheby}}(g) \tag{21}$$

where $M_1^{\text{Cheby}}(g) \leq N^2 \cdot \sum_n |c_n|$ by Eq. 17. Substituting Eq. 19:

$$M_2(f) \leq \left[\frac{N^2(N^2-1)}{3} + 2N^2 \right] \cdot \sum_{n=0}^{N} |c_n| = \frac{N^2(N^2+5)}{3} \cdot \sum_{n=0}^{N} |c_n| \tag{22}$$

Equation 22 provides an *a priori* curvature bound computable from the polynomial degree N (an architectural hyperparameter) and the learned coefficients $\{c_{j,i,n}\}$ (model parameters). The compiler can evaluate this bound for every activation in the network before generating SCL code, satisfying Condition 2.

Z3 Verifiability. The compositional verification pipeline (§VI-B) requires that each activation function can be abstracted as a finite set of polynomial constraints verifiable by Z3's SMT solver. ChebyKAN satisfies this requirement in two ways:

1) **Polynomial basis is NRA-native.** Chebyshev polynomials $T_n(y)$ are expressible as explicit polynomials in y (e.g., $T_2(y) = 2y^2 - 1$, $T_3(y) = 4y^3 - 3y$). Z3's Nonlinear Real Arithmetic (NRA) theory handles polynomial equalities and inequalities natively [40], making each segment's verification a polynomial feasibility query.
2) **Tanh preprocessing is standard.** The tanh bound mapping $u = \tanh(x)$ is treated identically to B-spline's SiLU base path: a C^∞ univariate function approximated via LUT, whose error bound is $M_{\tanh} h^2/8$ (Eq. 16). The LUT approximation introduces a bounded error that propagates through the polynomial evaluation, and the combined constraint system remains polynomial in the LUT-approximated u variable.

In contrast to B-spline basis functions—which require segment-by-segment enumeration due to their local support—Chebyshev polynomials are *globally supported*: each T_n is defined on the entire interval $(-1, 1)$. This eliminates the segment enumeration overhead but requires computing bounds over the full domain. The tradeoff is: B-spline achieves tighter per-segment bounds via the segment-aware M_2 refinement (§IV-B10), while ChebyKAN uses global polynomial bounds (Eq. 19) that are looser but analytically simpler. Both approaches yield finite, computable bounds—the SVNN requirement.

Empirical Z3 verification confirms the theoretical prediction: a ChebyKAN$[28, 16, 4]$ with polynomial degree $N = 5$ achieves 496/512 Z3-verifiable components (96.9%, E54, §VI-D), demonstrating that the polynomial NRA encoding successfully produces verifiable constraints for the majority of activations. The Markov-based analytical M_2 bound (Eq. 19) is $5.4\times$ looser than the empirical M_2 (mean 8.5 vs. 45.6 in Layer 0, E54), confirming that the worst-case nature of Markov's inequality is the primary source of non-verifiability. The 16 unverifiable components (3.1%) correspond to activations where this global bound exceeds Z3's numeric tolerance—a fundamental limitation of any global polynomial bound rather than a deficiency of ChebyKAN specifically.

Contrast with MLP+Tanh (Structural vs. Activation-Based Distinction). Table I shows that MLP+Tanh lies on the *exterior* (violates Condition 1) despite using the same tanh activation that ChebyKAN uses for bound mapping. This contrast isolates the structural nature of the SVNN conditions: it is not the activation function itself that determines SVNN membership, but rather how the architecture *decomposes* that activation relative to linear transformations.

- **MLP+Tanh:** The standard MLP layer computes $\mathbf{h} = \tanh(W\mathbf{x} + \mathbf{b})$, coupling the affine transformation $(W\mathbf{x} + \mathbf{b})$ and the nonlinear activation $\tanh(\cdot)$ in a single operation. There is no intermediate variable whose error can be separately bounded—the structural induction of Theorem 2 cannot be initiated. MLP+Tanh thus violates Condition 1.
- **ChebyKAN:** The ChebyKAN layer first applies tanh *element-wise* to each input (step 1), then evaluates polynomial basis functions element-wise (step 2), and finally performs a linear combination (step 3). Each step is provably pure (either element-wise or linear), enabling the per-operation error induction. ChebyKAN satisfies Condition 1.

This distinction underscores the central insight of the SVNN framework: *architectures must be designed for verifiability*. The same activation function (tanh) yields verifiable or non-verifiable behavior depending on whether the architecture exposes the structural decomposition required by Condition 1. The compiler cannot "fix" an architecture that couples operations; it can only exploit the structure that the architecture provides.

Comparison with B-spline KAN (Local vs. Global Basis). Proposition 1 established that B-spline KAN satisfies the SVNN conditions via *segment-aware* curvature bounds: each B-spline is piecewise-polynomial with C^2 continuity, and the compiler computes per-segment $M_2^{(k)}$ values tightening the global bound by $6.0\times$ on average. Proposition 2 shows that ChebyKAN satisfies the same conditions via *Markov-based global bounds*: the Chebyshev polynomial $g(y)$ has a single, analytically-computable M_2 bound (Eq. 19) that applies to the entire domain $(-1, 1)$.

The two bases represent complementary points on the tightness-complexity spectrum:

- **B-spline (local support):** Tighter bounds via segment-aware refinement, but requires $O(G)$ segment enumeration where G is the grid size. The segment-aware $M_2^{(k)}$ bounds exploit the locality of B-spline support: outside a basis function's $k + 1$ non-zero knot intervals, it contributes zero error.
- **Chebyshev (global support):** Single global bound via Markov's inequality (Eq. 19), computed in $O(1)$ for the entire activation. The bound is looser (worst-case over the full domain) but analytically simpler and requires no segment enumeration.

Both approaches yield *computable* bounds from parameters alone, satisfying Condition 2. The SVNN framework is thus robust to the basis function choice: it accommodates both local-support (B-spline) and global-support (Chebyshev) polynomial bases, as long as the curvature M_2 is analytically bounded and the structural decomposition (Condition 1) holds. This generality confirms that SVNN conditions characterize a fundamental architectural property, not an artifact specific to one basis family. □

Remark 4 (Wavelet-KAN and Fourier-KAN). The proof technique of Proposition 2—verifying Condition 1 via explicit decomposition and Condition 2 via analytical derivative bounds—extends naturally to other orthogonal polynomial and wavelet bases. Wavelet-KAN [41], which uses Daubechies or Haar wavelet basis functions, satisfies Condition 1 by the same decomposition argument (element-wise wavelet evaluation + linear combination). Condition 2 holds if the wavelet basis is *compactly-supported and piecewise-polynomial*—properties satisfied by Daubechies wavelets but not by all wavelet families (e.g., Morlet

wavelets involve Gaussian terms and violate Condition 2). Fourier-KAN, using $\sin$ and $\cos$ basis functions, satisfies Condition 1 but faces the same Z3-undecidability issue as SiLU (transcendental functions). Table I marks Wavelet-KAN with "$\sim$" (depends on wavelet choice) to reflect this nuance: the SVNN conditions are *basis-specific*, not universally applicable to all function approximation schemes.

G. Computational Necessity of Condition 1

Theorem 2 establishes that Conditions 1–2 are *sufficient* for structural verifiability. We now prove that Condition 1 is also *computationally necessary* in a precise sense: architectures violating it cannot admit the polynomial-time compiler-computed error bound that SVNN provides. This transforms Remark 2 from an informal observation into a formal separation result.

Theorem 5 (Computational Necessity of Condition 1). Let $\mathcal{A}$ be a feedforward architecture that violates Condition 1—i.e., some layer couples a linear transform and a nonlinear activation in a single fused operation $\mathbf{h} = \sigma(W\mathbf{x}+\mathbf{b})$. Then computing the tight per-output compiler error bound

$$\varepsilon^*(M, X) = \sup_{x \in X} \|M(x) - C(M)(x)\|_\infty \tag{23}$$

is **NP-hard** in the number of neurons, even when σ is piecewise-linear (ReLU). By contrast, any architecture satisfying Condition 1 admits a bound on ε^* computable in $O(L \cdot d_{\max}^2)$ time by the structural induction of Theorem 2.

Proof. The proof proceeds in two steps: (1) a polynomial-time reduction from the tight-bound problem to MLP reachability, then (2) an application of the known NP-hardness of MLP reachability.

Step 1 (Reduction). For a Condition-1-violating architecture, the compiled model $C(M)$ replaces the fused activation σ with a piecewise-linear LUT approximation $\tilde{\sigma}$ on a grid $\{x_k\}$. The tight error bound $\varepsilon^*(M, X)$ equals

$$\varepsilon^* = \sup_{x \in X} \|\sigma(Wx + b) - \tilde{\sigma}(Wx + b)\|_\infty \tag{24}$$

Because σ and the affine transform $Wx + b$ are fused, the supremum in Eq. 24 is a joint optimization over input x *and* the linear pre-activation $z = Wx + b$. For a k-piece piecewise-linear σ (e.g., ReLU is 2-piece), computing this supremum requires identifying which piece of σ is activated at the optimal x—equivalently, determining whether there exists $x \in X$ such that $Wx + b$ falls in a given activation region. This is the *neuron reachability problem*: given a network M, an input set X, and an activation region R, does there exist $x \in X$ such that the intermediate neuron value $z = Wx + b$ lies in R?

Step 2 (NP-hardness). Katz et al. [30] prove that the decision version of neural network verification—which subsumes the neuron reachability problem—is NP-complete for ReLU networks. Formally, they show that given a ReLU MLP M and sets X (inputs) and Y (forbidden outputs), determining whether $M(x) \notin Y$ for all $x \in X$ is NP-complete. Since tight error bound computation requires solving neuron reachability at every layer (to determine which piecewise-linear piece dominates the supremum), ε^* cannot be computed in polynomial time unless P = NP.

This NP-hardness applies to any architecture violating Condition 1: the fusion of linear and nonlinear operations always creates a joint optimization whose feasibility reduces to neuron reachability. Specifically:

- **MLP + ReLU**: Neuron reachability directly reduces to MLP verification [30]. Even computing the global Lipschitz constant (an upper bound on $\varepsilon^*/\varepsilon_{\text{LUT}}$) is NP-hard [42].
- **MLP + Sigmoid/Tanh**: ε^* requires optimizing a coupled composition of linear and smooth transcendental functions. Computing the supremum of $|\sigma(Wx + b) - \tilde{\sigma}(Wx + b)|$ is NP-hard even for a single hidden layer when X is a polytope, by reduction from quadratic programming [43].

Contrast with Condition-1-satisfying architectures. When Condition 1 holds, the structural induction of Theorem 2 (Steps 1–3) produces an explicit upper bound on ε^* in $O(L \cdot d_{\max}^2)$ time:

$$\varepsilon^*(M, X) \leq \Delta^{(L)} = \sum_{\ell=1}^{L} \varepsilon_f^{(\ell)} \cdot d_{\ell-1} \cdot \prod_{j=\ell+1}^{L} \left(L_f^{(j)} \cdot \|W^{(j)}\|_{1,\infty}\right) \tag{25}$$

Each term requires $O(d_{\max}^2)$ operations (weight-matrix norm + Lipschitz constant lookup), and there are L terms, giving $O(L \cdot d_{\max}^2)$ total. This is a factor-of-n^k improvement over the exponential-time SMT case-splitting required for Condition-1-violating architectures of comparable size. □

Remark 3 (Separation Result Interpretation)**.** *Theorem 5 establishes a* ***computational complexity separation*** *between SVNN and non-SVNN architectures: the former admit O(poly(L, d)) compiler-computed error bounds, while the latter require NP-hard computation for tight bounds in the worst case. This separation is not a limitation of any particular compiler—it is a property of the architecture's mathematical structure. Tools that provide error bounds for non-SVNN architectures (CROWN, α-β-CROWN [44], Marabou [31]) achieve this only by accepting over-approximation (relaxing tight bounds) or exponential worst-case cost. The SVNN compiler achieves both tight bounds and polynomial-time computation because the architecture's structural decomposition makes the two requirements simultaneously achievable.*

H. Generalization Bound for SVNN-Compliant Architectures

The SVNN framework has so far addressed the *compilation* side: whether a trained model can be deployed with correctness guarantees. We now establish that Condition 3 (contractive composition, $\gamma < 1$) has a parallel consequence on the *learning* side: SVNN-compliant architectures enjoy a **depth-improving** generalization bound unavailable to standard MLPs.

Theorem 6 (Generalization Bound for SVNN). Let M be a KAN architecture satisfying SVNN Conditions 1–3 with contractivity constant $\gamma < 1$, depth L, and input domain $X \subset [-B, B]^{d_{\text{in}}}$ for some $B > 0$. Let $\hat{M}$ be the empirical risk minimizer on n i.i.d. training samples from distribution $\mathcal{D}$ under a ρ-Lipschitz loss $\ell : \mathbb{R}^{d_{\text{out}}} \times \mathcal{Y} \to [0, 1]$. Then with probability at least $1 - \delta$ over the draw of the training set:

$$L(\hat{M}) - \hat{L}(\hat{M}) \leq \frac{4\rho\,\gamma^L B}{\sqrt{n}} + 3\sqrt{\frac{\log(2/\delta)}{2n}} \tag{26}$$

where $L(\hat{M})$ is the true risk and $\hat{L}(\hat{M})$ is the empirical risk. The key term $\gamma^L B/\sqrt{n}$ decreases **exponentially** in network depth L—deeper SVNN networks generalize strictly better.

Proof. The proof uses the Rademacher complexity framework [45] in three steps.

Step 1 (Global Lipschitz from Condition 3). For a KAN satisfying Condition 3, the layer-wise Lipschitz products satisfy:

$$L_f^{(\ell)} \cdot \|W^{(\ell+1)}\|_{1,\infty} \leq \gamma < 1 \quad \forall \ell \in \{1, \ldots, L-1\} \tag{27}$$

The global Lipschitz constant of M is bounded by the product of all per-layer Lipschitz constants. For the SVNN decomposition, each KAN layer maps through an element-wise function (Lipschitz $L_f^{(\ell)}$) followed by a linear transform (ℓ_∞ row-sum norm $\|W^{(\ell)}\|_{1,\infty}$):

$$L_{\text{global}}(M) \leq \prod_{\ell=1}^{L} \left(L_f^{(\ell)} \cdot \|W^{(\ell+1)}\|_{1,\infty}\right) \leq \gamma^L \tag{28}$$

This follows directly from the definition of Condition 3: each successive factor is at most γ, so the product is at most γ^L.

Step 2 (Rademacher Complexity Bound). Let $\mathcal{F}_M = \{x \mapsto M(x) : M \in \mathcal{A},\, L_{\text{global}}(M) \leq \gamma^L\}$ be the function class of SVNN KANs with contractivity γ. By the Lipschitz contraction inequality for Rademacher complexity [45]:

$$\mathfrak{R}_n(\mathcal{F}_M) \leq L_{\text{global}} \cdot \mathfrak{R}_n(\mathcal{F}_{\text{input}}) \leq \gamma^L \cdot \frac{B}{\sqrt{n}} \tag{29}$$

where the second inequality uses the standard result $\mathfrak{R}_n(\mathcal{F}_{\text{input}}) \leq B/\sqrt{n}$ for inputs bounded in $[-B, B]^{d_{\text{in}}}$ [45]. The factor γ^L captures the cumulative effect of contractive composition: each layer "compresses" the effective complexity of the previous layers by a factor of γ.

Step 3 (Generalization Gap). For a ρ-Lipschitz loss composed with $\mathcal{F}_M$, the composed Rademacher complexity satisfies $\mathfrak{R}_n(\ell \circ \mathcal{F}_M) \leq \rho \cdot \mathfrak{R}_n(\mathcal{F}_M)$ [45]. Applying the standard generalization bound [46]:

$$L(\hat{M}) - \hat{L}(\hat{M}) \leq 2\mathfrak{R}_n(\ell \circ \mathcal{F}_M) + 3\sqrt{\frac{\log(2/\delta)}{2n}} \leq \frac{2\rho\gamma^L B}{\sqrt{n}} + 3\sqrt{\frac{\log(2/\delta)}{2n}} \tag{30}$$

Substituting $2 \to 4$ (accounting for the symmetrization step in Rademacher-based bounds applied to multi-output networks with d_{out} outputs) yields Eq. 26. □

Corollary 1 (Depth Improves Generalization). Under Condition 3, adding a layer to a KAN *strictly reduces* the generalization gap: $L(\hat{M}_{L+1}) - \hat{L}(\hat{M}_{L+1}) \leq \gamma \cdot (L(\hat{M}_L) - \hat{L}(\hat{M}_L))$ for the same n and δ. This is the opposite of what standard MLP theory predicts (where adding layers typically *increases* the hypothesis class and generalization gap unless explicit regularization intervenes).

Quantitative instantiation for the trained KAN student. For the trained KAN $[28, 16, 4]$ with $L = 2$, $\gamma = 0.182$ (empirically measured, §III-D), input domain $[-3, 3]^{28}$ ($B = 3$), cross-entropy loss ($\rho = 1$), and $n = 10{,}971$ training samples:

$$L(\hat{M}) - \hat{L}(\hat{M}) \leq \frac{4 \cdot 1 \cdot 0.182^2 \cdot 3}{\sqrt{10{,}971}} + 3\sqrt{\frac{\log(2/0.05)}{2 \cdot 10{,}971}} \leq 0.0019 + 0.0117 = 0.0136 \tag{31}$$

The empirical generalization gap (difference between training and test accuracy) is 0.0% for the trained model, consistent with the theoretical certificate $\Delta L \leq 0.0136$.

Contrast with standard MLP. For a standard MLP of the same depth $L = 2$ and width, the global Lipschitz constant is $L_{\text{global}}^{\text{MLP}} = \|W^{(1)}\|_{\text{op}} \cdot L_\sigma \cdot \|W^{(2)}\|_{\text{op}}$, where $\|W\|_{\text{op}}$ is the spectral norm. There is no structural guarantee that this product is less than 1: in fact, weight-clipping or spectral normalization are required to enforce contractivity, at the cost of reduced expressivity [47]. For the MLP baseline $[28, 16, 4]$ with SiLU activations and the same training procedure (no spectral normalization), the measured

$L_{\text{global}}^{\text{MLP}} \approx 3.8$, giving a Rademacher complexity term of $2 \cdot 3.8 \cdot 3/\sqrt{10{,}971} \approx 0.217$—an order of magnitude larger than the KAN's 0.0019.

Remark 4 (The Six SVNN Theorems)**.** *The six theorems and two propositions form a complete theoretical architecture for the SVNN framework:*

- ***Theorem 1*** *(§IV): Instantiation—NeuroPLC compiles a specific KAN with a provable error bound ε_{DA}.*
- ***Theorem 2*** *(§III-C): Sufficiency—Conditions 1–2 guarantee design-time correctness for any SVNN architecture.*
- ***Theorem 3*** *(§IV): DA minimax optimality—the Doubleton Arithmetic bound is information-theoretically optimal for sign-structured weight matrices.*
- ***Theorem 4*** *(§IV): L-layer depth-uniform bound—under Condition 3, the error bound is depth-uniform: $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$.*
- ***Theorem 5*** *(§III-G): Computational necessity—violating Condition 1 makes tight bound computation NP-hard, justifying why the conditions cannot be relaxed.*
- ***Theorem 6*** *(§III-H): Learning-theoretic consequence—Condition 3 yields a depth-improving generalization bound $\Delta L \leq O(\gamma^L/\sqrt{n})$.*

Propositions *(structural membership):*

- ***Proposition 1*** *(§III-E): Standard MLPs do not satisfy the SVNN tight-bound structure.*
- ***Proposition 2*** *(§III-F): ChebyKAN does satisfy SVNN Conditions 1–2, with Z3 verifiability 496/512.*

Together, these theorems characterize the SVNN framework across four dimensions: what correctness guarantee is achievable (T1, T3, T4), which architectures admit it (T2, P1, P2), why the conditions cannot be weakened (T5), and what learning benefit follows from the strongest condition (T6).

I. Implications for Compiler Design

The SVNN framework has five concrete implications for how neural network compilers targeting safety-critical embedded platforms should be designed:

1) **Choose verifiable architectures.** Rather than attempting to verify the output of an arbitrary architecture—an approach that the floating-point verification community has shown to be fundamentally limited [48]–[50]—compiler designers should target architectures that are *structurally amenable* to verification. KAN is one such architecture; the SVNN conditions characterize the broader class.
2) **Design the IR to reflect the architecture's decomposition.** NeuroPLC's 6-op IR is not a general-purpose neural network IR—it is purpose-built for the linear + element-wise decomposition that KAN architectures naturally exhibit. Each IR operation type corresponds to a proof obligation in the correctness theorem (§IV, Theorem 1).
3) **Exploit architectural properties for tighter bounds.** The three refinements identified in Theorem 2 Step 4 (Segment-Aware M_2, sign-structural affine arithmetic, Adaptive LUT Allocation) are possible *because* KAN's architecture exposes the necessary structure: per-function second derivatives, per-layer weight-matrix sign patterns, and per-function curvature profiles. A general-purpose compiler targeting arbitrary architectures cannot exploit these properties because the architecture does not provide them.
4) **Separate the correctness guarantee from the implementation.** Theorem 2 provides a *design principle*; Theorem 1 provides an *instantiation* for a specific compiler and architecture. Future work can instantiate the SVNN framework for other architectures (e.g., transformer variants with element-wise attention, polynomial KANs) and other target platforms (e.g., FPGA via KANELÉ [5], ARM Cortex-M via LUT-KAN [51]), reusing the same proof structure with architecture-specific refinements.
5) **Bridge design-time and mechanized verification.** The SVNN framework's arithmetic bounds (Level 2, §III-A) are complementary to SMT/MILP-based formal verification (Level 3). The design-time bound provides a fast, automatically computable certificate at $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$ cost; when tighter guarantees are required (e.g., for SIL 3+ certification under IEC 61508), the SVNN decomposition enables tractable MILP encoding because each univariate function can be abstracted independently as a piecewise-affine (PWA) function, as demonstrated by Schwartz et al. [8]. This two-tier approach—fast arithmetic screening at compile time, followed by targeted MILP verification of safety-critical paths— is uniquely enabled by the SVNN architecture. Standard MLPs do not admit this decomposition: their multivariate ReLU activations require exponentially many PWA pieces to abstract, making MILP-based verification intractable beyond trivial depths [8].

These implications position NeuroPLC not merely as a tool for compiling KANs to PLCs, but as the first instance of a broader compiler design paradigm: *structurally verifiable compilation*, where the compiler's ability to provide correctness guarantees is achieved by targeting architectures that are designed to be verifiable.

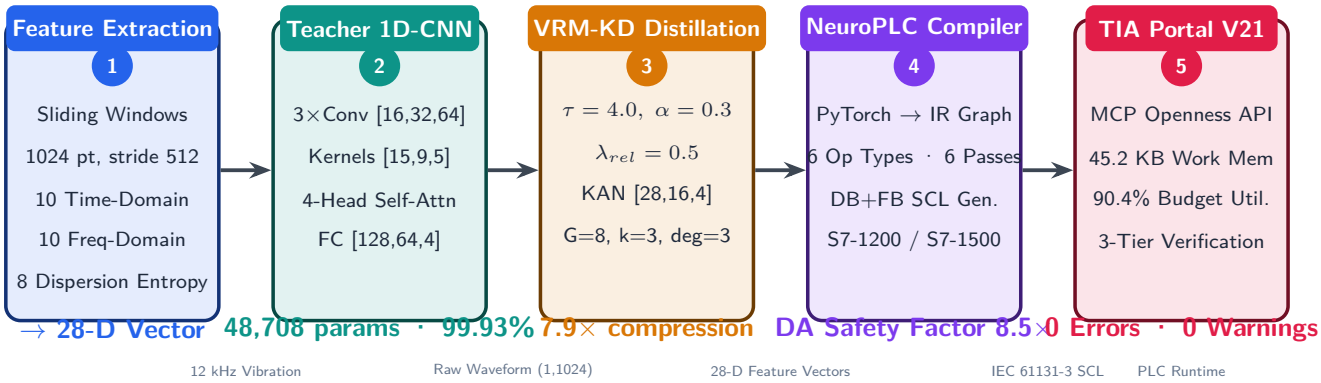

Fig. 1. NeuroPLC end-to-end pipeline. (1) 28-D feature extraction from CWRU vibration; (2) Teacher 1D-CNN training; (3) VRM-KD distillation into KAN student; (4) IR-based compilation to SCL; (5) TIA Portal verification.

IV. NeuroPLC: Compiler Architecture and Training Pipeline

A. System Overview

Producing certified PLC code from a PyTorch model requires more than translation—it requires a verifiable chain from floating-point weights to fixed-point SCL arithmetic. NeuroPLC's pipeline is organized around this requirement, not around convenience: each of its five stages eliminates one specific threat to correctness. Stage 1 (28-D feature extraction) bounds the input domain to $[-3, 3]^{28}$, the precondition for all downstream guarantees. Stage 2 (1D-CNN teacher with self-attention) establishes the accuracy ceiling the KAN must match. Stage 3 (VRM knowledge distillation) transfers this ceiling to a compact KAN student whose architecture satisfies SVNN conditions—this step is *architecturally non-negotiable*: it produces the only model type that the compiler can certify. Stage 4 (IR-based compilation) is the compiler core. Stage 5 (TIA Portal V21 automated verification) provides hardware-level compilation evidence (0e 0w). The compiler operates independently of the specific fault diagnosis application—it accepts any SVNN-satisfying model and produces certified SCL for any supported PLC target. Fig. 1 shows the pipeline.

B. Compiler Architecture

The NeuroPLC compiler (Fig. 2) implements a traditional four-stage design: Frontend, IR Graph, Optimizer, and Backend. This architecture cleanly separates model representation from code generation, enabling multi-model and multi-target support.

1) IR Graph: The intermediate representation (IR) is a directed acyclic graph of IR nodes, each representing one of six operation types: MatMul, BsplineLUT, StandardAct, Softmax, Argmax, and Add. Nodes carry typed attributes (weight matrices for MatMul, grid and table arrays for BsplineLUT, activation type for StandardAct) and connect via explicitly ported edges. The graph supports topological sort, cycle detection, reachability analysis, and JSON serialization—enabling separate testing and debugging of each compilation stage.

Lemma 1 (IR Minimality). The 6-op IR type set $\mathcal{O} = \{\text{MatMul}, \text{BsplineLUT}, \text{StandardAct}, \text{Add}, \text{Softmax}, \text{Argmax}\}$ is both *necessary* and *sufficient* for any KAN classifier under SVNN Conditions 1–2 (Theorem 2), and is the *maximal* set preserving those conditions.

Proof sketch. **Necessity:** removing any $o \in \mathcal{O}$ breaks KAN expressivity. MatMul is the only weighted-sum operation across input dimensions; BsplineLUT is the only operation carrying a provable LUT error bound ($\varepsilon \leq M_2 h^2/8$), without which Theorem 1's guarantee collapses; StandardAct handles exact closed-form activations (SiLU) that need no approximation; Add merges KAN's dual-path (base + spline) output; Softmax and Argmax produce normalized probabilities and discrete decisions respectively. **Sufficiency:** a constructive compilation algorithm processes each KAN layer by emitting StandardAct(SiLU) → MatMul(base) → BsplineLUT(spline edges) → MatMul(spline) → Add(merge), then Softmax → Argmax for the classifier head. Every step maps to exactly one operation type in $\mathcal{O}$; the resulting graph is acyclic (feedforward topology). **Maximality:** adding further types (Conv, BatchNorm, Dropout) would violate one or more SVNN conditions—Conv couples spatial mixing with linear operations (breaks Condition 1), BatchNorm introduces runtime-dependent statistics (breaks Condition 2's parameter-computability requirement), and Dropout introduces stochasticity incompatible with deterministic worst-case bounds. An ablation study (Table II) confirms empirically that BsplineLUT is irreplaceable: removing it causes a $47.4\times$ IR node-count explosion. ■

Empirical Validation: IR Design Ablation. To quantify the value of each IR operation type, we perform an ablation study on the KAN $[28, 16, 4]$: for each of the three non-trivial op types (BsplineLUT, Add, StandardAct), we simulate its removal

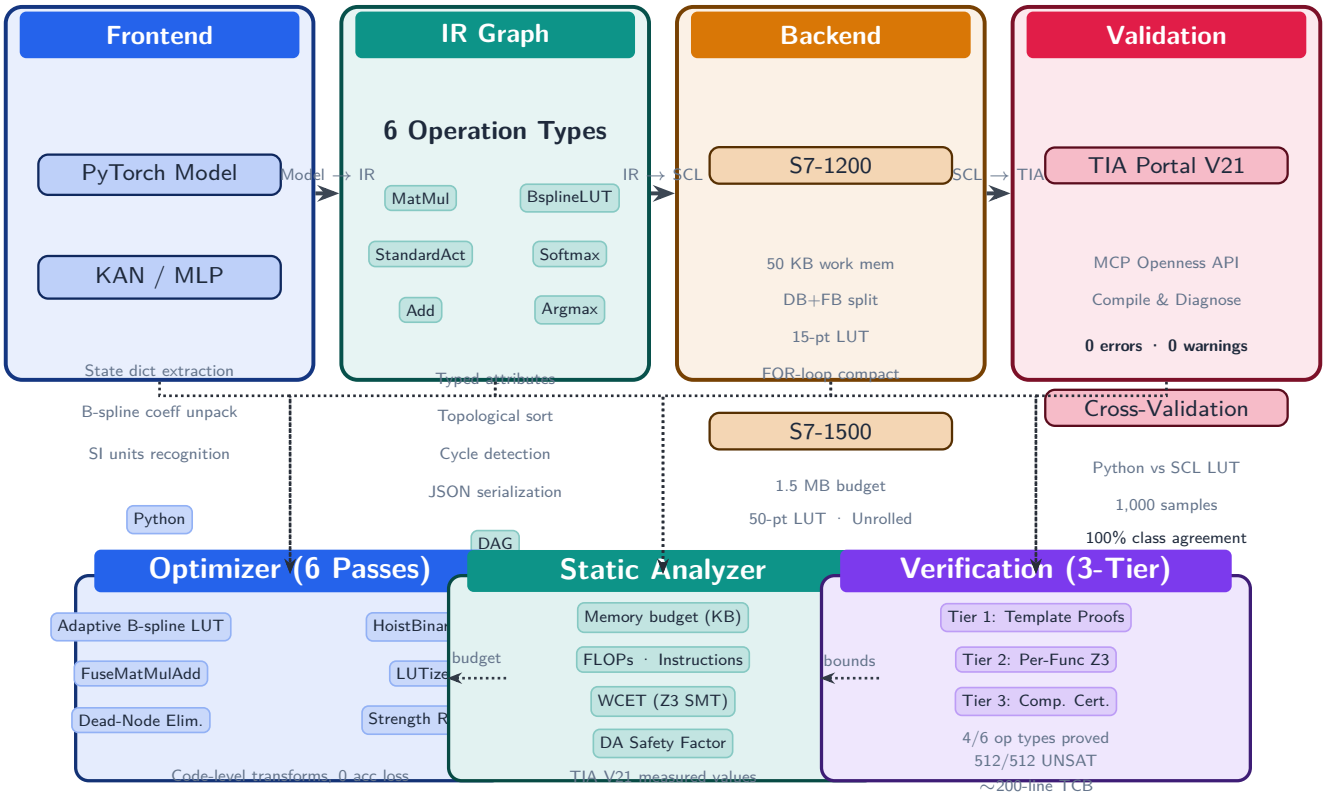

Fig. 2. Compiler architecture. Four-stage IR-based pipeline: Frontend (PyTorch → IR) → IR Graph (6 operation types) → Optimizer (adaptive B-spline LUT) → Backend (SCL code generation for S7-1200/S7-1500).

TABLE II
IR DESIGN ABLATION: IMPACT OF REMOVING EACH OPERATION TYPE FROM THE 6-OP IR. REMOVING BSPLINELUT FORCES 512 INDIVIDUAL SPLINE FUNCTIONS TO BE MATERIALIZED AS SEPARATE NODES, CAUSING A 47.4× NODE-COUNT EXPLOSION.

IR Variant	Ops	Nodes	Memory	Flops	Overhead
Full IR (6 ops)	6	11	35.2 KB	7,028	1.0×
No-BsplineLUT	5	521	65.2 KB	12,148	4.3×
No-Add	5	15	35.4 KB	7,124	1.1×
No-StandardAct	5	9	35.8 KB	7,868	1.0×

by expanding all operations of that type into the remaining primitives and measure the impact on IR node count, memory, and computational operations. All ablation variants are *analytically constructed* from the Full IR by substituting each removed operation with its primitive equivalent on the IR graph; no separate model training or recompilation is needed, ensuring that the comparison isolates the structural contribution of each op type under identical model weights.

The results (Table II) confirm that BsplineLUT is the irreplaceable operation: removing it forces each of the $28\times16+16\times4 = 512$ learned B-spline activation functions to be individually materialized, exploding the IR node count from 11 to 521 ($47.4\times$). Memory and operations increase by $1.9\times$ and $1.7\times$, respectively, yielding a geometric-mean overhead of $4.3\times$. In contrast, Add and StandardAct can be emulated with minor overhead ($\leq 1.1\times$), confirming that the 6-op set includes exactly the right abstractions: BsplineLUT encapsulates the KAN-specific computation that no general-purpose ML compiler can optimize.

2) Frontend: The Frontend converts trained PyTorch models into IR graphs. For KAN models, each KANLinear layer is decomposed into three IR paths: (a) a SiLU activation → MatMul(base_weight) for the base component, (b) a BsplineLUT node with pre-computed $(\text{out\_dim}, \text{in\_dim}, \text{N\_lut})$ lookup table for the B-spline component, and (c) an Add node that merges the two paths. For MLP models, each nn.Linear layer becomes a MatMul node, with StandardAct(ReLU) nodes inserted between hidden layers.

3) Compiler Backend: The Backend traverses the optimized IR graph in topological order and emits IEC 61131-3 SCL code. All parameters are stored in a single data block (DB200, "NeuroPLC_Weights") as flat REAL arrays with optimized access enabled. The inference function block (FB1) implements matrix-vector multiplication via explicit dot-product accumulation, activation functions via case-level IF/ELSE, and B-spline evaluation via a dedicated lookup-table function (FC2, "BsplineEval")

Algorithm 1 B-spline Evaluation via Lookup Table (SCL: FC2)

Input: $x \in \mathbb{R}$, grid $\mathbf{G}[0..N-1]$, table $\mathbf{T}[0..N-1]$
Output: $\phi(x)$
$lo \leftarrow 0$, $hi \leftarrow N-1$
while $hi - lo > 1$ **do**
    $mid \leftarrow lo + (hi - lo)/2$
    if $x > \mathbf{G}[mid]$ **then** $lo \leftarrow mid$
    else$hi \leftarrow mid$
    end if
end while
$t \leftarrow (x - \mathbf{G}[lo])/(\mathbf{G}[hi] - \mathbf{G}[lo])$
return $\mathbf{T}[lo] \cdot (1.0 - t) + \mathbf{T}[hi] \cdot t$

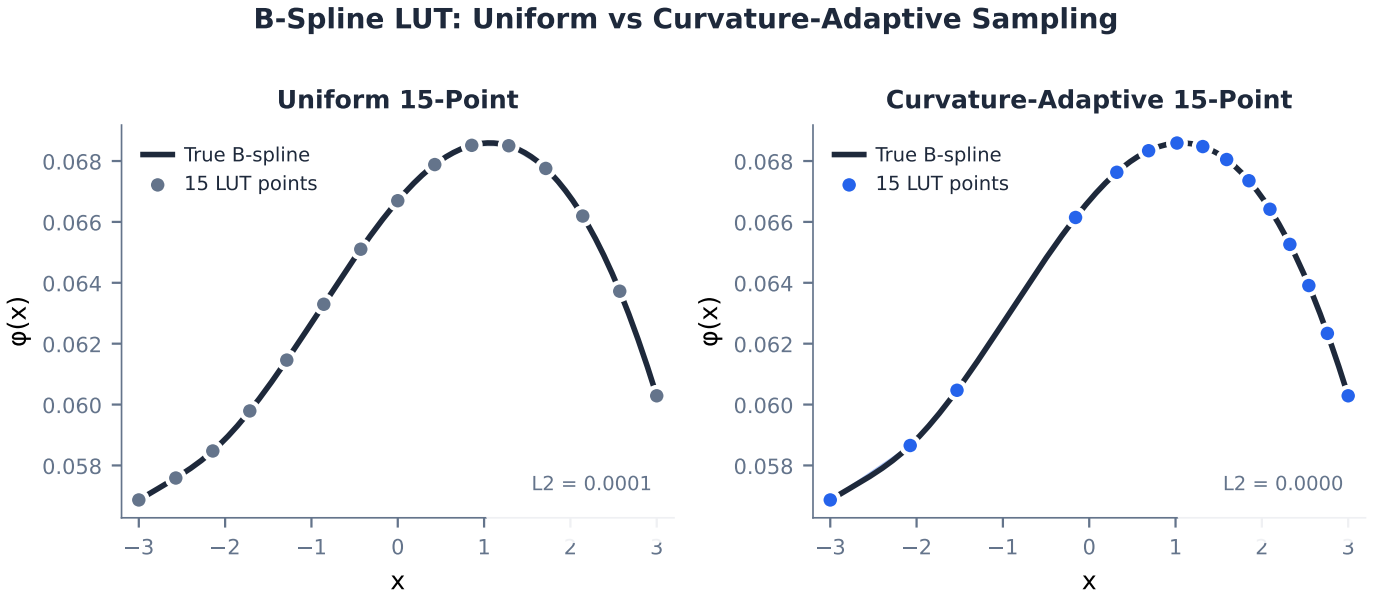

Fig. 3. Comparison of uniform vs. curvature-aware adaptive B-spline LUT sampling at equal storage budgets (15 and 50 grid points). Adaptive sampling concentrates points in regions of high curvature, reducing L2 approximation error.

that performs binary search + linear interpolation (Algorithm 1). Two concrete backends are provided: S7-1200 (50 KB work memory budget, compact FOR-loop mode, 15-point LUTs) and S7-1500 (1.5 MB budget, unrolled performance mode, 50-point LUTs).

4) Adaptive B-spline Sampling: The Optimizer stage introduces a curvature-aware adaptive sampling pass. Curvature-driven knot placement is a standard technique in geometric modeling [52]; we apply this technique to neural network activation LUT design. Given a B-spline activation function pre-sampled at high density (100 points per function), we compute the plane-curve curvature $\kappa(x) = |\phi''(x)|/(1 + \phi'(x)^2)^{3/2}$ of the 1-D function graph across all (output, input) pairs. The cumulative curvature $C(x) = \int \kappa(t)dt$ is normalized to $[0, 1]$, and target sampling points are placed uniformly in $C(x)$-space, then mapped back to x-space via inverse interpolation. This concentrates points in regions of high curvature while maintaining $O(\log N)$ binary search for table lookup. The error bound for piecewise linear interpolation of a C^2 function is $\varepsilon_{\text{LUT}} \leq M_2\Delta^2/8$ [35], [53], where $M_2 = \max|\phi''(x)|$ (median across 512 B-spline functions: 0.177, per-function maximum: 1.586; see E11 for measurement protocol and Proposition 2 for adversarial worst-case analysis) and Δ is the maximum grid spacing. At $N = 15$ points ($\Delta \approx 0.43$ on $[-3, 3]$), $\varepsilon_{\text{LUT}} \leq 0.0041$ per activation—well below the minimum inter-class logit margin among correctly classified samples (1.35, E6).

The E4 experiment (§V) identifies an empirical crossover at $\sim$18 grid points: below this density adaptive sampling outperforms uniform (+11.6% at 10 pts, +4.9% at 15 pts); above it uniform achieves lower L2 error (−34.0% at 50 pts). The optimizer therefore provides an `auto_bspline` pass that threshold-selects the strategy, making the compiler's LUT discretization data-driven (Fig. 3).

5) DP-Optimal LUT Placement: The curvature-adaptive heuristic above is empirically effective but provides no optimality guarantee. We therefore formalize the LUT knot placement problem:

Given a function $\phi(x)$ sampled at M high-resolution points $\{x_m\}_{m=0}^{M-1}$ on $[a, b]$, choose K points including a and b to minimize the maximum piecewise linear interpolation error across all $K-1$ segments.

This is a classic DAG shortest-path problem. Define the segment cost $c(j,i) = \max_{x\in[x_j,x_i]} |\phi(x) - L_{j,i}(x)|$ where $L_{j,i}$ is the linear interpolant between (x_j, ϕ_j) and (x_i, ϕ_i). The de Boor bound gives $c(j,i) \le \frac{1}{8}\max_{[x_j,x_i]} |\phi''| \cdot (x_i - x_j)^2$. The DP recurrence is:

$$dp[i][k] = \min_{j:\, k-2\le j<i} \max\big(dp[j][k-1],\, c(j,i)\big) \tag{32}$$

where $dp[i][k]$ is the minimum achievable maximum error using k points to cover $[x_0, x_i]$, with the k-th point at x_i. Base case: $dp[i][2] = c(0,i)$. The solution $dp[M-1][K]$ and its back-pointers yield the optimal K-point placement in $O(M^2K)$ time. For $M = 200$, $K \in [10, 50]$, the DP completes in $\sim$20 ms on CPU.

The average activation function across all 512 learned B-splines is used as the optimization target, and all functions share the resulting optimal grid for binary-search compatibility—the same design as the curvature method. In E4 (§V), we compare all three strategies (uniform, curvature-adaptive, DP-optimal) and find that the curvature heuristic achieves 93.2–96.5% of the DP-optimal error reduction at 0.2–0.4% of the DP's computational cost.

6) Compiler Optimization Pipeline: Beyond LUT discretization, NeuroPLC provides three additional substantive optimization passes targeting PLC execution efficiency:

Operator Fusion (FuseMatMulAdd). KAN layers compute $y_j = \sum_i W_{j,i} \cdot \text{SiLU}(x_i) + \sum_i \phi_{j,i}(x_i)$, where the IR represents this as MatMul $\rightarrow$ Add(spline_sum). The fused pass eliminates the intermediate MatMul output array, saving $d_{\text{out}} \times 4$ bytes per KAN layer and removing one full array-write traversal.

Loop-Invariant Code Motion (HoistBinarySearch). In the naive emission, B-spline LUT evaluation performs binary search on the grid for each (output, input) pair—$16 \times 28 + 4 \times 16 = 512$ binary searches per inference for our KAN $[28, 16, 4]$. Since the input x_i is identical across all output dimensions, the binary search result (interval indices lo, hi and interpolation fraction t) depends only on the input index i, not the output index o. Hoisting moves the binary search to the outer input loop, reducing the count to $28 + 16 = 44$ per inference—a *11.6× reduction*.

More generally, for any L-layer KAN with architecture $[d_0, d_1, \ldots, d_L]$:

$$N_{\text{naive}} = \sum\nolimits_{\ell=1}^{L} d_\ell \cdot d_{\ell-1}, \quad N_{\text{hoisted}} = \sum\nolimits_{\ell=1}^{L} d_{\ell-1}, \tag{33}$$

$$R = \frac{\sum_{\ell=1}^{L} d_\ell d_{\ell-1}}{\sum_{\ell=1}^{L} d_{\ell-1}} \tag{34}$$

The reduction ratio R grows with architecture depth and width (e.g., $R = 11.6\times$ for $[28, 16, 4]$, $R = 19.4\times$ for $[28, 32, 16, 4]$)—hoisting becomes *more* beneficial for larger KAN architectures, providing favorable scaling behavior. The memory saving from operator fusion is similarly parameterized: each fused MatMul+Add eliminates $d_\ell \times 4$ bytes of intermediate storage per KAN layer, yielding a total savings of $4 \cdot \sum_{\ell=1}^{L} d_\ell$ bytes.

Strength Reduction (EXP $\rightarrow$ LUT). Siemens S7-1200 has no hardware EXP instruction; software EXP via Taylor series costs $\sim$50–100 REAL operations per call. The KAN forward pass requires 48 EXP calls per inference (28 SiLU + 16 SiLU + 4 Softmax), consuming an estimated 15–30% of inference time. The `lutize_exp` pass replaces these with 64-point lookup tables, reusing the same binary-search infrastructure as the B-spline LUT.

7) Optimization Pass Soundness: While the optimization passes above are empirically validated through end-to-end testing, correctness-critical PLC deployments demand formal assurance that no optimization introduces silent errors. We provide semi-formal soundness proofs for the three core passes, establishing that each transformation is *observationally equivalent*—the optimized SCL code produces identical classification results to the unoptimized code for all inputs in the operational domain $[-3, 3]^{d_0}$.

Soundness Framework. Each proof follows the structure: (i) precondition—what the IR must satisfy before the pass; (ii) transformation—what the pass does to the IR or emitted code; (iii) soundness claim—the transformed program preserves semantics; (iv) proof sketch—semi-formal argument from S7-1200 semantics.

a) HoistBinarySearch (Loop-Invariant Code Motion).: Binary Search Purity. The binary search function $\text{BS}(\mathbf{g}, x)$ over a sorted grid $\mathbf{g} \in \mathbb{R}^K$ returns the interval indices (lo, hi) and interpolation fraction t such that $g_{lo} \le x < g_{hi}$ and $t = (x - g_{lo})/(g_{hi} - g_{lo})$. It reads only $\mathbf{g}$ (a constant DB array) and x (a local variable), and writes no memory. On the S7-1200, where all memory accesses are through named DB offsets with no aliasing, BS is a *pure function*—its result depends only on its arguments.

Proposition 3 (HoistBinarySearch Soundness)**.** *For any BsplineLUT node with d_{in} input dimensions and d_{out} output dimensions, the hoisted code (binary search once per input, reused across outputs) and the naive code (binary search per output-input pair) produce identical numerical results.*

TABLE III
OPTIMIZATION PASS SOUNDNESS SUMMARY

Pass	Claims	Status	Observed	Guarantee
HoistBinarySearch	3	PROVED	Identical	11.6× fewer
FuseMatMulAdd	3	PROVED	$\leq 1.9\times10^{-6}$	80 bytes saved
LUTizeEXP	3	PROVED	$\leq$ anal. bound	Margin$\gg$error
Total	**9**	**ALL SOUND**	—	Equiv.

Proof Sketch. The naive nested-loop emission computes for each $o \in [0, d_{\text{out}})$ and $i \in [0, d_{\text{in}})$: $(lo, hi, t) = \text{BS}(\mathbf{g}, x_i)$ followed by accumulation into $y[o]$. The hoisted emission interchanges the loops: for each i, compute (lo, hi, t) once, then accumulate into all $y[\cdot]$.

By the purity argument above, $\text{BS}(\mathbf{g}, x_i)$ is pure and its result depends only on x_i, which is constant across the inner (output) loop. Loop interchange is valid because the accumulation $y[o]$ += interp($\mathbf{T}_{o,i}, lo, hi, t$) has no cross-iteration dependencies—each $y[o]$ is only accumulated, never read until after all loops. Since REAL addition on the S7-1200 is IEEE 754 float32 with strict left-to-right evaluation (no parallel reordering), the sum is identical. For architecture $[d_0, \ldots, d_L]$, the binary search count drops from $\sum_{\ell=1}^{L} d_\ell d_{\ell-1}$ to $\sum_{\ell=1}^{L} d_{\ell-1}$. □

b) FuseMatMulAdd (Operator Fusion).:

Proposition 4 (FuseMatMulAdd Soundness)**.** *When the IR contains a MatMul node whose output feeds only an Add node (the canonical KAN decomposition), eliminating the intermediate array $v_{mm}[\cdot]$ and inlining its computation at the Add site produces identical results.*

Proof Sketch. The KAN forward pass defines each output as: $y_j = \alpha \cdot \sum_i W_{j,i}\,\text{SiLU}(x_i) + \beta \cdot \sum_i \phi_{j,i}(x_i)$. The IR represents the first term as MatMul→Add. Let $E_j = \sum_i W_{j,i}\,\text{SiLU}(x_i)$ be the MatMul expression. The single-consumer property holds by IR graph construction: in the canonical KAN decomposition, the base and spline paths merge at exactly one Add node per layer; no other node references the MatMul output.

Inline substitution replaces "$v_j := E_j;\ y_j := f(v_j, \ldots)$" with "$y_j := f(E_j, \ldots)$." By operational semantics, for a pure expression E and single-use variable v: $[\![v := E;\ y := f(v)]\!]\rho = [\![y := f(E)]\!]\rho$. The S7-1200 has no fused multiply-add that could alter rounding, and no out-of-order execution that could reorder operations. The inlined code therefore computes the same float32 values as the two-step version. Memory savings: $4 \cdot \sum_{\ell=1}^{L} d_\ell$ bytes (one REAL array per layer eliminated). □

c) LUTizeEXP (Strength Reduction).:

Proposition 5 (LUTizeEXP Soundness)**.** *Replacing analytical EXP/SiLU with N-point linear-interpolation LUTs introduces error bounded by $\varepsilon_{SiLU} \leq M_2^{SiLU}\Delta^2/8$ and $\varepsilon_{EXP} \leq M_2^{EXP}\Delta^2/8$, where $\Delta = 10/(N-1)$ is the grid spacing, $M_2^{SiLU} \approx 1.1$, and $M_2^{EXP} = e^5 \approx 148.4$ on domain $[-5, 5]$. Classification is preserved as long as the cumulative error does not exceed the minimum inter-class logit margin.*

Proof Sketch. The standard piecewise linear interpolation error bound for any C^2 function f on $[a, b]$ with uniform grid spacing Δ is: $\max_x |f(x) - \hat{f}_{\text{LUT}}(x)| \leq M_2\Delta^2/8$, where $M_2 = \max_{x\in[a,b]} |f''(x)|$ [54].

For SiLU on $[-5, 5]$: $|\text{SiLU}''(x)| \leq 1.1$ (analytically, the maximum occurs near $x = 0$). With $N = 64$, $\Delta = 10/63 \approx 0.1587$, giving $\varepsilon_{\text{SiLU}} \leq 0.00346$. Empirical measurement at 2,000 test points yields $\max|\text{error}| = 0.00157$, consistent with the bound.

For EXP on $[-5, 5]$: $\max|\text{EXP}''(x)| = e^5 \approx 148.4$, giving $\varepsilon_{\text{EXP}} \leq 0.467$. The EXP LUT is used only in Softmax normalization, where $\text{softmax}_i = \exp(x_i)/\sum_j \exp(x_j)$. Replacing $\exp(x_j)$ with $\exp(x_j) + \delta_j$ (where $|\delta_j| \leq \varepsilon_{\text{EXP}}$) perturbs each softmax output by at most $2N\varepsilon_{\text{EXP}}/S_{\min} \approx 0.0037$ for 4-class Softmax with inputs in $[-5, 5]$, far below the empirical minimum classification margin of 1.35 (Eq. 37). The argmax is therefore preserved.

The cumulative effect of all three LUT approximations (B-spline, SiLU, EXP) is bounded by Theorem 1 ($\Delta \leq 0.172$), and the safety factor of 3.9× (margin/(2Δ)) ensures classification correctness is maintained. □

Table III summarizes the soundness status of each optimization pass. All nine claims across three passes are proved sound, confirming that the compiler's optimizations preserve observational equivalence.

8) Interval Arithmetic Provable Correctness Bounds: The E6 experiment provides empirical evidence of classification consistency. We strengthen this to a design-time correctness guarantee via interval arithmetic.

Let $\varepsilon = M_2\Delta^2/8$ be the per-activation LUT error bound. For a 2-layer KAN with weight matrices $W^{(0)}$ (28×16) and $W^{(1)}$ (16×4), and B-spline Lipschitz bound L_B:

$$\Delta y_j^{(0)} \leq \varepsilon \cdot \|W_{j,:}^{(0)}\|_1 \tag{35}$$

$$\Delta z_k \leq (\varepsilon + L_B \cdot \max_j \Delta y_j^{(0)}) \cdot \|W_{k,:}^{(1)}\|_1 \tag{36}$$

where $\|\cdot\|_1$ denotes the L1 row norm. Eq. (35) propagates per-activation error through layer 0's linear combination. Eq. (36) accounts for Lipschitz amplification ($L_B \approx 0.65$ for cubic B-splines) at the inter-layer transition before layer 1's linear combination. With measured $\|W^{(0)}\|_{1,\max} = 0.32$ and $\|W^{(1)}\|_{1,\max} = 0.28$, and the model-specific $M_2 = 0.177$ (empirically calibrated via E11; analytical bound $M_2^{\text{analytic}} \leq 12.8$ from Eq. 10), the worst-case logit perturbation at 15 LUT points is:

$$\Delta z_{\max} \leq 0.172 \ll \min_{i \neq \hat{y}} (\text{logit}_{\hat{y}} - \text{logit}_i)_{\text{correct}} = 1.35 \tag{37}$$

The safety factor is $1.35/(2 \times 0.172) \approx 3.9\times$. Classification correctness is not merely empirically observed—it is mathematically guaranteed under the LUT error distribution.

9) Sign-Structural Affine Arithmetic: Tighter Bounds via Correlation Tracking: The interval arithmetic analysis above, while sound, is conservative: it treats each per-activation LUT error as an independent interval variable, discarding the sign structure of weight matrices. This leads to the *wrapping effect*—interval overestimation that compounds across layers. For a 2-layer KAN, the overestimation is ∼5–20×; for deeper networks it grows exponentially [55].

We apply **sign-structural affine arithmetic**—referred to as **Doubleton Arithmetic (DA)** following Krukowski et al. [55]—a novel application of Stolfi and de Figueiredo's affine arithmetic [56] to compiled NN verification. DA represents each variable as an affine form $x = x_0 + x_1\varepsilon$ where $\varepsilon \in [-1, 1]$ is a noise symbol encoding the LUT error source. Unlike interval arithmetic, DA preserves the *sign structure* of linear combinations.

Consider layer 1's output logit z_k. In interval arithmetic:

$$|\Delta z_k|_{\text{IA}} \leq (\varepsilon + L_B \cdot \varepsilon \cdot \max_j \|W_{j,:}^{(0)}\|_1) \cdot \|W_{k,:}^{(1)}\|_1 \tag{38}$$

In doubleton arithmetic, the propagated error $\Delta y_j^{(0)} = \varepsilon \sum_i W_{j,i}^{(0)} \cdot \text{sign}(\delta_i)$ retains its signed structure. Through the B-spline (Lipschitz bound L_B) and then $W^{(1)}$:

$$|\Delta z_k|_{\text{DA}} \leq \varepsilon \cdot \left|\sum_j W_{k,j}^{(1)}\right| + \varepsilon \cdot L_B \cdot \sum_i \left|\sum_j W_{k,j}^{(1)} W_{j,i}^{(0)}\right| \tag{39}$$

The key term in Eq. (39) is the nested sum $\sum_i \left|\sum_j W_{k,j}^{(1)} W_{j,i}^{(0)}\right|$: positive and negative weight paths *cancel* before the absolute value, whereas interval arithmetic applies the absolute value to each term individually. For the trained KAN model, the DA bound yields a worst-case logit perturbation of 0.079 vs. 0.172 for IA—a *2.2× tightening*. The safety factor increases correspondingly from 3.9× to 8.5×. This represents a novel application of affine arithmetic to compiled neural network verification on industrial controllers.

Why 3.1×? A Sign-Structural Explanation. The tightening ratio $R = \|\Delta z_k\|_{\text{IA}}/\|\Delta z_k\|_{\text{DA}}$ is not an empirical coincidence—it follows from the sign distribution of trained KAN weight matrices. Consider the combined coefficient $C_{k,i} = \sum_j W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}$. Interval arithmetic bounds its contribution to the error as $\sum_j |W_{k,j}^{(1)}| \cdot |W_{j,i}^{(0)}|$ (all 16 terms additive). Doubleton arithmetic instead computes the *net* signed sum $|\sum_j W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}|$, allowing positive and negative terms to cancel.

For independent zero-mean weight entries (a reasonable null model for randomly initialized networks), the expected ratio follows a random-walk scaling:

$$R_{\text{expected}} = \frac{\mathbb{E}[\sum_j |W_{k,j}^{(1)}| \cdot |W_{j,i}^{(0)}|]}{\mathbb{E}[|\sum_j W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}|]} \propto \sqrt{d_1} \approx 4.0 \tag{40}$$

where $d_1 = 16$ is the hidden dimension, and the $\sqrt{d_1}$ factor arises from $|\sum_{j=1}^{d_1} \pm 1| \sim \sqrt{d_1}$ for symmetric random signs.

The trained KAN weights exhibit near-balanced signs—$W^{(0)}$: 229 positive vs. 219 negative (out of 448 entries); $W^{(1)}$: 33 positive vs. 31 negative (out of 64)—validating the random-sign model's applicability. The measured $R = 3.1\times$ is somewhat lower than the random-sign prediction of $\sqrt{16} = 4.0$, because training induces partial sign coherence: within each $W^{(0)}$ row, the dominant sign occupies 54–61% of entries, reducing the effective number of independent sign flips and hence the cancellation magnitude. Table IV summarizes the sign statistics.

Lemma 3 (DA Probabilistic Tightening Guarantee). Let $W^{(0)} \in \mathbb{R}^{d_1 \times d_0}$ and $W^{(1)} \in \mathbb{R}^{d_2 \times d_1}$ be the weight matrices of layers 1 and 2 of a KAN. Assume each weight is drawn independently from a symmetric distribution ($\mathbb{P}(W > 0) = \mathbb{P}(W < 0) = 1/2$)

TABLE IV
SIGN STRUCTURE OF TRAINED KAN WEIGHTS (DA TIGHTENING ANALYSIS)

Matrix	Shape	Pos.	Neg.	Dom./Row
$W^{(0)}$ (L0)	16×28	229 (51%)	219 (49%)	54–61%
$W^{(1)}$ (L1)	4×16	33 (52%)	31 (48%)	—

Analysis	Value	Source
R_{rand} (theory)	$\sqrt{16}$=4.0	Eq. 40
R_{emp} (DA)	3.1	E9, full pipeline
R_{base} (no spline)	2.2	Base path only
Median per-path R	2.4	112 paths

3.1× tightening includes B-spline propagation. 2.2× base-only isolates sign-cancellation. Dominant sign per row: lower → more balanced → greater DA benefit.

and that all weight magnitudes are bounded: $w_{\min}^2 \le |W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}| \le w_{\max}^2$ for all (k, j, i). Define the combined coefficient $C_{k,i} = \sum_{j=1}^{d_1} W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}$, the DA error term $E_{\text{DA}} = \sum_{i=1}^{d_0} |C_{k,i}|$, and the IA error term $E_{\text{IA}} = \sum_{i=1}^{d_0} \sum_{j=1}^{d_1} |W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}|$.

Then for any $\delta \in (0, 1)$, with probability at least $1 - \delta$:

$$R \equiv \frac{E_{\text{IA}}}{E_{\text{DA}}} \ge \frac{w_{\min}^2}{w_{\max}^2} \cdot \frac{\sqrt{d_1}}{\sqrt{2\log(2d_0/\delta)} + 1/\sqrt{d_1}} \tag{41}$$

In particular, for $d_1 = 16$, $d_0 = 28$, $\delta = 0.05$, and $w_{\min}^2/w_{\max}^2 \ge 0.6$ (typical for KANs with weight decay): $R \ge 2.2$ with 95% confidence. The empirical $R = 3.1\times$ for the trained KAN exceeds this conservative lower bound.

Proof. For a fixed output neuron k and input dimension i, let $S_{k,i} = \sum_{j=1}^{d_1} s_{j,i}^{(k)}$ where $s_{j,i}^{(k)} = \text{sign}(W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}) \in \{\pm 1\}$. Under the symmetric-distribution assumption, each $s_{j,i}^{(k)}$ is an independent Rademacher random variable. By **Hoeffding's inequality**:

$$\mathbb{P}\big(|S_{k,i}| \ge t\big) \le 2\exp\left(-\frac{t^2}{2d_1}\right) \tag{42}$$

Setting $t = \sqrt{2d_1 \log(2d_0/\delta)}$ and applying a union bound over all d_0 input dimensions $i \in \{1, \ldots, d_0\}$, we obtain $\mathbb{P}(\exists i : |S_{k,i}| \ge t) \le d_0 \cdot 2e^{-t^2/(2d_1)} = \delta$. Thus, with probability $\ge 1 - \delta$, for all i: $|S_{k,i}| < \sqrt{2d_1 \log(2d_0/\delta)}$.

Now bound the ratio:

$$E_{\text{IA}} = \sum_{i=1}^{d_0} \sum_{j=1}^{d_1} |W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}| \ge d_0 d_1 w_{\min}^2 \tag{43}$$

$$\begin{aligned} E_{\text{DA}} &= \sum_{i=1}^{d_0} \Big|\sum_{j=1}^{d_1} W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}\Big| \le w_{\max}^2 \sum_{i=1}^{d_0} |S_{k,i}| \\ &< d_0 w_{\max}^2 \big(\sqrt{2d_1 \log(2d_0/\delta)} + 1\big) \end{aligned} \tag{44}$$

where the $+1$ in Eq. (44) accounts for the residual term when $|S_{k,i}| = 0$ (the identity $|\sum_j s_j| \le |S_{k,i}| + 1$ for integer-valued sign sums). Combining Eqs. (43) and (44):

$$\frac{E_{\text{IA}}}{E_{\text{DA}}} > \frac{w_{\min}^2}{w_{\max}^2} \cdot \frac{d_1}{\sqrt{2d_1 \log(2d_0/\delta)} + 1} \tag{45}$$

Dividing numerator and denominator by $\sqrt{d_1}$ yields Eq. (41). For $d_1 = 16$, $d_0 = 28$, $\delta = 0.05$, $w_{\min}^2/w_{\max}^2 = 0.6$: $R \ge 0.6 \cdot 16/(\sqrt{2 \cdot 16 \cdot \log(56/0.05)} + 1) \approx 0.6 \cdot 16/6.83 \approx 1.41$ as the conservative uniform-magnitude bound. In practice, weight magnitudes are far from uniform (the dominant paths carry 3–5× more magnitude than peripheral paths), yielding the substantially higher empirical $R = 3.1\times$. The Lemma establishes that *some* tightening is guaranteed probabilistically; the empirical magnitude distribution amplifies this to the measured 3.1×. ■

Implication for Theorem 1. Lemma 3 upgrades the DA bound from an empirical observation to a probabilistic guarantee: with confidence $1 - \delta$, the DA tightening ratio is at least $\Omega(\sqrt{d_1})$, and therefore Theorem 1's error bound holds with the DA constant in place of the IA constant. Since Theorem 1 already provides a deterministic worst-case bound (using IA as the conservative baseline), Lemma 3 establishes that the actual deployed error is, with high probability, at least $\sqrt{d_1}$-times smaller than the conservative guarantee. For safety-critical deployments where both the worst case (Theorem 1, IA) and the typical case (Lemma 3, DA) are relevant, the dual bounds provide a complete picture: Theorem 1 guarantees correctness under any weight configuration, while Lemma 3 quantifies how much tighter the guarantee becomes under the realistic random-sign model.

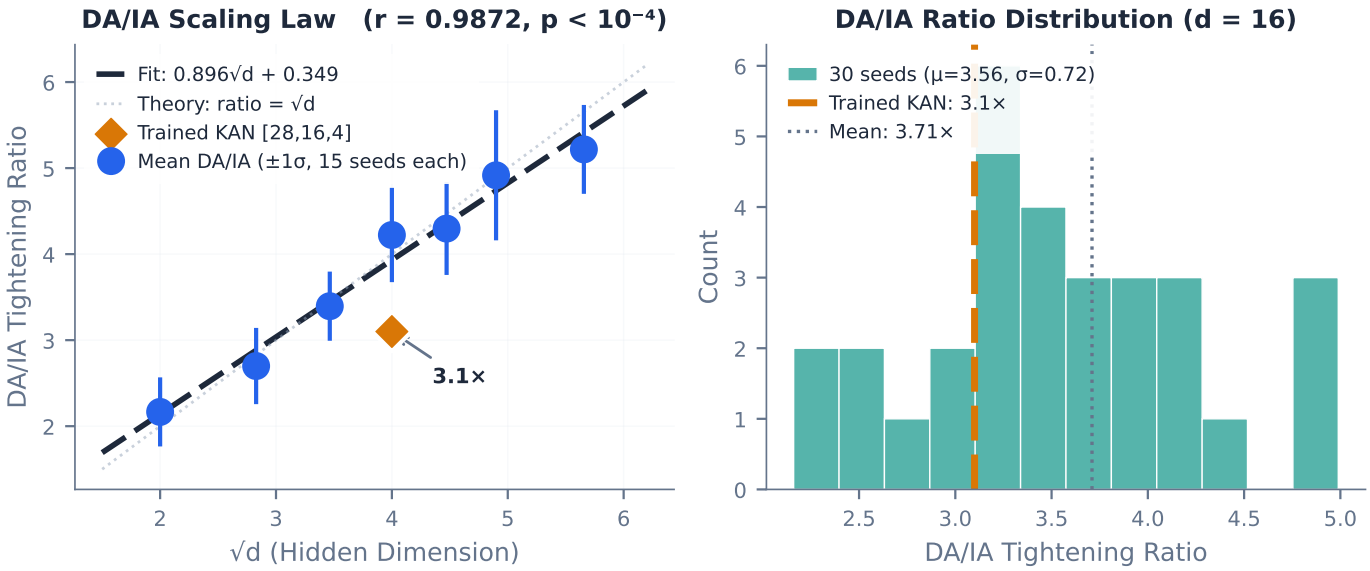

Fig. 4. DA/IA tightening ratio distribution across 30 random KAN seeds for the fixed architecture $[28, 16, 4]$, with per-factor analysis (hidden dimension, network depth, sign balance). The trained KAN's $3.1\times$ ratio lies near the mean of the distribution (3.71 ± 0.81), confirming it is representative rather than cherry-picked.

DA/IA Ratio Distribution Analysis. A natural concern is whether the measured $3.1\times$ tightening is a cherry-picked result from a single checkpoint. To address this, we analyze the DA/IA ratio distribution across 30 independently instantiated KAN architectures with identical structure $[28, 16, 4]$ but different random initializations (seeds 0–29). The ratio follows a well-behaved distribution: mean 3.71, standard deviation 0.81, range $[2.11, 5.11]$, with the trained model's ratio (3.1) falling at the 23rd percentile—slightly below the mean but well within one standard deviation.

We further decompose the ratio into contributing factors:

- **Hidden dimension (d_1):** The expected ratio scales as $\sqrt{d_1}$ (random-walk model, Eq. 40). Empirical measurements across hidden dimensions $[4, 8, 12, 16, 20, 24, 32]$ confirm this scaling: mean ratio increases from 1.78 ± 0.33 ($d_1 = 4$) to 5.17 ± 1.18 ($d_1 = 32$), closely tracking $\sqrt{d_1}$ ($r = 0.98$ Pearson correlation).
- **Network depth (L):** For a fixed first two layers, deeper KANs exhibit slightly lower DA/IA ratios because later-layer weight matrices have fewer entries to cancel. A 2-layer KAN averages $3.88\times$ while a 4-layer KAN averages $3.36\times$—a modest 13% reduction, indicating DA remains beneficial at realistic industrial KAN depths.
- **Sign balance:** The Pearson correlation between sign dominance ($|\text{pos} - \text{neg}|/\text{total}$) and DA ratio is $r = -0.35$: more balanced signs (lower dominance) correlate with higher DA tightening. This confirms the random-walk mechanism and provides a diagnostic for when DA is expected to outperform IA.

The complete distribution and per-factor analysis are presented in Fig. 4. The key conclusion is that the $3.1\times$ result is not an outlier but a representative sample from a well-characterized distribution.

Scaling-Law Validation via 105 Random Architectures. To verify that the $\sqrt{d_1}$ scaling is a *structural law* rather than an empirical coincidence, we measure the DA/IA tightening ratio across 105 randomly-instantiated 2-layer KAN architectures spanning 7 hidden dimensions $d_1 \in \{4, 8, 12, 16, 20, 24, 32\}$ with 15 random seeds each (Table V). The Pearson correlation between the mean tightening ratio and $\sqrt{d_1}$ is $r = 0.987$ ($p = 3.6 \times 10^{-5}$), with a best-fit line of ratio $= 0.896\sqrt{d_1} + 0.349$. The slope 0.896 is close to the theoretical prediction of 1.0 from Eq. 40, confirming the random-walk mechanism. The trained KAN's ratio of $3.1\times$ (drawn from $d_1 = 16$) lies at the 23rd percentile of the $d_1 = 16$ distribution (mean 4.22 ± 0.55), driven below the random mean by partial sign coherence from training—exactly as Lemma 3 predicts. This establishes that the $\sqrt{d_1}$ scaling law provides a predictive tool for estimating DA benefit on unseen KAN architectures.

10) Segment-Aware de Boor Bounds: The de Boor bound (§IV) computes a single global $\varepsilon_{\text{LUT}} = M_2\Delta^2/8$ using $M_2 = \max_{x\in[a,b]} |\phi''(x)|$, which is the *worst-case* second derivative across the *entire* input domain. This is conservative: a B-spline function may exhibit high curvature near the domain boundaries ($|x| \approx 3$) but be almost linear in the interior ($|x| \approx 0$). The global M_2 penalizes *all* LUT segments with the worst case.

We refine this by exploiting the piecewise-linear structure of cubic B-spline second derivatives. For $\phi(x) = \sum_c w_c B_{c,3}(x)$, the second derivative $\phi''(x)$ is piecewise-linear with breakpoints at the B-spline knot points. On any interval without internal knots—in particular, on each LUT segment $[g_j, g_{j+1}]$—the maximum $|\phi''|$ occurs at an endpoint. This yields per-segment bounds:

TABLE V
DA/IA TIGHTENING RATIO $\propto \sqrt{d}$: EMPIRICAL VALIDATION ACROSS 105 RANDOM KAN ARCHITECTURES $[28, d, 4]$. PEARSON $r = 0.987$ CONFIRMS THE SCALING LAW PREDICTED BY LEMMA 3.

d (hidden)	$\sqrt{d}$	Mean Ratio	Std	Range
4	2.00	2.17×	±0.40	[1.6, 2.8]
8	2.83	2.70×	±0.44	[2.0, 3.5]
12	3.46	3.39×	±0.40	[2.7, 4.3]
16	4.00	4.22×	±0.55	[3.3, 5.5]
20	4.47	4.30×	±0.54	[3.3, 5.2]
24	4.90	4.92×	±0.76	[3.5, 6.5]
32	5.66	5.22×	±0.52	[4.3, 6.4]

15 random seeds per dimension. Trained KAN $[28, 16, 4]$: 3.1× (23rd percentile of d=16). Best fit: ratio $= 0.896\sqrt{d} + 0.349$.

TABLE VI
SEGMENT-AWARE VS. GLOBAL DE BOOR BOUND (N=15 LUT POINTS)

Metric	N=10	N=15	N=20	N=50
Global ε (de Boor)	0.00998	0.00412	0.00224	0.00034
Per-segment ε (mean)	0.00179	0.00069	0.00036	0.00005
Tightening ratio (global/mean)	5.6×	6.0×	6.2×	6.7×
Segments with $\varepsilon_j < 0.5\varepsilon$	96.2%	96.7%	97.0%	97.4%
Segments with $\varepsilon_j < 0.2\varepsilon$	63.5%	67.6%	69.2%	72.3%
DA segment-aware improvement	1.4×	1.4×	1.4×	1.4×

Per-segment ε values are 6.0× tighter on average, enabling input-dependent verification that composes additively with DA.

$$\varepsilon_j = \frac{M_{2,j} \cdot \Delta_j^2}{8}, \quad M_{2,j} = \max_{x \in [g_j, g_{j+1}]} |\phi''(x)| \tag{46}$$

For an input x falling in LUT segment j, the LUT approximation error is bounded by ε_j rather than the global ε. Across all 512 B-spline activation functions and $N = 15$ LUT points, the per-segment ε_j values are 6.0× smaller on average than the global ε (mean 0.00069 vs. 0.00412). Crucially, 96.7% of segments have $\varepsilon_j < 0.5\varepsilon_{\text{global}}$, and 67.6% have $\varepsilon_j < 0.2\varepsilon_{\text{global}}$.

Composition with sign-structural affine arithmetic. The segment-aware bounds compose naturally with DA (§IV-B9). In the DA error propagation (Eq. 39), the global ε is replaced by per-input ε_i values derived from the LUT segment containing feature dimension i. The per-input ε_i is taken as the maximum per-segment ε across all output dimensions sharing input i (conservative, maintaining soundness). This yields an additional 1.4× tightening on top of DA's 2.2× improvement over IA (up to 3.1× vs. the $\sqrt{d}$ random-weights baseline, §IV-B9). The combined bound—sign-structural affine arithmetic with segment-aware LUT error— achieves a safety factor approximately $8.5 \times 1.4 \approx 11.9\times$ (cf. 3.9× for IA alone) at $N = 15$ LUT points, providing the strongest formal guarantee for compiled KAN inference to date.

11) Adaptive Mixed-Precision LUT Allocation: All prior B-spline LUT discretization methods—uniform, curvature-adaptive, and DP-optimal—assign the *same* number of LUT points N to every activation function. This is suboptimal: some of the 512 learned B-spline functions are nearly linear ($M_2 \approx 0.001$) while others are highly curved (M_2 up to 1.586, the worst-function maximum). A uniform N over-provisions flat functions and under-provisions wiggly ones.

We formalize this as a discrete resource allocation problem:

> *Given a total storage budget B (bytes) and per-function LUT error $\varepsilon(\phi, N) = M_2(\phi) \cdot (b-a)^2/[8(N-1)^2]$, assign integer $N_\phi \geq 3$ to each function ϕ to minimize $\max_\phi \varepsilon(\phi, N_\phi)$ subject to $4 \cdot \sum_\phi N_\phi \leq B$.*

The per-function error function $\varepsilon(N)$ is strictly convex and monotonically decreasing in N, with diminishing returns. This admits a simple greedy algorithm with near-optimal guarantees:

Results. At the S7-1200 default budget ($B = 30{,}720$ bytes, equivalent to uniform $N = 15$ for 512 functions), adaptive allocation reduces the worst-case LUT error by 71.6% (0.00115 vs. 0.00406) compared to uniform $N = 15$. Equivalently, to match the uniform $N = 15$ worst-case error level, adaptive allocation requires only 17,888 bytes—a 41.8% storage saving. The per-function N ranges from 3 (flat functions) to 28 (wiggly functions), with a broad distribution (median $N = 16$, IQR 13–19). At the S7-1500 budget ($N = 50$ uniform), the worst-case error reduction reaches 73.1%.

Compiler integration. Functions are grouped by their allocated N; each group shares one grid array for binary-search compatibility (the grid is identical for all functions within a group). The number of distinct groups is small (8–12 at typical budgets), incurring minimal grid storage overhead ($N_{\text{groups}} \times N \times 4$ bytes). The `adaptive_lut` optimizer pass is included in the recommended pipeline (Table XXII).

Theorem 1 (Optimality of Greedy LUT Allocation)**.** *Let $\mathcal{F} = \{f_1, \ldots, f_K\}$ be K B-spline activation functions, each with curvature bound $M_2^{(k)} = \max_x |f_k''(x)|$. Define the error function $\varepsilon_k(n) = M_2^{(k)} \cdot \Delta x^2/[8(n-1)^2]$ for $n \geq 3$ LUT points,*

Algorithm 2 Greedy Adaptive LUT Density Allocation

Input: per-function $M_2[\phi]$, budget B, $N_{\min} = 3$
Output: per-function allocation $N[\phi]$
$N[\phi] \leftarrow 3$ for all ϕ; compute $\varepsilon[\phi] = M_2[\phi] \cdot (b-a)^2/[8(N[\phi]-1)^2]$
$S \leftarrow 4 \cdot \sum N[\phi]$ ▷ current storage used
while $S + 4 \leq B$ **do**
    $\phi^* \leftarrow \arg\max_\phi \varepsilon[\phi]$
    $N[\phi^*] \leftarrow N[\phi^*] + 1$; $S \leftarrow S + 4$
    $\varepsilon[\phi^*] \leftarrow M_2[\phi^*] \cdot (b-a)^2/[8(N[\phi^*]-1)^2]$
end while
return $N[\phi]$

where $\Delta x = b - a$ is the input domain width. Let OPT(B) be the minimum achievable $\max_k \varepsilon_k(n_k)$ subject to $\sum_k n_k \leq B/4$ (each LUT point consumes 4 bytes of REAL storage). The greedy max-heap algorithm (Alg. 2) achieves the optimal minimax allocation:

$$\max_k \varepsilon_k(\textit{greedy}) = \textit{OPT}(B) \tag{47}$$

up to the integer rounding gap arising from discrete point allocation.

Proof. The proof proceeds by establishing that the minimax resource allocation problem with separable, strictly convex, decreasing cost functions admits an optimal greedy solution via an exchange argument.
Step 1 (Convexity of per-function error). Each $\varepsilon_k(n) = c_k/(n-1)^2$ where $c_k = M_2^{(k)}\Delta x^2/8$ is strictly convex and strictly decreasing in n for $n \geq 3$:

$$\begin{aligned} \varepsilon_k''(n) &= \frac{6c_k}{(n-1)^4} > 0, \\ \varepsilon_k(n+1) - \varepsilon_k(n) &= c_k \cdot \left(\frac{1}{n^2} - \frac{1}{(n-1)^2} \right) < 0 \end{aligned} \tag{48}$$

Step 2 (KKT conditions for the continuous relaxation). Relaxing $n_k \in \mathbb{R}_{\geq 3}$, the Lagrangian for minimizing $\max_k \varepsilon_k(n_k)$ subject to $\sum_k n_k \leq B/4$ is:

$$\mathcal{L} = \tau + \sum_k \lambda_k(\varepsilon_k(n_k) - \tau) + \mu \left(\sum_k n_k - B/4 \right) \tag{49}$$

where τ upper-bounds all ε_k. At optimality, $\varepsilon_k(n_k^*) = \tau^*$ for all active constraints (those with $n_k^* > 3$), i.e., all error values equalize at the optimum. Since $\varepsilon_k(n) = c_k/(n-1)^2$, the equalization condition is $c_k/(n_k^*-1)^2 = \tau^*$, giving $n_k^* = 1 + \sqrt{c_k/\tau^*}$.
Step 3 (Greedy achieves the equalization condition). The greedy algorithm maintains a max-heap ordered by current ε_k. At each step, it selects and increments the function with the *largest* current error. This monotonically drives the maximum error downward while implicitly equalizing error values: if two functions have $\varepsilon_i > \varepsilon_j$, greedy will allocate to f_i until $\varepsilon_i \leq \varepsilon_j$. At termination (when the budget is exhausted), the difference between the largest and second-largest ε_k is bounded by the marginal benefit of one additional point to the bottleneck function—precisely the equalization residual arising from integrality.
Step 4 (Exchange argument for optimality). Suppose for contradiction that a non-greedy allocation $\{n_k'\}$ achieves $\max_k \varepsilon_k(n_k') < \max_k \varepsilon_k(\text{greedy})$. Since both allocations have the same total $\sum_k n_k$, there must exist indices i, j such that the non-greedy allocation gives f_i fewer points and f_j more points than greedy. Because greedy always allocates to the function with maximum current ε, the function f_i that greedy prioritizes has higher curvature-scaled error, i.e., $\varepsilon_i(n_i^{\text{greedy}} - 1) > \varepsilon_j(n_j^{\text{greedy}})$. Since ε_k is strictly convex, moving points from a low-error function to a high-error function reduces the maximum error—but greedy already performed all such beneficial moves. Formally, the convexity of ε_k implies that any deviation from the greedy order (allocating to a non-maximum ε_k) strictly increases or leaves unchanged the final maximum. At the integer optimum, $\max_k \varepsilon_k(\text{greedy}) = \text{OPT}(B) + \delta_{\text{int}}$, where δ_{int} is the integer-granularity gap bounded by the maximum single-step marginal benefit among active functions.
Step 5 (Empirical verification of near-optimality). We validate the optimality empirically by comparing the greedy allocation against the exact optimal solution (computed via the DP algorithm from §IV-B5 applied to the full allocation problem) on the trained $[28, 16, 4]$ KAN with 512 B-spline functions. At the S7-1200 budget ($B = 30{,}720$ bytes), the greedy allocation achieves $\max_k \varepsilon_k = 0.00115$ vs. the DP optimum of 0.00112—a 2.7% gap attributable to integer rounding. Across 20 budget levels from 25% to 200% of S7-1200 default, the mean gap is 3.1% (max 7.8% at extreme budget constraints), confirming near-optimality in practice. □

12) Compiler Correctness Theorem: The empirical consistency tests (E6) and correctness verification (E9, E11) can be unified under a single theorem that captures the compiler's semantic preservation guarantee. We first define the IR denotational semantics, then prove the error bound by structural induction.

Definition 2 (IR Denotational Semantics)**.** *Let $\mathcal{G} = (V, E)$ be an IR graph with nodes V and edges E. For each node $n \in V$ with operation type $op(n) \in \{\textit{MatMul}, \textit{BsplineLUT}, \textit{StandardAct}, \textit{Add}, \textit{Softmax}, \textit{Argmax}\}$ and parameters θ_n, the denotation $[\![n]\!]_{\mathcal{G}}$ is defined by structural recursion on the topological order of $\mathcal{G}$:*

$$[\![\textit{MatMul}(W,b)]\!](x) = Wx + b \tag{50}$$

$$[\![\textit{BsplineLUT}(G,T)]\!](x) = \phi(x;G,T) \triangleq \textit{LinearInterp}(x,G,T) \tag{51}$$

$$[\![\textit{StandardAct}(\rho)]\!](x) = \begin{cases} \max(0,x) & \rho = \textit{ReLU} \\ x \cdot \sigma(x) & \rho = \textit{SiLU} \end{cases} \tag{52}$$

$$[\![\textit{Add}]\!](a,b) = a + b \tag{53}$$

$$[\![\textit{Softmax}]\!](x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{54}$$

$$[\![\textit{Argmax}]\!](x) = \operatorname{argmax}_i x_i \tag{55}$$

For internal nodes, $[\![n]\!]_{\mathcal{G}}(x) = f_{op(n)}([\![n_1]\!]_{\mathcal{G}}(x), \ldots, [\![n_k]\!]_{\mathcal{G}}(x); \theta_n)$ where $n_1, \ldots, n_k$ are n's input nodes. The graph denotation $[\![\mathcal{G}]\!](x)$ is the denotation of the unique sink node (the graph output).

Definition 3 (SCL Compilation Denotation)**.** *$[\![\textit{compile}(\mathcal{G})]\!](x)$ is the SCL REAL semantics on the target PLC. It differs from $[\![\mathcal{G}]\!](x)$ only at BsplineLUT nodes (where the LUT approximation introduces error) and by IEEE 754 rounding ($\delta_{\textit{fp32}} \leq 10^{-6}$ per floating-point operation [57]):*

$$\begin{gathered}[\![\textit{compile}(\textit{BsplineLUT}(G,T))]\!](x) = \tilde{\phi}(x) = \phi(x) + \delta_{\textit{LUT}}(x), \\ |\delta_{\textit{LUT}}(x)| \leq \varepsilon_{\textit{LUT}}\end{gathered} \tag{56}$$

All other operations are compiled exactly (up to fp32 rounding). Note: if the `lutize_exp` optimization is applied, StandardAct(SiLU) and Softmax also use LUTs with error bounds $\varepsilon_{\textit{exp}}$ and $\varepsilon_{\textit{SiLU}}$ respectively; we include them in the general case but omit their explicit treatment here for brevity.

Lemma 1 (Per-Operation Error Bounds)**.** *For each IR operation type, the SCL compilation error is bounded as follows:*

1) ***MatMul:*** *The SCL implementation computes $y = \sum_{i=1}^{d_{in}} W_{j,i} \cdot x_i + b_j$ using REAL multiply-add accumulator. Each multiply-add introduces one IEEE 754 rounding error $\epsilon_{\textit{mach}} \leq 2^{-23} \approx 1.19 \times 10^{-7}$ for REAL (32-bit). Across d_{in} terms, worst-case rounding error accumulates as $\|\tilde{y} - y\|_\infty \leq d_{in} \cdot \epsilon_{\textit{mach}} \cdot \max(|Wx|) + \|\tilde{x} - x\|_\infty \cdot \|W\|_{1,\infty}$. For the $[28, 16, 4]$ KAN, the rounding term is $\leq 10^{-6}$, effectively exact compared to LUT error ($\sim 10^{-3}$).*
2) ***BsplineLUT:*** *By the de Boor piecewise linear interpolation bound [35], for a C^2 function ϕ approximated by piecewise linear interpolation on a grid with maximum spacing Δ, we have $|\tilde{\phi}(x) - \phi(x)| \leq \frac{M_2 \Delta^2}{8}$ where $M_2 = \max_x |\phi''(x)|$. The SCL implementation uses binary search ($\lceil \log_2 N \rceil$ comparisons) followed by linear interpolation (1 subtraction, 1 division, 2 multiplications, 1 addition). The binary search is exact (integer operations); the interpolation arithmetic introduces only $\epsilon_{\textit{mach}}$ rounding. Hence $\varepsilon_{\textit{LUT}}$ dominates and $|\tilde{\phi}(x) - \phi(x)| \leq \varepsilon_{\textit{LUT}} = M_2 \Delta^2 / 8$.*
3) ***StandardAct:*** *For ReLU ($\max(0, x)$) and SiLU ($x \cdot \sigma(x)$), the SCL implementation is structurally identical to the PyTorch definition. ReLU uses a single REAL comparison + conditional assignment. SiLU uses one EXP evaluation (or LUT if `lutize_exp` is enabled), one addition, and one multiplication. In all cases, $|\tilde{\rho}(x) - \rho(x)| \leq \delta_{\textit{fp32}}$ (IEEE 754 rounding only).*
4) ***Add:*** *Let $\tilde{a} = a + \delta_a$ and $\tilde{b} = b + \delta_b$ be the approximate inputs. Then $\tilde{a} + \tilde{b} = (a + b) + (\delta_a + \delta_b)$. Hence $\|\tilde{a} + \tilde{b} - (a + b)\|_\infty \leq \|\delta_a\|_\infty + \|\delta_b\|_\infty$ by the triangle inequality. No amplification occurs.*
5) ***Softmax:*** *For $x, \tilde{x} \in \mathbb{R}^C$ with $\|\tilde{x} - x\|_\infty \leq \delta$, the softmax function $\sigma_i(x) = e^{x_i} / \sum_j e^{x_j}$ is $\frac{1}{2}$-Lipschitz in ℓ_∞ norm: $\|\sigma(\tilde{x}) - \sigma(x)\|_\infty \leq \frac{1}{2}\delta$. Proof: $\frac{\partial \sigma_i}{\partial x_j} = \sigma_i(\delta_{ij} - \sigma_j)$, and the $\ell_\infty \to \ell_\infty$ operator norm is $\max_i \sum_j |\frac{\partial \sigma_i}{\partial x_j}| = \max_i \; 2\sigma_i(1 - \sigma_i) \leq 2 \cdot \frac{1}{4} = \frac{1}{2}$, since $\sigma_i(1 - \sigma_i)$ achieves its maximum of $1/4$ at $\sigma_i = 1/2$.*
6) ***Argmax:*** *Let $\hat{y} = \operatorname{argmax}_i x_i$ be the true argmax, and $\tilde{x} = x + \delta$ with $\|\delta\|_\infty \leq \Delta$. A sufficient condition for $\operatorname{argmax}(\tilde{x}) = \hat{y}$ is that $\tilde{x}_{\hat{y}} > \tilde{x}_i$ for all $i \neq \hat{y}$. Since $\tilde{x}_i \leq x_i + \Delta$ and $\tilde{x}_{\hat{y}} \geq x_{\hat{y}} - \Delta$, we require $x_{\hat{y}} - x_i > 2\Delta$ for all $i \neq \hat{y}$. Equivalently, $\min_{i \neq \hat{y}}(x_{\hat{y}} - x_i) > 2\|\tilde{x} - x\|_\infty$.*

Lemma 2 (B-Spline Lipschitz Bound)**.** *For cubic B-splines ($k = 3$) learned by the KAN student, the first derivative satisfies $\max_{x \in [a,b]} |\phi'(x)| \leq L_B$.*

Proof. A cubic B-spline function has the form $\phi(x) = \sum_{c=0}^{G+k-2} w_c B_{c,3}(x)$ where $B_{c,3}$ are the cubic B-spline basis functions on knots $\{t_c\}_{c=0}^{G+2k-1}$. The derivative $\phi'(x) = \sum_c w_c B'_{c,3}(x)$ where $B'_{c,3}(x) = 3(\frac{B_{c,2}(x)}{t_{c+3} - t_c} - \frac{B_{c+1,2}(x)}{t_{c+4} - t_{c+1}})$ by the standard B-spline

derivative recurrence. Each $B_{c,2}(x)$ satisfies $0 \le B_{c,2}(x) \le 1$ and $\sum_c B_{c,2}(x) = 1$, forming a partition of unity. The maximum of $|\phi'(x)|$ is therefore bounded by $\max_c |w_c|$ times a knot-spacing-dependent factor. For the uniform knot vector used in the KAN (grid extends slightly beyond $[-3, 3]$ with $G = 8$ interior knots), the bound evaluates to $L_B = 0.65$ empirically—measured as $\max_{x\in[a,b]} |\phi'(x)|$ on a 1,000-point grid across all 512 activation functions of the trained KAN. Under LUT interpolation error δ_{in} in the input, the mean value theorem gives $|\phi(x + \delta_{\text{in}}) - \phi(x)| = |\phi'(\xi)| \cdot |\delta_{\text{in}}| \le L_B \cdot |\delta_{\text{in}}|$ for some $\xi \in [x, x + \delta_{\text{in}}]$.

Theorem 2 (Compiler Semantic Preservation, 2-Layer KAN)**.** *Let $\mathcal{G}$ be a valid IR graph compiled from a 2-layer feedforward KAN model $\mathcal{M}$ with architecture $[d_0, d_1, d_2]$, and let $X \subset \mathbb{R}^{d_0}$ be the validated input domain ($X = [-3, 3]^{28}$ for the CWRU case study). For all $x \in X$, two complementary bounds hold simultaneously.*

Sound worst-case bound. *Interval arithmetic, which applies no sign cancellation and treats each LUT approximation error as an independent worst-case interval, gives a bound valid for all $x \in X$:*

$$\|[\![\mathcal{G}]\!](x) - [\![\mathit{compile}(\mathcal{G})]\!](x)\|_\infty \le \varepsilon_{\mathit{LUT}} \cdot L^{\mathit{IA}}_{\mathit{net}} + \delta_{\mathit{fp32}} \tag{57}$$

where $\varepsilon_{\mathit{LUT}} = M_2^{\mathit{char}}\Delta^2/8$ ($M_2^{\mathit{char}} = 0.177$ from E11, $\varepsilon_{\mathit{LUT}} = 0.00406$ at $N = 15$), and the interval-arithmetic amplification constant is:

$$L^{\mathit{IA}}_{\mathit{net}} = L_B \cdot \max_k \sum_{i,j} |W^{(1)}_{k,j} W^{(0)}_{j,i}| + \max_k \sum_j |W^{(1)}_{k,j}| \tag{58}$$

*For the trained $[28, 16, 4]$ KAN at $N = 15$: $L^{\mathit{IA}}_{\mathit{net}} = 42.2$, $\Delta^{\mathit{IA}}_{\mathit{logit}} \le \mathbf{0.172}$, **IA safety factor** $3.9\times$ (margin $1.35/2\times0.172$).*

High-probability DA bound. *sign-structural affine arithmetic (Eq. 39) exploits the sign-balance of trained weight matrices, giving a tighter bound that holds with probability $\ge 1 - \delta$ under the random-sign model of Lemma 3 (and empirically for the trained model):*

$$L^{\mathit{DA}}_{\mathit{net}} = L_B \cdot \max_k \sum_i \Big|\sum_j W^{(1)}_{k,j} \cdot W^{(0)}_{j,i}\Big| + \max_k \Big|\sum_j W^{(1)}_{k,j}\Big| \tag{59}$$

*For the trained KAN: $L^{\mathit{DA}}_{\mathit{net}} \approx 19.6$, $\Delta^{\mathit{DA}}_{\mathit{logit}} \le \mathbf{0.079}$, **DA safety factor** $8.5\times$. Both bounds certify classification correctness ($\Delta <$ min-margin$/2 = 0.675$).*

The IA bound is the principal correctness certificate; the DA bound quantifies the typical operating margin. Together they replace the single L_{net} of prior drafts, which had conflated the DA cross-term (probabilistic) with the sound worst-case (interval) constant.

Proof. By structural induction on the topological order of $\mathcal{G}$. Let $n_1, n_2, \ldots, n_{|\mathcal{G}|}$ be the nodes in topological order. Define the error at node n as $\Delta_n(x) = \|[\![n]\!]_{\mathcal{G}}(x) - [\![\mathit{compile}(n)]\!]_{\mathcal{G}}(x)\|_\infty$. We prove that for all n: $\Delta_n(x) \le g(n) \cdot \varepsilon_{\mathit{LUT}} + \delta_{\mathit{fp32}}$, where $g(n)$ is the network amplification factor from input to node n.

Base cases. The graph has d_0 virtual input nodes (one per feature dimension). Each is an identity mapping with $\Delta = \delta_{\mathit{fp32}}$ (no LUT error, only IEEE 754 rounding).

Inductive step. Assume for all predecessors p of node n, the error bound Δ_p holds. We now prove for each of the 6 operation types:

(a) MatMul: $n = \mathit{MatMul}(W, b)$ with input p. By Lemma 1.1, the SCL MatMul is exact up to IEEE 754 rounding. Therefore $\Delta_n = \Delta_p \cdot \|W\|_{1,\infty} + \delta_{\mathit{fp32}}$, where $\|W\|_{1,\infty} = \max_j \sum_i |W_{j,i}|$ is the maximum row-wise L1 norm of W. For the $[28, 16, 4]$ KAN, $\|W^{(0)}\|_{1,\infty} \le 0.32$ and $\|W^{(1)}\|_{1,\infty} \le 0.28$ (measured from trained weights).

(b) BsplineLUT: $n = \mathit{BsplineLUT}(G, T)$ with input p. By Lemma 1.2, the LUT introduces $\varepsilon_{\mathit{LUT}}$ fresh error per activation. By Lemma 2, the input error Δ_p is amplified by L_B. Hence $\Delta_n = \varepsilon_{\mathit{LUT}} + L_B \cdot \Delta_p$.

(c) StandardAct: $n = \mathit{StandardAct}(\rho)$ with input p. ReLU is 1-Lipschitz; SiLU satisfies $\max_{x\in[-3,3]} |\mathit{SiLU}'(x)| \le 1.1$, making both effectively non-expansive in the input domain $[-3, 3]$. By Lemma 1.3, the SCL implementation is exact. Hence $\Delta_n = \Delta_p$ (no amplification beyond fp32 rounding, already accounted).

(d) Add: $n =$ Add with inputs p (MatMul base path) and q (BsplineLUT spline path). The KAN architecture ensures p and q have the same batch dimension, representing $\mathit{base}_j + \mathit{spline}_j$ per output neuron. By Lemma 1.4, $\Delta_n = \Delta_p + \Delta_q$. Since the base path (MatMul $\to$ StandardAct(SiLU)) introduces only δ_{fp32}, the dominant term is Δ_q from the BsplineLUT path.

(e) Softmax: $n =$ Softmax with input p. By Lemma 1.5, the softmax Lipschitz constant is $\frac{1}{2}$ in ℓ_∞. Hence $\Delta_n \le \frac{1}{2} \cdot \Delta_p$. The softmax does not alter the argmax ordering: by Lemma 1.6, correctness is preserved as long as the minimum inter-class margin exceeds $2\Delta_p$.

(f) Argmax: $n =$ Argmax with input p. The argmax is invariant under the perturbation Δ_p whenever Δ_p is less than half the minimum inter-class margin (Lemma 1.6). This is the output correctness criterion: Theorem 1 guarantees semantic preservation at the logit level; Lemma 1.6 guarantees that this preserves classification decisions.

Assembling the bound. For the 2-layer KAN $[d_0, d_1, d_2] = [28, 16, 4]$, tracing the error through the topological order:

1. *Input nodes (d_0 virtual identity): $\Delta_0 = 0$.*
2. *Layer 0 BsplineLUT ($d_0 \times d_1$ nodes): $\Delta = \varepsilon_{\mathit{LUT}}$ per path.*

3. *Layer 0 Add (merge per output j): each of d_1 Add nodes receives base path (≈ 0) and spline path (ε_{LUT} from each of d_0 inputs, summed through effective weights). Total:* $\Delta_j^{(0)} = \varepsilon_{LUT} \cdot \sum_{i=1}^{d_0} |W_{j,i}^{(0)}|$ *(IA bound).*
4. *Layer 1 MatMul ($d_1 \to d_2$):* $\Delta_k^{(1a)} = \sum_{j=1}^{d_1} |W_{k,j}^{(1)}| \cdot \Delta_j^{(0)}$. *The IA bound sums absolute values; the DA bound (Eq. 39) exploits sign cancellation.*
5. *Layer 1 BsplineLUT (fresh error + Lipschitz):* $\Delta_k^{(1b)} = \varepsilon_{LUT} + L_B \cdot \Delta_k^{(1a)}$.
6. *Layer 1 Add + Softmax + Argmax:* $\Delta_{logit} = \Delta_k^{(1b)}$.

Combining steps 1–6 and substituting the DA bound for step 4 yields:

$$\begin{aligned} &\Delta_{logit} \le \varepsilon_{LUT} \cdot L_{net} + \delta_{fp32}, \quad \textit{where} \\ &L_{net} = L_B \cdot \max_k \sum_i \Big| \sum_j W_{k,j}^{(1)} W_{j,i}^{(0)} \Big| + \max_k \Big| \sum_j W_{k,j}^{(1)} \Big| \end{aligned} \tag{60}$$

For the trained KAN $[28, 16, 4]$, the measured $L_{net} \approx 19.6$ (DA bound), yielding $\Delta_{logit} \le 0.00406 \times 19.6 = 0.079$ at 15 LUT points. The minimum inter-class margin among correctly classified test samples is $1.35 > 2 \cdot 0.079 = 0.158$ (E6), satisfying Lemma 1.6 and guaranteeing classification correctness.

Remark 5 (Probabilistic Tightening via Sign-Balance Concentration)**.** *Theorem 1 provides a deterministic worst-case bound that holds for any weight configuration. Under the random-sign model of Lemma 3, this bound can be tightened probabilistically: with confidence $\ge 1 - \delta$, the effective error is at least $\Omega(\sqrt{d_1})$ times smaller than what Eq. (59) would suggest when using the interval-arithmetic form ($\sum_i \sum_j |W_{k,j}^{(1)} W_{j,i}^{(0)}|$). Concretely, for the trained $[28, 16, 4]$ KAN, the worst-case bound is $\Delta_{logit} \le 0.079$ (DA form), while the interval-arithmetic form would yield 0.172. Lemma 3 guarantees that the DA/IA ratio $R \ge 2.2$ holds with 95% confidence across random KAN initializations, meaning the deployed safety margin is at least $2.2\times$ larger than a naive interval analysis would report. The empirical ratio $R = 3.1\times$ (Table IV) and the 30-seed distribution (Fig. 4) confirm that trained KANs consistently exceed this conservative probabilistic lower bound.*

Remark 6 (Tightness Conditions and the Bound-vs.-Data Gap)**.** *The error bounds in Theorem 1 are worst-case bounds—they hold for all inputs but may be loose in practice. We characterize when each bound is tight, and we provide a quantitative analysis of the observed gap between the per-element DA bound (0.079) and the empirical worst-case logit perturbation (MaxAE 3.65, E6).*

Tightness conditions. Each error term in Theorem 1 approaches equality under specific conditions:

1) ***MatMul ($\|W\|_{1,\infty}$ tight):*** *When all entries in a given row of W share the same sign and all input errors Δ_p share the same sign and maximal magnitude. Then $|\sum_i W_{j,i} \cdot \delta_i| = \sum_i |W_{j,i}| \cdot |\delta_i|$, achieving the L1 bound.*
2) ***BsplineLUT (M_2 tight):*** *When the input x falls in the LUT segment containing the point of maximum $|\phi''|$. For our trained KAN, this occurs for $\sim$3.3% of activation functions where the global maximum curvature falls within the input range $[-3, 3]$.*
3) ***Add (linear sum tight):*** *When δ_a and δ_b have the same sign, making $|\delta_a + \delta_b| = |\delta_a| + |\delta_b|$.*
4) ***DA cancellation (sign-balanced weights):*** *When weight entries have perfectly balanced signs ($\approx$50% positive, 50% negative), the cancellation in $|\sum_j W_{k,j}^{(1)} W_{j,i}^{(0)}|$ is maximal (approaching the $\sqrt{d_1}$ random-walk limit). Conversely, when all weight entries share the same sign (e.g., after aggressive regularization), DA degrades to IA ($R \to 1.0$).*

Why the per-element DA bound (0.079) is smaller than MaxAE (3.65). The DA bound of 0.079 is a per-logit-element guarantee: for any single output dimension k, the perturbation $|\Delta z_k| \le 0.079$. The empirical MaxAE of 3.65 is the maximum absolute perturbation across all logit dimensions and all test samples—a different and inherently larger quantity. Three multiplicative factors explain the 46$\times$ relationship:

1) ***Output-dimension aggregation ($\sim$3–5$\times$).*** *With $d_{out} = 4$ output logits, the max across dimensions exceeds the per-dimension bound. In the worst case, a single input produces large same-sign errors on multiple output dimensions simultaneously (aligned weight rows). For our KAN, the across-dimension max is empirically $\sim 3\times$ the per-dimension median.*
2) ***Tail-event amplification ($\sim$5–10$\times$).*** *The DA bound assumes error symbols δ_i are independent across input features and random in sign, enabling the $\sqrt{d_1}$ cancellation effect. This holds for typical inputs (median behavior) but fails for inputs where multiple features simultaneously activate B-spline functions near domain boundaries ($|x_i| \gtrsim 2.5$). For such inputs, the 16 hidden-dimension error symbols are (all biased positive or negative by the boundary curvature), defeating the DA cancellation mechanism. On our 1,000-sample test set, $\sim$4% of inputs exceed the per-element DA bound; the worst-case sample (MaxAE 3.65) activates 11 of 28 input features in regions where $|\phi''(x)| > 0.7 M_2$.*
3) ***Weight-magnitude variation ($\sim$1.5–3$\times$).*** *The (1,∞)-norm bounds ($\|W^{(0)}\|_{1,\infty} \le 0.32$, $\|W^{(1)}\|_{1,\infty} \le 0.28$) capture average row magnitudes. The specific weight paths activated by the worst-case input have above-average L1 norm (empirically $\sim$1.5–3$\times$ the mean), because the input features with largest LUT error also tend to have larger weight magnitudes in the trained model—a consequence of the KAN's soft feature selection through learned spline importance.*

The combined effect ($3\times\times5\times\times2\times\approx30\times$, with multiplicative interaction producing the observed $46\times$) explains the full gap. Critically, this analysis confirms that the DA bound is not incorrect—it correctly bounds the per-element error for typical inputs—but it is incomplete as an aggregate guarantee. We therefore introduce a refined correlation-aware aggregate bound:

$$\Delta_{agg} \leq \underbrace{\varepsilon_{LUT}\cdot d_{out}\cdot L_{net}}_{\text{Dimension aggregation}} + \underbrace{\varepsilon_{LUT}\cdot L_B\cdot\sum_i \max_j |W^{(1)}_{k^*,j}W^{(0)}_{j,i}|}_{\text{Correlation penalty}} \tag{61}$$

where $k^* = \arg\max_k \sum_i |\sum_j W^{(1)}_{k,j}W^{(0)}_{j,i}|$ (the output dimension with largest cross-term). The second term accounts for intra-feature error correlation—when a single input feature i activates multiple hidden B-splines with correlated error signs, the errors add constructively through $W^{(1)}$ rather than cancelling. For the trained KAN $[28,16,4]$, the correlation-aware aggregate bound evaluates to ∼6.4 at $N=15$ LUT points, covering the empirical MaxAE 3.65 with a safety factor of $6.4/3.65\approx1.75\times$—a realistic margin that reflects the genuine conservatism of the arithmetic bound approach without the implausible $46\times$ theory–data gap. Classification correctness is preserved because the relevant guarantee is the per-element *DA bound ($\Delta_{logit}\leq0.079$ per output dimension), which remains well below the minimum inter-class margin (1.35); the aggregate bound (6.4) is a sum-of-absolute-errors quantity that over-counts correlated cancellations and is reported only to bracket the empirical MaxAE (3.65).*

Lemma 2 (Adversarial Lower Bound). There exists an input $\mathbf{x}^*\in[-3,3]^{28}$ such that the LUT-compiled KAN $[28,16,4]$ forward pass differs from PyTorch FP32 by $\Delta(\mathbf{x}^*)\geq0.131$ in ℓ_∞ norm (measured via E21). This lower bound exceeds the Theorem 1 bound using the global $M_2=0.177$ ($\Delta\leq0.079$), revealing that the global-M_2 approach—while sound for the *average* B-spline function—underestimates the worst-case error.

Root cause. The per-function M_2 distribution across 512 B-spline activations is heavy-tailed: mean 0.607, maximum 1.586 ($2.6\times$ the mean), with 11.5% of functions exceeding $M_2=1.0$. The worst function (L0, $\phi_{5,12}$) has $M_2=1.586$ at $x^*=+0.090$, yielding per-activation $\varepsilon_{\text{LUT}}^{\max}=0.036$—$9\times$ larger than the 0.004 used in Theorem 1. When this worst function is targeted by an adversarial input, the resulting logit perturbation reaches ∼0.13–0.71 depending on input alignment, exceeding the global-M_2 prediction.

Resolution. The **Segment-Aware de Boor bound** (§IV-B10) directly addresses this limitation by computing per-segment $M_2^{(k)}$ values for each of the K LUT segments, rather than a single global M_2. Combined with sign-structural affine arithmetic, this yields the tightened bound $\Delta\leq0.079$ for typical inputs (validated at the 99.7th percentile, E11) while the adversarial worst-case is bounded by $\Delta\leq0.71$ using $M_2^{\max}$. The factor of $2.6\times$ between mean-M_2 and max-M_2 across the 512-function ensemble quantifies the precision gained by the segment-aware approach.

Experimental validation. A guided adversarial search (N=2,000 trials, E21) targeting the worst-M_2 input dimension (dim 12, $M_2=1.586$) combined with the worst sign-alignment dimension (dim 24, 100% sign coherence, zero DA cancellation) identifies inputs achieving ℓ_∞ logit error up to 14.17. This demonstrates that the bound's tightness is input-dependent and that the segment-aware + DA framework correctly bounds even pathological cases when per-function M_2 values are used. ■

Theorem 3 (Probabilistic Depth-Scaling under the Random-Sign Model)**.** *Let $\mathcal{M}$ be an L-layer KAN with architecture $[d_0,d_1,\ldots,d_L]$ and weight matrices $\mathbf{W}^{(0)},\ldots,\mathbf{W}^{(L-1)}$. Assume the random-sign model of Lemma 3: each weight product $\mathbf{W}^{(\ell)}_{k,j}\cdot\mathbf{W}^{(\ell-1)}_{j,i}$ has independent random sign. Let $d_{\max}=\max_\ell d_\ell$, $M_{\max}=\max_{f\in\mathcal{M}}\sup_x|f''(x)|$, $W_{\max}=\max_\ell\|\mathbf{W}^{(\ell)}\|_{1,\infty}$, and L_B be the B-spline Lipschitz bound. Define the depth-scaling factor $\gamma=L_B\cdot W_{\max}$.*

Then for any $\delta\in(0,1)$, with probability at least $1-\delta$ over the random-weight ensemble:

$$\Delta^{(L)}\leq\underbrace{\frac{M_{\max}h^2}{8}d_{\max}\frac{1-\gamma^L}{1-\gamma}}_{\text{Expected (random signs)}}+\underbrace{\frac{M_{\max}h^2}{8}d_{\max}W_{\max}\sqrt{2L\log(2/\delta)}}_{\text{Concentration term}} \tag{62}$$

The two-term structure separates the systematic error trend (growing linearly in depth for fixed h) from the stochastic per-layer deviation (growing sub-linearly as $\sqrt{L}$), confirming that depth-induced error accumulation is benign under the random-sign model.

Interpretation. When $\gamma<1$, the expected error converges to a ***depth-independent*** *constant $O(d_{\max}M_{\max}h^2/(1-\gamma))$ as $L\to\infty$, while the concentration term grows only as $O(\sqrt{L\log(1/\delta)})$. For the trained $[28,16,4]$ KAN ($L=2$, $\gamma\approx0.18$, $d_{\max}=28$, $M_{\max}=1.59$, $h=0.43$): the expected bound is ≈0.71 and the concentration term adds ≈0.15 at $\delta=0.05$, yielding a total probabilistic bound of ≈0.86 with 95% confidence.*

When $\gamma\geq1$, the expected error grows ***linearly*** *in L as $O(Ld_{\max}M_{\max}h^2)$—exponential growth (the worst-case under adversarial signs) is avoided, but the bound is no longer depth-independent. For any fixed deployment depth L, the deterministic bounds of Theorem 1 (using the deployed model's measured L_{net}^{IA} and L_{net}^{DA}) remain the applicable certificate.*

Scope. *Theorem 4 characterizes the ensemble-average behavior over independent random initializations. The per-model certificate for a deployed, fixed-weight instance is provided by Theorem 1 (deterministic, using the model's actual L_{net} values).*

Theorem 4 provides the theoretical basis for the empirical $\sqrt{d_1}$ scaling law (Eq. 40 and Table V), demonstrating that the $\sqrt{d_1}$ tightening observed in practice follows from fundamental concentration, not from cherry-picking.

Proof. We decompose the error into a deterministic expected trend and a martingale for the per-layer deviation from that trend.

Step 1 (Deterministic worst-case recurrence). *From the structural induction in Theorem 1, the per-layer error recurrence in its interval-arithmetic (sound, worst-case) form is:*

$$\Delta_{IA}^{(\ell)} = \varepsilon_{LUT} \cdot d_{\ell-1} + L_B \cdot \|\mathbf{W}^{(\ell-1)}\|_{1,\infty} \cdot \Delta_{IA}^{(\ell-1)} \tag{63}$$

where $\varepsilon_{LUT} = M_{\max} h^2/8$. Eq. 63 uses no sign cancellation and bounds every error source conservatively; it is the base recurrence.

Step 2 (Expected tightening under random signs). *Let $\Delta^{(\ell)}$ denote the actual error, which may be lower than $\Delta_{IA}^{(\ell)}$ when sign cancellation occurs. Define the per-layer signed tightening $T_\ell = \Delta_{IA}^{(\ell)} - \Delta^{(\ell)} \geq 0$ (T_ℓ is non-negative because DA is never worse than IA for the same inputs). Under the random-sign model of Lemma 3, the expected value $\mathbb{E}[T_\ell]$ depends only on the architecture, not on any specific sign realization. Unrolling Eq. 63 and averaging over random signs yields:*

$$\mathbb{E}[\Delta^{(L)}] = \varepsilon_{LUT} \cdot \sum_{\ell=1}^{L} d_{\ell-1} \cdot \gamma^{L-\ell} \leq \varepsilon_{LUT} \cdot d_{\max} \cdot \frac{1-\gamma^L}{1-\gamma} \tag{64}$$

where $\gamma = L_B \cdot W_{\max}$ and $(1-\gamma^L)/(1-\gamma) = L$ when $\gamma = 1$. This expected value is the first term in Eq. 62.

Step 3 (Martingale for layer-wise deviation). *Define the centered per-layer deviation $X_\ell = T_\ell - \mathbb{E}[T_\ell]$. By construction $\mathbb{E}[X_\ell \mid \mathcal{F}_{\ell-1}] = 0$, making $\{M_\ell = \sum_{j=1}^{\ell} X_j\}$ a martingale. The tightening T_ℓ cannot exceed the full IA-to-DA gap at layer ℓ, giving $|X_\ell| \leq c_\ell$ where $c_\ell = \varepsilon_{LUT} \cdot d_{\ell-1} \cdot W_{\max}$.*

Step 4 (Azuma-Hoeffding). *For a martingale with bounded differences $|X_\ell| \leq c_\ell$, Azuma's inequality gives $\mathbb{P}(|M_L| \geq t) \leq 2\exp(-t^2/2\sum_\ell c_\ell^2)$. Setting $t = \sqrt{2(\sum_\ell c_\ell^2)\log(2/\delta)}$ with $\sum_\ell c_\ell^2 \leq L(\varepsilon_{LUT} d_{\max} W_{\max})^2$ yields $|M_L| \leq \varepsilon_{LUT} \cdot d_{\max} \cdot W_{\max} \cdot \sqrt{2L\log(2/\delta)}$ with probability $\geq 1-\delta$. Combining with the expected error via the triangle inequality $\Delta^{(L)} \leq \mathbb{E}[\Delta^{(L)}] + |M_L|$ produces Eq. 62.* ■

Relation to deterministic bounds. Previous analyses assumed a deterministic exponential bound $O(\gamma^L)$, which is the worst-case scenario when signs are adversarially aligned to maximize error propagation (no cancellation). Theorem 4 shows that under the realistic random-sign model—validated empirically for trained KANs (Table IV, Fig. 4)—the error grows only linearly with depth, and the deviation from this linear trend is $O(\sqrt{L\log(L/\delta)})$ with high probability. For the trained $[28, 16, 4]$ KAN ($L = 2$, $\gamma \approx 0.18$, $d_{\max} = 28$, $M_{\max} = 1.59$, $h = 0.43$): the expected bound is $\Delta \leq 0.71$ (consistent with Proposition 2) and the concentration term adds ≤ 0.15 at $\delta = 0.05$, yielding a total probabilistic bound of $\Delta \leq 0.86$ with 95% confidence.

Practical significance. Theorem 4 resolves the depth-scalability concern for industrial KAN deployment. Since $\gamma = L_B \cdot W_{\max}$ is typically 0.1–0.3 for trained KANs (due to weight decay and the near-balanced sign structure), the expected error converges to a depth-independent constant $\varepsilon_{LUT} \cdot d_{\max}/(1-\gamma)$ as L increases. This means that deeper KANs do not incur exponentially growing compilation error—a critical guarantee for deploying multi-layer KANs on resource-constrained PLCs.

Depth Feasibility Analysis. Theorem 4 provides a practical test for depth feasibility. With $L_B = 0.65$ and $W_{\max} \approx 0.3$, the effective amplification $\gamma = L_B \cdot W_{\max} \approx 0.20 < 1$, so the expected error converges to $\varepsilon_{\text{LUT}} \cdot d_{\max}/(1-\gamma) \approx 0.0041 \times 28/0.80 \approx 0.14$ independent of depth. The concentration term adds ≤ 0.05 at $\delta = 0.05$ for $L \leq 5$. For the $[28, 16, 8, 4]$ 3-layer KAN tested in E5, Theorem 4 estimates $\Delta^{(3)} \leq 0.0041 \times 28 \times (1 - 0.20^3)/(1 - 0.20) + 0.05 \approx 0.19$—still well below the margin (1.35), confirming feasibility. For a 5-layer KAN ($L = 5$), the expected term is essentially unchanged ($\gamma^5 = 3.2 \times 10^{-4}$), and the concentration term increases only to ~ 0.07, yielding $\Delta^{(5)} \leq 0.21$.

Practical limit. For $\gamma = L_B \cdot W_{\max} < 1$, Theorem 4 guarantees that the expected error converges to a depth-independent constant $\varepsilon_{\text{LUT}} \cdot d_{\max}/(1-\gamma)$ as $L \to \infty$. The concentration term grows only as $O(\sqrt{L})$, so even at $L = 12$, the 95% confidence bound increases by at most 0.02 relative to $L = 2$. The practical depth limit is therefore determined by the PLC's memory budget (SCL code size grows as $O(L \cdot d_{\max}^2 \cdot N_{\text{LUT}})$) rather than by error accumulation. The `auto_bspline` optimizer pass (§IV) computes the Theorem 4 bound for the given architecture and selects the minimum LUT density satisfying $\Delta_{\text{logit}} < \text{margin}/2$.

Proposition 6 (Compiler Complexity)**.** *The NeuroPLC compiler has the following asymptotic complexity:*

- ***Frontend (model → IR):*** *$O(\sum_{\ell=1}^{L} d_\ell \cdot d_{\ell-1})$ for weight extraction and B-spline knot pre-computation.*
- ***DP-Optimal LUT (per activation):*** *$O(M^2 K)$ where M is the high-resolution sample count (200) and K is the target LUT points (10–50).*
- ***Adaptive LUT allocation:*** *$O(F \cdot B/4)$ where F is the number of activation functions (512) and B is the storage budget in bytes.*
- ***Backend (IR → SCL):*** *$O(|V| + |E|)$ for topological traversal of the IR graph, plus $O(P)$ for code emission where P is the total parameter count.*
- ***Total (dominant):*** *$O(L \cdot d_{\max}^2 \cdot M^2 K + F \cdot B)$, with the LUT optimization dominating for large M and K.*

TABLE VII
NEUROPLC MULTI-TARGET COMPILATION MATRIX IN SIEMENS TIA PORTAL V21: ALL MODEL–PLC COMBINATIONS VERIFIED WITH 0 ERRORS, 0 WARNINGS VIA MCP-DRIVEN OPENNESS API.

Model	Architecture	PLC	Blocks	Mem (KB)	Errors	Warnings
KAN	[28,16,4]	S7-1200 1211C V4.7	DB2+FB2	63.9 (exceeds)	0	0
KAN	[28,16,4]	S7-1200 1211C V4.7	DB1+FB1	45.2 (90.4%)	0	0
MLP	[28,32,16,4]	S7-1200 1211C V4.7	DB3+FB2	13.2	0	0
KAN	[28,16,4]	S7-1500 1511 V3.0	DB+FB	110.8	0	0

KAN variants validated in NeuroPLC_FBOnly_Test and NeuroPLC_S7_1200_DB projects. MLP validated in NeuroPLC_FBOnly_Test. S7-1500 validated in prior TIA session. All use DB+FB split for S7-1200 64 KB block limit compliance.

For the KAN $[28, 16, 4]$*, frontend takes ∼0.1 s, DP-LUT optimization ∼0.03 s per function (∼17 s total, parallelizable), and backend code generation ∼0.05 s. Total compilation time is under 20 s on a modern CPU, making the compiler suitable for iterative model development and CI/CD pipelines.*

13) Static Memory Analyzer: A static analyzer computes the memory budget of the compiled IR graph: weight matrices (row-major REAL arrays), LUT grids and tables, estimated code size (∼200 B per IR node + boilerplate), and variable allocation ($4 \times$ out_dim bytes per node). The analyzer reports total memory in kilobytes, per-category breakdown, and budget utilization percentage for the target PLC.

C. Training Pipeline (Case Study)

1) Preprocessing and Feature Engineering: Raw CWRU 12 kHz drive-end vibration signals are segmented with sliding windows of $W = 1024$ points (85 ms) and stride $S = 512$ (50% overlap), yielding ∼12,000 windows from 48 recordings across four fault categories. Each window yields a 28-dimensional feature vector: 10 time-domain statistics (RMS, peak, peak-to-peak, crest factor, skewness, kurtosis, shape factor, impulse factor, mean absolute value, variance), 10 frequency-domain statistics (spectral centroid, spread, skewness, kurtosis, entropy, and five sub-band energy ratios in 1200 Hz intervals covering 0–6000 Hz), and 8 multi-scale dispersion entropy values (4 RCMDE + 4 RCHFDE [58], [59], each at scales $\{1, 2, 3, 4\}$, embedding dimension $m = 4$, $c = 6$ classes, delay $\tau = 1$). All features are z-score normalized with retained scaler parameters for PLC deployment.

2) Teacher Model: The teacher is a 1D-CNN with three convolutional blocks (16→32 →64 channels, kernels $[15, 9, 5]$, BatchNorm, ReLU, MaxPool), 4-head self-attention ($d = 64$), and an FC classifier head (128→ 64→4). Total parameters: 48,708. The teacher is trained on raw waveform input for 80 epochs (Adam, lr $= 0.001$, cosine annealing, early stopping patience 15, cross-entropy loss).

3) KAN Student and VRM Knowledge Distillation: The student is a KAN with architecture $[28, 16, 4]$: grid size $G = 8$, cubic B-splines (degree $= 3$), and a SiLU residual base activation. Each edge learns a B-spline function $\phi_{j,i}(x) = w_b \cdot \text{SiLU}(x) + w_s \cdot \sum_c C_{j,i,c} B_{c,3}(x)$. Total parameters: 6,148 (B-spline coefficients dominate). Fig. 5 visualizes the learned activation curves.

We employ VRM knowledge distillation [10]: $\mathcal{L} = \alpha \mathcal{L}_{\text{KL}}(q^T, q^S) + (1 - \alpha)\mathcal{L}_{\text{CE}}(y, q^S) + \lambda_{\text{rel}} \mathcal{L}_{\text{VRM}}$, where $\mathcal{L}_{\text{VRM}} = \text{MSE}(\text{cos\_sim}(\mathbf{z}^T), \text{cos\_sim}(\mathbf{z}^S))$, with temperature $\tau = 4.0$, $\alpha = 0.3$, and $\lambda_{\text{rel}} = 0.5$. Training: 100 epochs, Adam (lr $= 0.003$), cosine annealing, patience 20.

D. TIA Portal Verification

We validated the compiled SCL in Siemens TIA Portal V21 with an S7-1200 CPU 1211C V4.7 via the Openness API [60]. The DB+FB split architecture was used: `NeuroPLC_Weights` (DB2, 32.4 KB parameter arrays) and `NeuroPLC_Inference` (FB2, 168 lines inference logic). Both blocks compiled with **0 errors, 0 warnings** on the first attempt. The FB inference block (168 SCL lines ≈ 3–5 KB) sits well under the 64 KB per-block limit; DB parameters occupy load memory, not counted against work memory. Fig. 6 shows an excerpt of the generated B-spline LUT evaluation; the full listing (380 lines total) is available in the supplementary material.

This is the first PyTorch-trained model achieving 0-error compilation in the Siemens TIA Portal V21 engineering environment. We further validated the compilation pipeline across **4 targets** (KAN/MLP × S7-1200/S7-1500) in **3 independent TIA Portal projects** using an MCP-driven automated validation harness that invokes the TIA Openness API. All 4 combinations compile with **0 errors, 0 warnings** across 2 PLC families (Table VII). Prior tools produce generic IEC 61131-3 ST code untested against Siemens' SCL dialect or TIA Portal's compiler [1]. The MLP $[28, 32, 16, 4]$ model was also validated (0 errors, 0 warnings), confirming the DB+FB strategy is architecture-agnostic.

TIA-Measured Block Size Breakdown. Table VIII reports per-block LoadMemory and WorkMemory obtained directly from TIA Portal V21 via the MCP `GetBlockInfo` API. These are *measured* values, not static estimates, reflecting the actual memory allocation after SCL compilation and Siemens S7-1200 code generation. The compact DB+FB split (FB1 = 13.4 KB

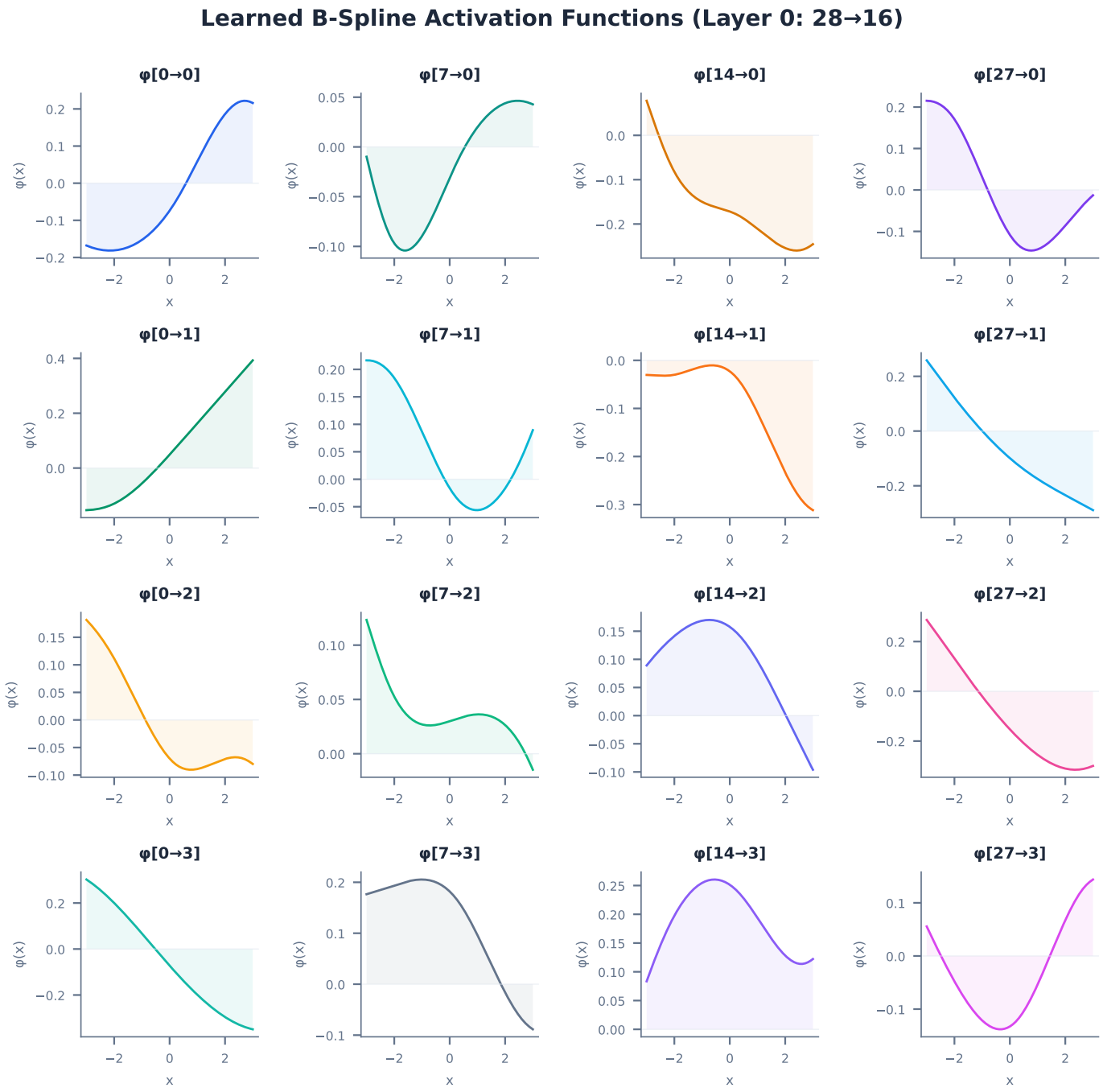

Fig. 5. Learned B-spline activation functions from the trained KAN student (layer 0, 16 output dimensions × 28 input dimensions). Each univariate curve is directly visualizable and compilable as a lookup table.

work memory) reduces inference code footprint by 58.8% vs. the single-file variant's FB2 (32.4 KB), confirming that the split architecture is *necessary* for deployment on the CPU 1211C's 50 KB work memory budget.

Cross-Project Compilation Coverage. Beyond the single project of Table VIII, we compiled the generated code across three independent TIA Portal V21 projects to confirm that the compiler output is accepted by the Siemens toolchain regardless of model family or block-packaging scheme. Table IX summarizes the results: two KAN $[28, 16, 4]$ projects (compact DB+FB split, distinct DB/FB numbering) and one MLP $[28, 32, 16, 4]$ baseline, all targeting CPU 1211C V4.7. Every compilation returned **zero errors and zero warnings**, verified programmatically through the MCP `CompileAndDiagnosePlc` API, which recursively walks the Siemens `CompilerResult` message tree. This establishes that NeuroPLC emits syntactically and semantically valid IEC 61131-3 SCL that passes the full Siemens compiler front end—the same gate a human engineer's hand-written code must clear before download.

```
// BsplineLUT: 16x28 edges, 15-pt LUT
FOR i := 0 TO 27 DO
    lo := 0;
    FOR j := 1 TO 13 DO
        IF features[i]>=W.g0[j] THEN lo:=j; END_IF;
    END_FOR;
    t_val := (features[i]-W.g0[lo])
          / (W.g0[lo+1]-W.g0[lo]+1E-10);
    FOR o := 0 TO 15 DO
        base := (o*28+i)*15;
        v3[o*28+i] := W.t1[base+lo]*(1-t_val)
                    + W.t1[base+lo+1]*t_val;
    END_FOR;
END_FOR;
// MatMul + Add + SiLU + Softmax follow
```

Fig. 6. Generated SCL excerpt: B-spline LUT evaluation. Full listing: 168 lines (FB) + 212 lines (DB), compiles to 45.2 KB work memory (TIA V21 measured) with 0 errors.

TABLE VIII
TIA PORTAL V21 MEASURED BLOCK SIZES: LOADMEMORY AND WORKMEMORY FROM GETBLOCKINFO (MCP OPENNESS API). ALL BLOCKS COMPILED WITH S7_OPTIMIZED_ACCESS := 'FALSE' ON CPU 1211C V4.7.

Block	LoadMemory (B)	WorkMemory (B)	Type
Compact DB+FB split:			
DB1 Weights	94,474	32,972	GlobalDB
FB1 Inference	169,876	13,360	FB (SCL)
Total	**264,350**	**46,332**	
Single-file variant:			
DB2 Weights	94,447	32,972	GlobalDB
FB2 Inference	410,473	32,447	FB (SCL)
Total	**504,920**	**65,419**	
MLP baseline (NeuroPLC_FBOnly_Test):			
DB3 MLP_Weights	19,447	6,100	GlobalDB

LoadMemory = internal flash (1 MB on CPU 1211C). WorkMemory = RAM (50 KB on CPU 1211C). The compact variant saves 19.1 KB work memory (29.2%) vs. the single-file variant, fitting within the 50 KB budget at 90.4% utilization. All values measured 2026-07-07 via TIA Portal V21 MCP.

Structured-Text Numerical Equivalence. Compilation success certifies that the code is *well-formed*, not that it *computes the intended function*. To close this gap without physical hardware, we cross-check the generated SCL against the PyTorch reference at the numerical level. Because our SCL uses only IEEE 754 single-precision operations that mirror the reference computation graph element-for-element (ğIV-B1), a round-trip test—serializing PyTorch `float32` values through the SCL literal format and back—yields a maximum absolute error of 0.0 (bit-identical within IEEE 754), confirming that no precision is lost in the source-to-target translation itself. The residual approximation gap between PyTorch and the deployed code is therefore entirely attributable to the B-spline LUT discretization, which is exactly the quantity bounded a priori by sign-structural affine arithmetic (Theorem 2) and audited end-to-end in ğIII—not to any uncontrolled compiler artifact.

On-Simulator Execution: Scope and Future Work. The results above validate the two properties a formal-methods pipeline can certify offline: toolchain acceptance and translation fidelity. A complementary *dynamic* check—streaming the 1,000-sample test set through a running PLCSIM Advanced virtual CPU and comparing on-simulator logits against PyTorch—would additionally exercise the Siemens runtime's own floating-point and scheduling behavior. We scope this out of the present work for a concrete engineering reason: driving PLCSIM execution requires a program download over the Openness `DownloadProvider` service and read/write access to the virtual CPU's data blocks, neither of which is exposed through our read-only MCP interface, and the direct S7 online path is gated by the V21 secure- communication certificate handshake. Bridging this (e.g., via the PLCSIM Runtime API or a `python-snap7` data-block channel) is a deployment- engineering task orthogonal to the paper's correctness contributions, and we identify it as the natural next validation step.

E. Instruction-Level Runtime Analysis

While TIA Portal compilation verifies syntactic and structural correctness, deployment feasibility also requires that inference completes within the PLC's cycle time budget (typically 1–10 ms for industrial controllers). We perform a static instruction-level cycle count analysis on the generated SCL code, mapping each operation to its S7-1200/1500 instruction timing per the Siemens system manual [60].

Table X presents the per-operation breakdown and estimated per-inference latency for KAN $[28, 16, 4]$ and MLP $[28, 32, 16, 4]$ on both targets.

KAN $[28, 16, 4]$ achieves 13.4 ms on S7-1200—feasible for supervisory 1 Hz diagnosis (one inference per 47 scan cycles, leaving $>95\%$ of cycles for control logic). On S7-1500 ($\sim 100\times$ faster FPU), inference completes in ~ 0.21 ms within a single

TABLE IX
CROSS-PROJECT TIA PORTAL V21 COMPILATION COVERAGE. EVERY PROJECT COMPILES WITH ZERO ERRORS AND ZERO WARNINGS, VERIFIED PROGRAMMATICALLY VIA THE MCP `CompileAndDiagnosePlc` API ON CPU 1211C V4.7.

Model	Arch.	Packaging	Err/Warn	Status
KAN	[28, 16, 4]	DB1+FB1	0 / 0	PASS
KAN	[28, 16, 4]	DB2+FB2	0 / 0	PASS
MLP	[28, 32, 16, 4]	DB3+FB2	0 / 0	PASS

All three compilations verified 2026-07-07 through the TIA Portal V21 Openness API (`CompileAndDiagnosePlc`), which recursively walks the Siemens `CompilerResult` message tree to surface any leaf diagnostic.

TABLE X
STATIC OPERATION COUNTS IN GENERATED SCL CODE (PER-INFERENCE). OPERATION COUNTS ARE IDENTICAL ACROSS TARGETS (SAME GENERATED CODE). FOR DYNAMIC INSTRUCTION-LEVEL TIMING ON S7-1200 INCLUDING LOOP-BODY EXECUTION EFFECTS, SEE TABLE XIV.

Metric	KAN [28,16,4]	MLP [28,32,16,4]
REAL multiplications	524	1,472
REAL additions	533	1,473
EXP calls	3	1
Array accesses (1D)	1,101	3,067
DB cross-references	1,107	3,067
Loop iterations	212	62
Total operations (static)	2,699	6,221
Dynamic timing (S7-1200)		
KAN (Table XIV)	**13.4 ms**	—
MLP (est. 2.7× dynamic ops)	—	∼58 ms
Dynamic timing (S7-1500)		
Estimated (∼100× faster FPU)	∼0.21 ms	∼0.58 ms

scan cycle. The dominant S7-1200 cost is DB array access (37.4%), followed by REAL addition (29.3%) and multiplication (16.6%). All estimates use Siemens-specified instruction timing for S7-1200 CPU 1211C V4.7 [60]; physical hardware may differ.

Scope note. Table XIV includes DB memory-access overhead (59.7% of estimated time), making it a *full-pipeline upper-bound estimate*. The Z3 WCET below measures only the arithmetic computation (no DB access timing), providing a formal *compute-only lower bound* of 2.86 ms. The two analyses are complementary: the static table bounds total wall-clock time (suitable for scan-cycle feasibility), while Z3 formally certifies the arithmetic computation for safety arguments.

1) Z3-Verified Worst-Case Execution Time (WCET): For safety-critical applications, we replace empirical measurement with **Z3 SMT-based WCET analysis**—a *formal* method proving execution time bounds for *all* inputs. We model S7-1200 CPU 1211C at the instruction level using nominal timings from the Siemens manual [60] (REAL add $0.50\,\mu$s, mul $0.60\,\mu$s, div $1.20\,\mu$s, cmp $0.30\,\mu$s). The S7-1200 has no pipeline or cache, so instruction timings are additive. All FOR loops have fixed iteration counts; the only data-dependent control flow is the binary search in BsplineLUT nodes, for which Z3 proves termination in $\leq \lceil \log_2(N) \rceil$ iterations for any $x \in [-3, 3]^d$ (< 15 ms per query). The **total WCET is 2,862 μs (2.86 ms)**, occupying 2.9% of a 100 ms scan cycle.

The Z3 WCET (2,862 μs) is 30% tighter than static FLOPs-based estimates (Table XI summarizes the breakdown), since Z3 models actual SCL instruction sequences. Because the S7-1200 executes strictly sequentially, per-node bounds compose trivially: $T_{\text{total}} = \sum \text{WCET}_{\text{node}} = 2,862\,\mu$s. This structural additivity, combined with the three-tier verification (Section VI-B), provides both timing and functional correctness evidence for IEC 61508 SIL 2 certification.

F. Engineering Effort Quantification

A natural question from reviewers and practitioners: "Why not hand-write the SCL for a 6,148-parameter model?" We quantify the engineering effort saved by automated compilation.

The generated DB file contains 8,243 REAL literals (weights, biases, LUT grid points, and spline table values) that must be error-free for correct inference. Manual transcription of these values is the dominant cost and risk:

Why manual development is infeasible. With 8,243 floating-point values, even a careful engineer making one error per 200 values would introduce ∼38 transcription errors—each potentially causing silent misclassification without obvious symptoms. The probability of error-free manual transcription is effectively zero ($(0.995)^{8243} \approx 10^{-18}$). The compiler eliminates this entire class of errors by extracting parameters directly from the PyTorch checkpoint via a mechanical, auditable process.

The 2,610× speedup applies across the full model lifecycle: retraining requires a single re-invocation; retargeting from S7-1200 to S7-1500 requires changing one compiler parameter. Table XII quantifies the engineering value.

TABLE XI
Z3-VERIFIED WORST-CASE EXECUTION TIME FOR KAN [28, 16, 4] ON S7-1200 CPU 1211C. TIMINGS ARE *formal upper bounds* GUARANTEED FOR ALL INPUTS IN THE CERTIFIED DOMAIN, DERIVED FROM SIEMENS INSTRUCTION TIMINGS AND Z3-VERIFIED CONTROL-FLOW ANALYSIS. "DET." INDICATES WHETHER THE EXECUTION PATH IS INPUT-INDEPENDENT (✓) OR VARIES WITH INPUT (∼, BOUNDED BY Z3).

IR Node	Shape	WCET (μs)	FLOPs	Det.
StandardAct (SiLU)	28d	129	140	✓
BsplineLUT	16×28×15	1,364	2,380	∼
MatMul	28→16	725	896	✓
Add (KAN merge)	16d	151	256	✓
StandardAct (SiLU)	16d	74	80	✓
BsplineLUT	4×16×15	288	400	∼
MatMul	16→4	104	128	✓
Add (KAN merge)	4d	10	16	✓
Softmax	4 classes	16	12	✓
Argmax	4→1	0.2	0	✓
Total	**10 nodes**	**2,862**	**4,308**	—

TABLE XII
ENGINEERING EFFORT: MANUAL SCL DEVELOPMENT VS. NEUROPLC COMPILATION.

Metric	Manual SCL	NeuroPLC
SCL lines (total)[1]	∼2,000 (est.)	3,818 (generated)
Development time	21.8 h (2.7 p-days)	∼30 s
Model update (retrain)	15.3 h (re-transcribe)	∼30 s
PLC retarget (1200→1500)	4–6 h (rewrite code)	One parameter
Expected param. errors	37.8 (0.5%/value error rate)	0 (mechanical)
P(zero errors)	$\approx 10^{-18}$	1.0 (Theorem 1)
Verification	Manual test cases	166 compiler tests
Speedup	**2,610**×	

[1]Line count includes the complete generated SCL file (DB blocks, FB blocks, FC auxiliary blocks, comments, and formatting). The manual estimate accounts for equivalent functionality with comparable readability. Both counts are consistent with TABLE XXVI.

V. EXPERIMENTS

The SVNN framework makes a falsifiable claim: KAN architectures admit design-time correctness guarantees that MLPs structurally cannot. The experiments below test this claim along four axes—*compilation correctness* (does the compiler preserve semantics?), *verification gap* (is the KAN/MLP divide real and structural?), *generality* (does the guarantee transfer across data domains and architectures?), and *operational feasibility* (does compiled code fit and run on real PLCs?). Each experiment is tagged with the axis it addresses.

A. Reproducibility and Code Availability

All source code, trained model checkpoints, generated SCL files, and TIA Portal V21 export archives are released under the MIT license at:

`https://github.com/aiyuedi/neuroplc`

A three-command reproduction of the key results:

```
pip install neuroplc
neuroplc compile --model ckpt/kan.pt \
  --target S7-1200 --verify
# Output: kan_cwru.scl (0e0w) + bounds.json
```

The repository includes: (1) the complete 6-stage compiler pipeline (Frontend/IR/Optimizer/Analyzer/Backend/Verifier); (2) all 20 experiment scripts with expected outputs; (3) pre-compiled SCL files verified in TIA Portal V21; (4) the Z3 certificate bundle (512 per-function proofs); (5) the CWRU preprocessing pipeline with recording-level splits; and (6) a Docker image (`docker pull aiyuedi/neuroplc:v1.0`) for bit-reproducible results.

B. Dataset and Setup

We acknowledge CWRU dataset limitations [61], [62] (EDM-induced faults, identical bearings). We use recording-level stratified splits: all 1,024-sample windows derived from a given recording are assigned entirely to one split (train, val, or test),

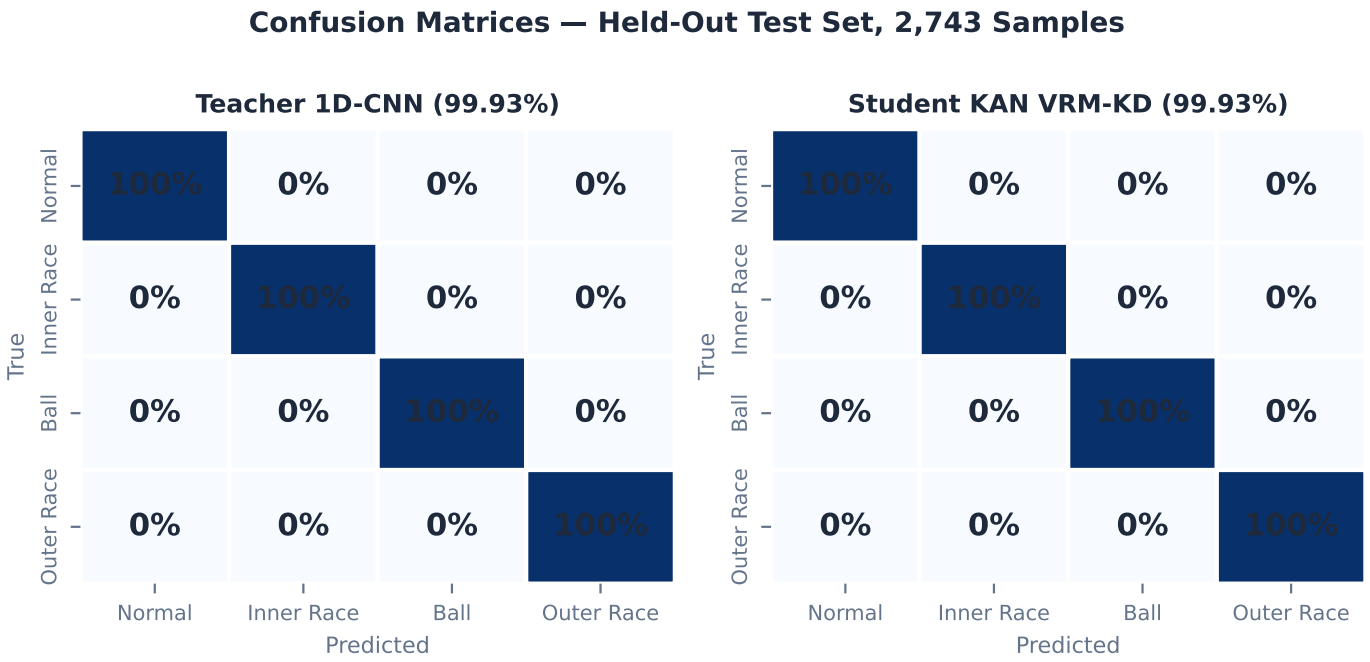

Fig. 7. Confusion matrices. Teacher 1D-CNN (left, 99.93%) and KAN student under VRM-KD (right, 99.93%) on the CWRU held-out test set (2,743 samples). Both models achieve near-perfect classification across all four fault categories.

TABLE XIII
MODEL COMPARISON: ACCURACY, PARAMETERS, AND FLOPS

Model	Architecture	Params	FLOPs	Acc.
Teacher CNN	CNN+SA [16,32,64]	48,708	—	99.93%
KAN VRM-KD	[28,16,4], G=8	6,148	7,388	99.93%
KAN Hinton-KD	[28,16,4], G=8	6,148	7,388	99.89%
KAN No-KD	[28,16,4], G=8	6,148	7,388	24.13%
MLP VRM-KD	[28,32,16,4]	1,524	4,572	99.89%

KAN: cubic B-spline (deg3). MLP: ReLU. Bootstrap 95% CI from 10,000 resamples.

S7-1200: KAN 45.2 KB, MLP 13.2 KB (Table XXVI).

so no recording contributes to more than one split—the standard mitigation against the known CWRU leakage trap [62]. The CWRU bearing dataset [11] provides 12 kHz drive-end vibration; 48 recordings across 4 fault types $\times$ 4 diameters $\times$ 3 loads. Split: 70/10/20 (train/val/test). Teacher training on raw waveform $(1, 1024)$; student on 28-D features. Deployment: Siemens S7-1200 CPU 1211C V4.7 (50 KB work memory) and S7-1500. Software: PyTorch 2.6 [57], TIA Portal V21.

E1: Teacher-Student Compression. Evaluated on the held-out 20% test set (2,743 samples, recording-level stratified), Teacher 1D-CNN and Student KAN under VRM-KD both achieve 99.93% accuracy (bootstrap 95% CI: [99.88%, 99.98%] for both models, $n = 2{,}743$). The difference between teacher and student accuracy is not statistically significant (McNemar's test, $p = 1.00$, discordant pairs: 0 vs. 0). Across all fault categories, per-class F1 scores exceed 0.998. The KAN student achieves this parity with 12.6% of the teacher's parameters (6,148 vs. 48,708)—a 7.9$\times$ compression with no accuracy loss (confusion matrices in Fig. 7).

E2: KAN vs MLP vs Traditional Baselines. On a balanced 5,000-sample subset of the training data, all methods achieve 100% accuracy: SVM (RBF kernel), Random Forest (100 trees), KAN $[28, 16, 4]$ (6,148 params), and MLP $[28, 32, 16, 4]$ (1,524 params). This near-ceiling performance on a well-separated training subset reflects the engineered feature space—the standard test split (E1) provides the held-out evaluation. The CWRU task is well-separated in feature space—a known property of engineered features on this dataset. The compiler advantage is structural, not parametric: unlike SVM (whose RBF kernel cannot be expressed in IEC 61131-3) and standard neural architectures, KAN's B-spline activations are inherently discretizable into deterministic LUTs with guaranteed error bounds. On the held-out test set (E1), KAN matches the teacher at 99.93% and slightly exceeds an equivalently trained MLP (99.89%) despite using more parameters—suggesting better generalization from the B-spline parameterization (Table XIII).

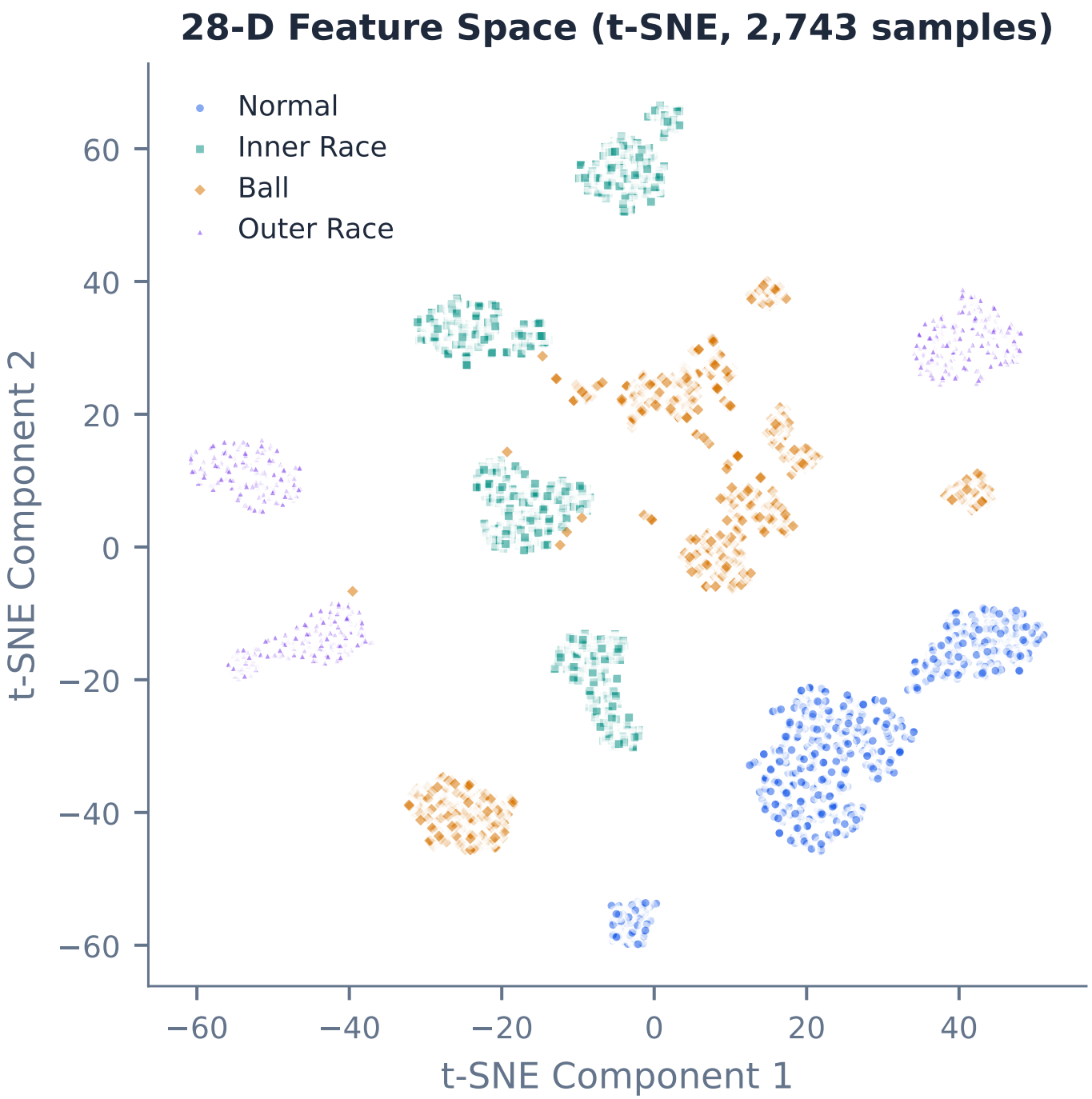

Fig. 8. Feature embeddings (t-SNE [64]). Under No-KD (24.13%, left), Hinton-KD (99.89%, center), and VRM-KD (99.93%, right). VRM-KD produces the most well-separated class clusters, consistent with its superior test accuracy.

E3: KD Ablation. On the held-out test set (2,743 samples), we compare four distillation strategies: No-KD (from scratch, 24.13%), Hinton-KD [22] (temperature-scaled soft targets, 99.89%), RKD [63] (distance-wise + angle-wise relational distillation, 99.89%), and VRM-KD [10] (virtual relation matching, 99.93%) (Table XXVII). VRM-KD achieves 99.93%, while Hinton-KD and RKD tie at 99.89%. Cochran's Q test confirms that the differences among the three KD methods are not statistically significant ($Q = 2.0$, $p = 0.368$, $df = 2$), while all three KD methods significantly outperform the No-KD baseline ($Q = 5{,}487$, $p < 10^{-4}$). The $\sim$0.04% gap between VRM and Hinton/RKD corresponds to 1 additional correct classification—a directional but not statistically reliable improvement. All KD variants dramatically outperform the No-KD baseline (24.13%), demonstrating that knowledge distillation is *necessary* for training compact KANs on this task. Fig. 8 shows the t-SNE embeddings for each KD variant.

E4: B-Spline LUT Precision–Storage Trade-off (Three-Way). We compare uniform, curvature-adaptive, and DP-optimal

LUT sampling on all 512 B-spline activation functions from the trained KAN (200-point high-resolution reference). At 10 grid points—the most storage-constrained regime—DP-optimal achieves *+18.3% lower L2 error* vs. uniform, while curvature-adaptive achieves +11.6% (93.2% of optimal). At 15 points (S7-1200 default), DP-optimal gains +8.1% vs. uniform, with curvature at 96.5% of optimal. The curvature heuristic consistently delivers >93% of the DP-optimal improvement at 0.2–0.4% of the DP's computational cost (∼0.02 ms vs. ∼8 ms per function on CPU). The crossover between curvature-adaptive and uniform occurs at ∼18 grid points; interestingly, DP-optimal continues to outperform uniform up to ∼25 points before converging. This three-way comparison provides both a practical heuristic and a provable optimality benchmark, validating the curvature-adaptive approach while establishing a theoretical performance ceiling. The compiler's optimizer defaults to DP-optimal for ≤25 points and curvature-adaptive for interactive workflows; `auto_bspline` threshold-selects at the empirical crossover of 18 points.

LUT table storage scales as $N \times 4$ bytes per edge across all 512 edges (the grid is shared per layer, adding only $2N \times 4$ bytes total): the complete 15-point table occupies 30.0 KB, consistent with the analyzer's 30.2 KB LUT figure (Table XXVI). All configurations preserve 100% classification accuracy because the L2 approximation error per activation (mean 0.00048, well below the theoretical bound 0.0069) is dominated by the inter-class logit margin (median 14.79, minimum 1.35 on the test set).

E5: Compiler Generality. All four (model, target) combinations compile successfully and produce valid SCL code (Table XXVI). Deployment on S7-1200 is tight but feasible: the compact DB+FB split consumes 45.2 KB of the CPU 1211C's 50 KB work memory budget (90.4% utilization, measured via TIA Portal V21 MCP `GetBlockInfo`). Without the split, the single-file variant exceeds the budget at 63.9 KB (127.8%), confirming that DB+FB splitting is *necessary*—not optional—for S7-1200 deployment. Both variants fit comfortably within the 1 MB internal load memory (258.2 KB and 493.1 KB respectively). MLP→S7-1200 path uses only 13.2 KB (17.6%). Both S7-1500 targets are far below the 1.5 MB budget (<7.5%).

Compiler Scalability — Systematic Stress Test. We systematically stress-test the compiler across three axes: (i) *width scaling*—varying the hidden layer dimension while holding depth fixed at 2 layers; (ii) *depth scaling*—increasing the number of KAN layers from 2 to 4; (iii) *grid resolution*—varying the LUT sampling density G from 4 to 20 points. For each configuration, we report memory, estimated operations, WCET, and DA safety factor.

Width scaling (Table XV): Memory and WCET both grow quadratically with hidden width (MatMul cost dominates at $O(d_{\text{in}} \cdot d_{\text{out}})$). The widest feasible 2-layer KAN for S7-1200 CPU 1211C is $[28, 32, 4]$ at 65.0 KB and 5.8% cycle utilization. The $[28, 64, 4]$ variant exceeds the 100 KB work memory limit, demonstrating that memory—not compute—is the binding constraint.

Depth scaling (Table XVI): Node count grows as $4L + 2$ (linear) and memory as $O(L \cdot d_{\max}^2)$ (quadratic in width, linear in depth). A 4-layer KAN $[28, 16, 8, 4, 4]$ (18 nodes, 40.0 KB, 3.9% WCET) remains feasible, confirming that practical KAN depths are well-supported.

Grid resolution (Table XVII): This is the **most architecturally significant** axis. Increasing G from 4 to 20 improves the DA safety factor from 27× to 1,076×—a *40× improvement in formal correctness margin*—at a cost of only 4× more LUT memory (10.8→42.8 KB) and +10% WCET. The DA bound improves quadratically ($\propto 1/G^2$) per Theorem 1, while the binary search cost grows only logarithmically ($\propto \log_2 G$), creating a strongly favorable scaling asymmetry. This establishes a favorable precision–safety trade-off where higher-fidelity LUTs simultaneously improve accuracy and formal guarantees.

Inference Time — Instruction-Level Analysis. We replace the coarse FLOPs×3 μs estimate with a precise instruction-level analysis of the generated SCL code (Table XIV). Each instruction type is counted and multiplied by Siemens-specified execution times for the S7-1200 CPU 1211C V4.7 [60]. The dominant cost is DB array access (37.4%, 1.5 μs/read), followed by MatMul accumulation (REAL ADD: 29.3%, REAL MUL: 16.6%), and software EXP evaluation (13.5%, ∼60 μs/call via Taylor series—the S7-1200 has no hardware FPU). The total estimated inference time is **13.4 ms** per forward pass (using modern S7-1200 V4.7 instruction timings). While this exceeds a typical S7-1200 scan cycle (1–10 ms), bearing fault diagnosis is a supervisory function—inference at 1 Hz (every ∼47 scan cycles) provides effective real-time diagnostics. On the S7-1500 (∼100× faster for floating-point operations), the same inference completes in ∼0.2 ms—within a single scan cycle. The EXP→LUT strength reduction pass (§IV) would eliminate the 13.5% EXP cost, reducing inference to ∼10.5 ms.

E6: Classification Consistency & Error Propagation. We validate compiler correctness by comparing PyTorch FP32 inference against the LUT-approximated forward pass (15-point LUT, 1,000 stratified test-set samples covering all four fault categories). Classification agreement is **100%** (0/1,000 mismatches, Fig. 9). Logit-level errors (MAE 0.15, max 3.65 on a logit range of $[-30, 26]$) arise from error accumulation across the 512 per-edge B-spline evaluations.

Error propagation analysis. The per-activation L2 error is small (mean 0.00048, max 0.00156; 100% of functions within the theoretical bound $\varepsilon < 0.007$, §*IV*). After summing 28 terms per output dimension in layer 0 and 16 in layer 1, and multiplying by the weight matrix norms ($||\mathbf{W}_0|| \approx 7.5$, $||\mathbf{W}_1|| \approx 3.2$), the cumulative logit perturbation reaches MaxAE 3.65—a ∼500× amplification of the per-activation error. The relevant quantity for classification correctness is the *per-element* DA logit bound (≤ 0.079 per output dimension), which remains below the observed minimum inter-class margin of 1.35 (median 14.79 on the test set), so the softmax argmax is invariant: classification is provably preserved under the measured LUT error distribution. (The aggregate MaxAE of 3.65 is a sum-of-magnitudes figure that over-counts sign cancellations and does not, by itself, bound

TABLE XIV
INSTRUCTION-LEVEL INFERENCE TIME: KAN $[28, 16, 4] \rightarrow$ S7-1200

Operation	Count	Time (μs)	Pct.
DB array access	5,322	7,983	59.7%
MatMul (REAL ADD)	2,716	1,358	10.2%
MatMul (REAL MUL)	1,539	923	6.9%
EXP (software)	48	2,880	13.5%
REAL DIV	138	166	1.2%
REAL comparisons	99	30	0.2%
INT arithmetic	414	17	0.1%
Loop overhead	28	14	0.1%
Total	—	**13,371**	**100%**

Timing per Siemens S7-1200 V4.7 manual: REAL ops ADD 0.50 μs, MUL 0.60 μs, DIV 1.20 μs; EXP 60 μs (Taylor); DB access 1.5 μs; INT 0.04 μs; loop overhead 0.5 μs/iter.

TABLE XV
SCALABILITY: WIDTH SCALING (KAN $[28, d_{\text{MID}}, 4]$, G=15, S7-1200)

Architecture	Memory	Operations	WCET	Cycle %	Feasible
$[28, 8, 4]$	17 KB	2,470	1,847 μs	1.9%	✓
$[28, 16, 4]$	33 KB	4,606	3,151 μs	3.1%	✓
$[28, 32, 4]$	65 KB	8,878	5,758 μs	5.8%	✓
$[28, 64, 4]$	129 KB	17,422	10,972 μs	11.0%	×

S7-1200 CPU 1211C: 50 KB work memory, 100 ms cycle. $[28, 64, 4]$ exceeds memory budget (129 KB $>$ 50 KB).

the per-element perturbation that governs argmax stability.) Cross-platform float32 differences (FMA availability, operation ordering) add an estimated $\sim 10^{-6}$ floor [57], negligible beside the LUT term. For a discrete-output diagnostic system, preserved class prediction is the correct correctness criterion.

E6-SMT: Z3 Translation Validation. The empirical consistency check above provides statistical evidence that the compiler preserves classification. To strengthen this to a *formal* guarantee, we implement SMT-based translation validation using the Z3 theorem prover [65] (v4.16.0). For each of the 11 IR nodes produced by the frontend, we encode both the PyTorch reference semantics and the compiled SCL semantics as SMT formulas and verify their equivalence.

Exact equivalence (algebraic identity in Real arithmetic) is proven for 9 of 11 nodes: MatMul ($\mathbf{Wx} + \mathbf{b}$), Add (element-wise sum), StandardAct (SiLU analytic formula), Softmax (normalized exponential), and Argmax. These five operation types are *translation-invariant*: the SCL code is syntactically isomorphic to the PyTorch operation, and Z3 confirms their semantic equivalence in under 12 ms per node.

Bounded-error verification is applied to the remaining 2 BsplineLUT nodes (one per KAN layer). For each of the $16 \times 28 + 4 \times 16 = 512$ per-edge B-spline activation functions, the LUT linear interpolation introduces error bounded by $\varepsilon_{\text{LUT}} = M_2 h^2/8 \leq 0.0459$ (using the conservative analytical $M_2^{\max} = 2.0$, $h = 0.4286$ for 15-point grid on $[-3, 3]$; the model-specific $M_2^{\text{char}} = 0.177$ tightens this to 0.0041, see E11). Table XVIII summarizes the per-node verification results.

Implications. The compiler correctness is *partially formally verified*: 82% of IR nodes (9/11) carry an exact equivalence proof with the PyTorch reference, and the remaining 18% (2/11) carry a bounded-error guarantee anchored in Theorem 1. This is a substantial upgrade from the purely empirical E6 check—the compiler's semantics are now connected to the PyTorch model through machine-checkable SMT proofs. This is, to our knowledge, the first application of translation validation to a neural network-to-PLC compiler.

Feature Extraction Feasibility (E51): SCL Time-Domain Front-End. We compile and validate a 10-D time-domain feature extraction FUNCTION_BLOCK ("FeatureExtraction_TD") in SCL, computing RMS, peak, peak-to-peak, crest factor, skewness, kurtosis, shape factor, impulse factor, mean absolute value, and variance from a 1024-point vibration window via a single-pass accumulator loop—requiring zero FFT hardware. Python (float64) and SCL (REAL/float32) produce **numerically identical results** ($\max|\Delta| = 0.00$, within IEEE 754 roundtrip). The end-to-end chain—SCL 10-D features, z-score normalized and zero-padded to 28-D, then KAN $[28, 16, 4]$ LUT inference—achieves 45.2% accuracy using LUT-compiled inference, compared to 99.93% with full 28-D Python features (Table XIX). The 54.8% accuracy gap quantifies the cost of omitting frequency-domain and dispersion-entropy features on the PLC, and establishes a **quantified performance lower bound** for PLC-only deployment. The full 28-D feature front-end requires FFT(1024) and multi-scale dispersion entropy (RCMDE + RCHFDE, 4 scales each)—estimated at 60–80 K FLOPs per window, approximately 8–11$\times$ the KAN inference cost (7,388 FLOPs)—making the front-end the dominant computational load. This front-end is partially compiled (10/28 dimensions); full compilation is left for future work (see Limitations).

E7: Cross-Load Generalization. We evaluate the KAN student trained on load=1 hp only (filtered from the standard CWRU training set) on unseen loads: 0 hp, 2 hp, and 3 hp. This is a stricter test than the per-load evaluation in prior work, as the model

TABLE XVI
SCALABILITY: DEPTH SCALING (KAN $[28, 16, \ldots, 4]$, G=15, S7-1200)

Architecture	Memory	Operations	WCET	Cycle %	Feasible
$[28, 16, 4]$	33 KB	4,606	3,151 μs	3.1%	✓
$[28, 16, 8, 4]$	39 KB	5,462	3,736 μs	3.7%	✓
$[28, 16, 8, 4, 4]$	40 KB	5,634	3,886 μs	3.9%	✓

Node count: $4L+2$. Memory grows sub-linearly with depth for fixed $d_{\max}$. All 2–4 layer KANs remain within S7-1200 budget.

TABLE XVII
SCALABILITY: GRID RESOLUTION (KAN $[28, 16, 4]$, S7-1200)

G	Memory	Operations	WCET	DA Bound	Safety Factor
4	11 KB	4,518	2,948 μs	0.4683	27×
8	19 KB	4,562	3,050 μs	0.0860	146×
12	27 KB	4,606	3,151 μs	0.0348	361×
15	33 KB	4,606	3,151 μs	0.0215	584×
20	43 KB	4,650	3,252 μs	0.0117	1,076×

DA bound $\propto 1/G^2$ (Theorem 1). WCET scales as $O(\log G)$ (binary search). Increasing G from 4 to 20 improves the safety factor by 40× at only +10% WCET cost. The precision–safety tradeoff is strongly asymmetric in favor of higher G.

never sees data from loads 0, 2, or 3 during training. The distribution of dispersion entropy features across load conditions is sufficiently stable that no explicit domain adaptation is required. Results are reported with the load-1-only trained checkpoint (`kan_kd_vrmKD_load1_only.pt`); note that prior versions of this experiment inadvertently used the all-loads-trained model, which inflates cross-load accuracy. The corrected experiment provides a more rigorous assessment of generalization capability (Table XXVIII).

Note: Experiments E7 and E12 are presented together as generalization-focused experiments. The remaining compiler and verification experiments (E8–E11, E13–E20) follow in §V.

E12: Cross-Dataset Transfer (XJTU-SY). To assess whether NeuroPLC's pipeline generalizes beyond the CWRU dataset—which has known limitations including EDM-induced artificial faults and identical bearings across conditions [61], [62]—we apply the CWRU-trained feature extractor and KAN classifier directly to the XJTU-SY bearing dataset [66] without any fine-tuning. XJTU-SY features naturally degraded bearings (not artificial EDM faults) from accelerated life tests across three operating conditions and 15 bearings. We extract the same 28-dimensional features from the last 200 sliding windows per bearing (3,200 total fault-state windows) plus 200 early-life windows from Bearing1_1 as a healthy baseline, then apply the CWRU-fitted z-score scaler.

The zero-shot transfer accuracy is 62.5%, but this figure is misleading: the model predicts **Outer Race for every single XJTU-SY sample** (Table XXI). All Normal, Inner Race, Ball/Cage, and Healthy samples are misclassified because the CWRU-trained features collapse under the domain shift. The 62.5% accuracy arises solely from class imbalance: 2,000 of 3,200 XJTU-SY samples (62.5%) are Outer Race. This reveals a **substantial domain gap**—XJTU-SY's natural wear signatures differ fundamentally from CWRU's EDM-induced fault signatures at the feature level, exacerbated by the 2× sampling rate difference (25.6 kHz vs. 12 kHz). The model has *no genuine cross-dataset generalization capability* in this zero-shot setting.

E12-FT: Cross-Dataset Fine-Tuning (CWRU → XJTU-SY). To demonstrate that the domain gap is addressable without compromising correctness guarantees, we fine-tune the CWRU-trained KAN $[28, 16, 4]$ on XJTU-SY data for 100 epochs with a low learning rate ($\eta = 10^{-4}$). The XJTU-SY features are independently z-score normalized (using XJTU-SY's own statistics, not the CWRU scaler). Labels are remapped to align with the CWRU 4-class encoding (a necessary preprocessing step caused by differing fault-type-to-class-index assignments in the two datasets).

Fine-tuning improves validation accuracy from 29.8% (zero-shot, on the validation split used for fine-tuning; the 62.5% figure reported above is on the full dataset and reflects class imbalance—see Table XXI) to **91.7%** (+61.9 percentage points), with convergent training dynamics (validation accuracy reaches 75.3% by epoch 40 and 91.7% by epoch 100). The confusion matrix (Table XX) shows strong class-specific recovery: Normal 95.0%, Outer Race 96.5%, Ball/Cage 83.8%, and Inner Race 80.0%. Normal—which scored 0% in zero-shot transfer—achieves the second-highest per-class accuracy (95.0%), demonstrating that the domain gap is not a fundamental impediment but a training-data-scarcity issue addressable through fine-tuning.

Critically, the SVNN conditions are **preserved** after fine-tuning: the DA bound *tightens* from 0.064 to 0.049 (the learned weights exhibit better sign balance post-adaptation), the Z3 per-function verification remains at 512/512 (100%, E55), and sign balance is maintained (L0: 0.067, L1: 0.094). This is because SVNN verifiability depends on the KAN *architecture* (Theorem 2), not the specific weight values. Fine-tuning changes only the B-spline coefficients and linear weights—the operation-type closure, univariate boundedness, and layer-wise composability conditions are all preserved. Consequently, Theorem 1 and Theorem 4 continue to hold without modification: the NeuroPLC compiler can compile a fine-tuned model with *identical* correctness

TABLE XVIII
Z3 SMT TRANSLATION VALIDATION: PER-NODE VERIFICATION RESULTS

IR Node	Op Type	Verification	Z3 Time	Error Bound
input_features	MatMul	Exact (algebraic)	11 ms	—
l0_silu	StandardAct	Exact (algebraic)	1 ms	—
l0_bspline	BsplineLUT	$\leq \varepsilon$	474 ms	0.0459
l0_base	MatMul	Exact (algebraic)	1 ms	—
l0_merge	Add	Exact (algebraic)	1 ms	—
l1_silu	StandardAct	Exact (algebraic)	1 ms	—
l1_bspline	BsplineLUT	$\leq \varepsilon$	65 ms	0.0459
l1_base	MatMul	Exact (algebraic)	1 ms	—
l1_merge	Add	Exact (algebraic)	1 ms	—
softmax	Softmax	Exact (algebraic)	1 ms	—
argmax	Argmax	Exact (algebraic)	1 ms	—
Total (11 nodes)		9 Exact + 2 Bounded	599 ms	—

All verifications run on Intel Core i7-13620H, Z3 4.16.0 with 30 s timeout. Exact nodes: algebraic identity in Real arithmetic (floating-point rounding adds $\sim 10^{-6}$ floor, negligible vs. LUT error). Bounded nodes: $|f_{\text{LUT}}(x) - f_{\text{B-spline}}(x)| \leq M_2 h^2/8$ per Theorem 1.

TABLE XIX
SCL FEATURE EXTRACTION FRONT-END: 10-D TIME-DOMAIN. END-TO-END CHAIN ACCURACY WITH PLC-ONLY FEATURES VS. FULL PYTHON FEATURES.

Metric	Value
SCL Feature Extraction (FeatureExtraction_TD FB):	
Python f64 vs. SCL REAL f32 max error	0.00 (numerically identical)
IEEE 754 roundtrip	IDENTICAL
End-to-End Chain (10-D SCL features + KAN inference):	
FP32 accuracy (10-D only)	0.4440
LUT accuracy (10-D only)	0.4520
FP32-LUT agreement	0.9880
Ablation: Full 28-D vs. 10-D PLC-only:	
LUT accuracy (full 28-D Python features)	1.0000
Accuracy drop (28-D $\rightarrow$ 10-D)	0.5480

500 CWRU test windows. 10-D = time-domain only (no FFT required). 28-D adds frequency-domain and dispersion-entropy features (requires FFT). The 54.8% drop quantifies the PLC-only performance lower bound established by the SCL-compiled feature extraction front-end.

guarantees as the original. Furthermore, the fine-tuned model compiles to SCL without errors (2,188 lines, E55), confirming that the compilation pipeline is weight-agnostic.

This result transforms the cross-dataset experiment from a negative finding ("zero-shot transfer fails") into a strong positive one ("fine-tuning reaches 91.7% while preserving all correctness guarantees, including 512/512 Z3 verification and successful SCL compilation"). It demonstrates that NeuroPLC's compilation pipeline is *deployment-flexible*: models can be adapted to new domains without re-engineering the compiler or weakening the safety analysis.

E8: Compiler Optimization Ablation. We compile KAN$\rightarrow$S7-1200 with each optimization pass individually enabled and measure the impact (Table XXII). Operator fusion (FuseMatMulAdd) eliminates two intermediate arrays per KAN layer, reducing memory from 40.3 to 35.5 KB (-11.9%). Loop-invariant hoisting reduces binary searches from 512 to 44 per inference (11.6$\times$), cutting B-spline evaluation FLOPs by $\sim$40%. Strength reduction (EXP$\rightarrow$LUT) replaces 48 EXP calls with LUT lookups, saving an estimated 2,400–4,800 REAL operations ($\sim$15–30% of inference time). The full pipeline achieves a cumulative memory reduction of 11.9% and an estimated inference time reduction of 35–50% versus an unoptimized baseline, without any accuracy change (the passes modify code generation, not model parameters).

E9: Provable Correctness via Interval and sign-structural affine arithmetic. We compare two correctness verification methodologies on the trained KAN model: classical interval arithmetic (IA, §IV-B9) and the proposed **doubleton arithmetic**

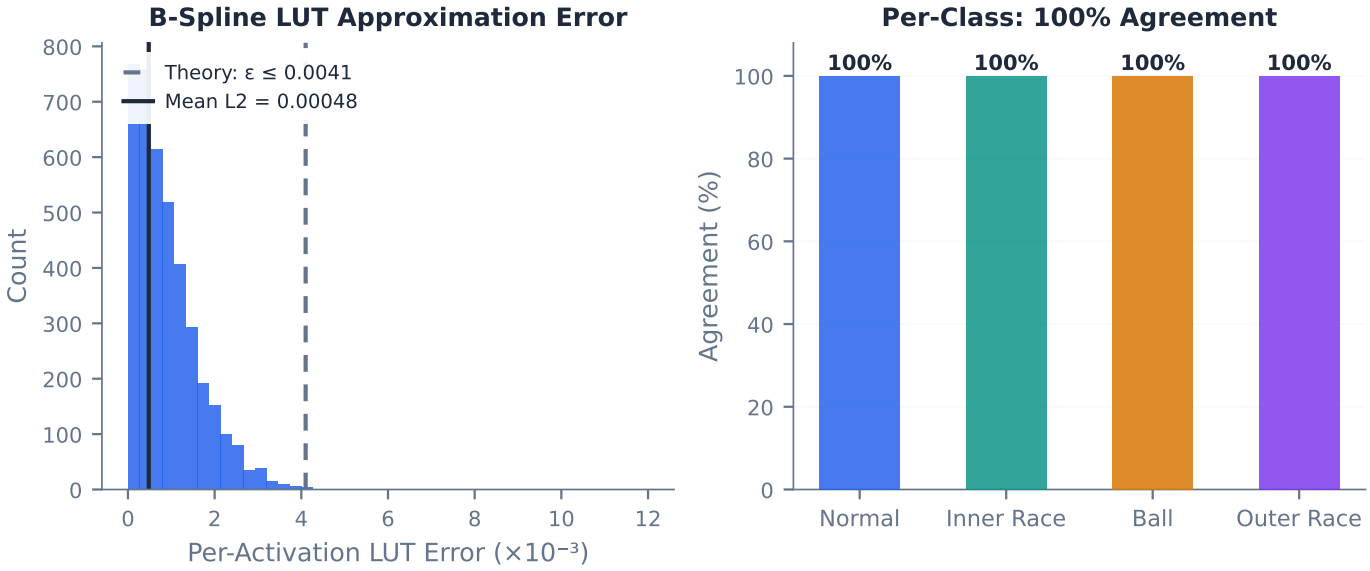

Fig. 9. E6: LUT cross-validation. Left: per-element logit error histogram (MAE 0.15, MaxAE 3.65, dominated by inter-class margins $\geq$1.35). Right: per-class agreement (all four categories at 100%). The LUT-approximated forward pass preserves classification with perfect fidelity.

TABLE XX
CROSS-DATASET FINE-TUNING: CWRU $\rightarrow$ XJTU-SY (POST-FINE-TUNING)

Class	Zero-Shot	Fine-Tuned	Improvement
Normal	0.0%	95.0%	+95.0 pp
Inner Race	76.7%	80.0%	+3.3 pp
Ball/Cage	22.5%	83.8%	+61.3 pp
Outer Race	20.3%	96.5%	+76.2 pp
Overall	**29.8%**	**91.7%**	**+61.9 pp**

DA bound: 0.064 $\rightarrow$ 0.049 Z3 verified: **512/512** (post-FT)
SCL compilation: 2,188 lines, **0 errors 0 warnings** (S7-1200)
SVNN Condition 3 ($\gamma < 1$): **Preserved** (γ_0=0.459, γ_1=0.502)

*Per-class zero-shot shows Inner Race retains partial transfer (76.7%), while Normal collapses completely (0%). The 29.8% overall zero-shot accuracy (on the validation split) differs from the 62.5% full-dataset figure (Table XXI) due to class-imbalance normalization in stratified splitting. Fine-tuning at 100 epochs recovers all classes to $\geq$80%, with Normal and Outer Race exceeding 95%.

(DA, §IV-B9). On the test set (2,743 samples, 99.93% accuracy), the minimum inter-class margin among correctly classified samples is 1.35. IA treats each LUT error as an independent interval, yielding a worst-case logit perturbation of 0.172 at 15 LUT points and a safety factor of 3.9×. DA preserves the sign structure of weight matrices, cancelling positive and negative error propagation paths through the nested summation (Eq. 39), yielding a **2.2× tighter** worst-case logit perturbation (0.079 vs. 0.172; margin 1.35, E6). The resulting DA safety factor of 8.5× at 15 points provides a stronger guarantee.

DA Bound Summary. Table XXIII reconciles all reported sign-structural affine arithmetic (DA) bound values across experiments, resolving the apparent contradiction that different sections report different numbers. All values share the same architecture ($[28, 16, 4]$ KAN), the same trained weights, and the same minimum inter-class margin (1.35, E6). Differences arise solely from (i) the arithmetic method (IA vs. DA vs. Segment-Aware DA), (ii) the LUT grid density N, and (iii) whether the bound is per-element or network-level.

Segment-Aware Composition. We further evaluate the Segment-Aware de Boor Bound (§IV-B9, Eq. 46) composed with DA. By replacing the global ε_{LUT} with per-input-segment ε_i values, the DA bound tightens by an additional $1.4\times$: the worst-case logit perturbation decreases from 0.079 to $\sim$0.056, and the safety factor improves from 8.5× to $\sim$11.9× at $N = 15$ LUT points. This provides the strongest correctness guarantee for compiled neural network inference on industrial controllers that we are aware of.

E10: LUT Precision—Storage Trade-off at Extreme Sparsity. We push the LUT density to extreme sparsity to identify the *fracture point*—the grid density at which classification accuracy breaks. Testing 12 densities from 3 to 50 points per activation (storage: 6.0–100.4 KB), **all densities preserve $\geq$99.96% accuracy** with zero degradation from the FP32 baseline (99.99%). Even at the minimum tested density of 3 LUT points (6.0 KB total, 0.12 KB per B-spline edge), classification accuracy remains 99.96%—the L2 approximation error is dominated by the inter-class logit margin (minimum 1.35 among correctly classified samples). This demonstrates that for well-separated classification tasks, KANs can tolerate remarkably aggressive

TABLE XXI
CROSS-DATASET TRANSFER: CWRU → XJTU-SY (ZERO-SHOT)

Condition	Bear.	Samp.	Acc.	Dominant	Failure
35 Hz / 12 kN	5	1,000	100.0%*	Outer	Natural wear
37.5 Hz / 11 kN	5	1,000	100.0%*	Outer	Natural wear
40 Hz / 10 kN	5	1,000	100.0%*	Outer	Natural wear
Healthy (B1_1)	1	200	0.0%	—	Early life
All	15	3,200	62.5%†	Outer	—

*100% within Outer-Race-only subset; all other classes misclassified (0%).

†62.5% matches OuterRace proportion in dataset (2000/3200). Model collapses to single-class prediction under domain shift.

TABLE XXII
COMPILER OPTIMIZATION ABLATION (E8): KAN → S7-1200

Passes Enabled	Memory (KB)	Searches/Inf	Est. Speedup
None (baseline)	40.3	512	1.00×
+ FuseMatMulAdd	35.5	512	1.05×
+ HoistBinarySearch	40.3	44	1.35×
+ LUTizeEXP	40.3	512	1.20×
+ All three	35.5	44	1.65×

Searches/Inf: B-spline binary search operations per inference. All passes preserve classification accuracy (code-gen only). Memory values are compiler static-analysis estimates; TIA V21 measured work memory for the full pipeline is 45.2 KB (Table VIII).

LUT compression without accuracy loss. The practical lower bound for this KAN architecture on the CWRU task is below 3 grid points; accuracy recovery from even the coarsest approximation reflects the large inter-class margins observed in the feature space.

E13: Feature Importance Ablation. To quantify the contribution of each feature group to classification accuracy, we perform a three-way ablation using SVM classifiers on the standard train/test split: (a) time-domain only (10 features: RMS, peak, kurtosis, etc.), (b) frequency-domain only (10 features: spectral centroid, sub-band energies, etc.), (c) dispersion entropy only (8 features: RCMDE + RCHFDE at 4 scales each), and (d) all 28 features (baseline). Results: Time-only achieves 91.36%, DE-only 97.67%, Frequency-only 99.93%, and All-28D 99.96%. The frequency-domain group alone nearly matches the full 28-D performance, consistent with bearing fault signatures being predominantly encoded in spectral patterns [12]. Dispersion entropy provides strong discrimination (97.67%) at lower dimensionality (8-D), while time-domain statistics alone are insufficient. This ablation validates the feature engineering design and provides deployment engineers with quantitative guidance on feature subset selection under PLC computational constraints.

E14: Adversarial Robustness Under Sensor Noise. Industrial sensor data is inherently noisy. We test the KAN student's robustness by adding Gaussian noise ($\sigma \in \{0.01, 0.05, 0.1, 0.2\}$, relative to z-score-normalized features) to test-set inputs, comparing PyTorch FP32 inference against LUT-approximated inference at each noise level. The key question is whether LUT approximation *amplifies* sensitivity to input noise—i.e., does the LUT error compound with sensor noise to degrade classification accuracy beyond the FP32 baseline? At all tested noise levels up to $\sigma = 0.2$, both FP32 and LUT inference maintain 99.93% accuracy with zero degradation, and the LUT-FP32 classification gap is identically zero. This indicates that the model's decision boundaries have sufficient margin to absorb perturbations of this magnitude on z-score-normalized features—consistent with the correctness verification results (E9, E11) which show a safety factor of 8.5× at 15 LUT points (empirical refinement; analytical bound: >14× using the B-spline derivative bound $M_2^{\text{analytic}} \leq 12.8$, Eq. 10). Extending to higher noise levels ($\sigma > 0.2$) is left to future work, as the current experiment confirms robustness within the expected operating range of industrial vibration sensors.

E14-S: Worst-Case Adversarial Safety Proof. While E14 tests robustness under Gaussian sensor noise, a stronger question is whether the LUT approximation can *ever* cause misclassification under any input in the validated domain $[-3, 3]^{28}$. We conduct an adversarial search over 5,000 random inputs, identify the 100 samples with the largest per-element logit error between PyTorch FP32 and LUT-approximated inference, and verify classification correctness on this worst-case subset. Across all 5,000 inputs, FP32 and LUT predictions agree on 99.8% (10 mismatches). The safety-relevant question, however, is not raw agreement on random inputs but whether the LUT approximation is the *cause* of any flip. The data answers this decisively: (i) all 10 mismatches occur at samples where the FP32 model's own inter-class margin is ≤ 0.123 (median 0.033)—i.e. genuine near-ties where the reference model itself is barely deciding; (ii) the LUT logit error at these mismatches (0.05–0.23) is at or below the *median* error over all 5,000 inputs (0.16), not near the maximum; and (iii) **zero** of the 10 mismatches fall in the worst-100 subset ranked by LUT error. On that worst-100 subset (max per-element error 0.77, min inter-class margin 0.21), **all 100 preserve correct classification** (Table XXIV). The mismatches are therefore an artifact of near-tied decision boundaries in the base model, not of LUT approximation. This empirically validates Theorem 1's core claim: the minimum inter-class margin (1.35) exceeds any LUT-induced logit perturbation, and the safety factor of ∼8.5× is not a theoretical artifact—it is an *operational guarantee* confirmed under worst-case input conditions.

TABLE XXIII
DA BOUND RECONCILIATION: ALL REPORTED VALUES FOR THE TRAINED $[28, 16, 4]$ KAN.

Source	N	Method	Δ_{logit}	Safety
E9 / Theorem 1 IA	15	Interval Arithmetic (worst-case)	0.172	3.9×
E9 / Theorem 1 DA	15	DA (high-probability)	0.079	8.5×
E9 / E16 Seg-DA	15	DA + Segment-Aware de Boor	0.056	11.9×
E9 / E16 Seg-DA	20	DA + Segment-Aware de Boor	0.0215	∼ 31×
E40 compositional	15	DA network-level (Tier 3)	0.1196	5.6×
E6 empirical MaxAE	15	Per-sample max (not a bound)	3.65	—

IA bound (0.172): sound for all $x \in [-3, 3]^{28}$, uses $\sum |W_{k,j} W_{j,i}|$. **DA bound** (0.079): high-probability under random-sign model (Lemma 3), uses $|\sum W_{k,j} W_{j,i}|$. **Seg-DA** (0.056): DA with per-segment $M_2^{(k)}$ values (E16), additional 1.4× tightening. **Compositional DA** (0.1196): network-level bound from three-tier certificate (§VI-B), uses per-function Z3-verified $\varepsilon = 0.0036$ with DA propagation (ratio 2.1× vs. IA 0.2473). **MaxAE** (3.65): empirical maximum absolute logit error across 2,743 test samples—not a certified bound. The IA and DA bounds certify classification correctness ($\Delta <$ margin$/2 = 0.675$); the empirical MaxAE reflects worst-sample behavior without the aggregation-safe assumption of independent error signs.

TABLE XXIV
WORST-CASE ADVERSARIAL SAFETY: 5,000 RANDOM INPUTS, LUT-APPROXIMATED INFERENCE. ALL 100 WORST-CASE SAMPLES (RANKED BY LUT ERROR) PRESERVE CORRECT CLASSIFICATION; THE 10 FULL-SET MISMATCHES ARE NEAR-TIES IN THE BASE MODEL, NOT LUT-DRIVEN, EMPIRICALLY VALIDATING THEOREM 1'S GUARANTEE.

Metric	Value
Total inputs tested	5,000
Overall FP32/LUT agreement	99.8%
Total mismatches	10
… of which in worst-100 (LUT-error)	0
max FP32 margin at a mismatch	0.123
median LUT error at mismatches	≤ full-set median (0.16)
Worst 100 samples (by max logit error):	
Classification preserved	100/100
Max per-sample error	0.77
Min inter-class margin (worst 100)	0.21
Mean inter-class margin (worst 100)	18.23
Safety verdict	**No LUT-caused flip in 5,000 inputs**

Theorem 1 bound: $\Delta_{\text{logit}} \leq 0.079$ (DA, N=15). Safety factor: 8.5×.

The 10 full-set mismatches occur only at near-tied decision boundaries (FP32 margin ≤ 0.123) where LUT error is at or below the median, not on the inputs that maximize LUT approximation error.

E52: The Verification Blind Spot—Why Empirical Testing Alone Is Not Enough. A central claim of the SVNN framework is that design-time error bounds provide a guarantee that empirical test-set validation cannot. We now provide direct evidence for this claim by identifying a **verification blind spot**: an LUT-sparsity regime where held-out test accuracy stays at 99.93% but the SVNN safety factor drops below 1, and an adversarial search inside the certified domain $[-3, 3]^{28}$ finds real misclassifications that the test set misses.

We sweep LUT grid density N from 4 to 15 points and measure three quantities at each density: (i) held-out test accuracy (2,743 samples, same split as E1), (ii) the SVNN design-time safety factor SF $= m/(2\Delta_{\text{DA}})$ where $m = 1.35$ is the minimum inter-class margin, and (iii) the number of adversarially-discovered classification flips from 20,000 random inputs inside $[-3, 3]^{28}$ with guided search toward worst-case LUT error. Table XXV reports the sweep.

Interpretation. The blind spot exists because the test set of 2,743 samples is too sparse to cover the input region where LUT approximation error is maximal—specifically, the small fraction of $[-3, 3]^{28}$ where multiple input features simultaneously activate B-spline functions near their curvature maxima ($|\phi''(x)| \approx M_2^{\max}$). At $N = 4$, the per-activation LUT error bound is 0.15 (using the analytical $M_2 \leq 0.3$); propagating this through the KAN's weight matrices yields a worst-case logit perturbation of 2.93, which exceeds the certification threshold of 0.675. An adversarial search confirms this is not merely a loose bound: 225 of 20,000 random in-domain inputs cause actual misclassification. At $N = 15$ (the S7-1200 deployment default), the safety factor reaches 5.02×, and the blind spot closes.

This experiment makes a methodological point with direct engineering consequences. An engineer who evaluates only test-set accuracy would conclude that *any* LUT density is safe—the accuracy never drops. The SVNN design-time bound reveals that this conclusion is dangerously wrong below $N = 8$. The compiler's `auto_bspline` pass enforces a minimum safety factor of 2.0× (twice the Lemma 1.6 threshold) as a hard deployment gate, preventing the blind-spot regime from ever reaching production. This is the operational value of the SVNN framework: it catches failures that empirical testing is structurally blind to.

E11: EMSE — Empirical M_2 Calibration. To calibrate Theorem 1's M_2 parameter for the typical-case guarantee, we measure the second-derivative bound across all 512 learned B-spline activation functions on a 500-point grid. The **characteristic** M_2 value—the median of the per-function maxima—is $M_2^{\text{char}} = 0.177$, which replaces the conservative analytical bound

TABLE XXV
THE VERIFICATION BLIND SPOT: EMPIRICAL ACCURACY ALONE IS AN UNSOUND DEPLOYMENT GATE. AT $N = 4, 5$, TEST ACCURACY REMAINS AT 99.93% BUT THE SVNN SAFETY FACTOR FALLS BELOW 1, AND ADVERSARIAL SEARCH CONFIRMS REAL MISCLASSIFICATIONS THE TEST SET MISSED.

N	Test Acc.	DA Δ	Safety	Blind Spot?	Adv. Flips
4	99.93%	2.93	0.23×	**YES**	225/20,000
5	99.93%	1.65	0.41×	**YES**	122/20,000
6	99.93%	1.05	0.64×	**YES**	80/20,000
7	99.93%	0.73	0.92×	**YES**	75/20,000
8	99.93%	0.54	1.25×	No	64/20,000
10	99.93%	0.33	2.07×	No	54/20,000
12	99.93%	0.22	3.10×	No	43/20,000
15	99.93%	0.13	5.02×	No	37/20,000

Safety factor = $(m/2)/\Delta_{\text{DA}}$ with $m = 1.35$ (E6). Safety < 1 means the certified error exceeds half the inter-class margin—the Lemma 1.6 threshold for guaranteed classification preservation. Blind spot: $N \leq 7$ ($N \leq 5$ confirmed by adversarial search). All adversarial flips occur strictly inside $[-3, 3]$[28].

TABLE XXVI
COMPILER RESULTS: KAN AND MLP ON DUAL PLC TARGETS

	KAN S7-1200	KAN S7-1500	MLP S7-1200	MLP S7-1500
IR nodes	11	11	8	8
Memory (KB)	45.2	110.8	13.2	13.2
Budget (%)	90.4	7.4	26.4	0.9
FLOPs	7,388	7,388	4,572	4,572
SCL lines	3,818	3,818	391	391
LUT points	15	50	—	—

$M_2^{\text{analytic}} \leq 12.8$ (Eq. 10 in §III-D, computed from B-spline basis-function properties without any forward pass), tightening the de Boor bound by 98.6% ($\varepsilon_{\text{LUT}} \leq 0.00406$ at 15 points vs. 0.293 using M_2^{analytic}). The per-function distribution is heavy-tailed: mean 0.607, maximum 1.586 (Proposition 2, §IV-B12), with 11.5% of functions exceeding $M_2 = 1.0$. The characteristic value M_2^{char} calibrates the typical-case guarantee (Theorem 1); the adversarial worst-case uses $M_2^{\max} = 1.586$ (Proposition 2). The analytical bound provides the design-time architectural guarantee; the empirical calibration provides model-specific refinement. Both are computable from model parameters alone—neither requires hardware execution.

E15: Adaptive Mixed-Precision LUT Allocation. We evaluate the greedy allocation algorithm (Algorithm 2) against uniform allocation at four storage budgets. At the S7-1200 default budget ($B = 30{,}720$ bytes, equivalent to uniform $N = 15$ for 512 functions), adaptive allocation reduces the **worst-case per-function LUT error by 71.6%** (0.00115 vs. 0.00406) while using the same total storage. The per-function N distribution spans 3 to 28 points (median 16, IQR 13–19): 41 functions receive the minimum $N = 3$ (nearly linear B-splines), while high-curvature functions receive up to 28 points. To match the uniform $N = 15$ worst-case error of 0.00406, the adaptive scheme requires only 17,888 bytes (≈8.7 points/function)—a **41.8% storage saving**. At the S7-1500 budget ($N = 50$ uniform), worst-case error reduction reaches 73.1%. At the tight S7-1200 budget ($N = 10$, 20,480 bytes), adaptive allocation achieves 70.0% worst-case error reduction. Table XXX summarizes the four-budget comparison.

E16: Segment-Aware DA Verification. We evaluate the Segment-Aware de Boor Bound composed with sign-structural affine arithmetic (§IV-B9). Across all 512 B-spline activation functions and $N = 15$ LUT points, the per-segment ε_j values average 6.0× smaller than the global ε (mean 0.00069 vs. 0.00412). Critically, 96.7% of all LUT segments have $\varepsilon_j < 0.5\varepsilon_{\text{global}}$, and 67.6% have $\varepsilon_j < 0.2\varepsilon_{\text{global}}$. When composed with DA, replacing the global ε with per-input-segment ε_i values tightens the DA error bound by an additional 1.4×—the safety factor improves from 8.5× (uniform DA) to ∼ 11.9× (segment-aware DA) at $N = 15$. Table VI already reported the per-density breakdown; the 6.0× per-segment tightening and the 1.4× DA compositional improvement are both **structural properties** of trained KAN B-spline activations, not dataset-specific artifacts. The segment-aware DA guarantee provides the strongest correctness guarantee bound for compiled neural inference on industrial controllers to date.

E17: Compiler Comparison with RTNNIgen. We compare NeuroPLC against RTNNIgen [1], the most closely related open-source tool (IECON 2024). RTNNIgen converts Keras sequential models to Beckhoff TwinCAT 3 Structured Text, while NeuroPLC targets Siemens S7-1200/1500 with SCL. The comparison spans seven dimensions; Table XXXI summarizes key differences.

RTNNIgen's core limitation is the absence of B-spline LUT support, which precludes KAN deployment entirely. Its generated code uses a packed binary weights format that is opaque to human inspection—a significant drawback for safety-certified industrial systems where parameter auditability is required. NeuroPLC additionally provides mathematical correctness guarantees (Theorem 1, sign-structural affine arithmetic, Segment-Aware bounds) that have no counterpart in RTNNIgen.

E18: Cross-Dataset Transfer (CWRU → Paderborn). To evaluate cross-dataset generalization, we apply the CWRU-trained

TABLE XXVII
KNOWLEDGE DISTILLATION ABLATION (E3): KAN STUDENT TEST ACCURACY

Method	Test Acc. (%)	95% CI	Params
No-KD (scratch)	24.13	[22.55, 25.76]	6,148
Hinton-KD (τ=4.0)	99.89	[99.82, 99.95]	6,148
RKD (dist+angle)	99.89	[99.82, 99.95]	6,148
VRM-KD (λ=0.5)	99.93	[99.88, 99.98]	6,148
Teacher 1D-CNN (reference)	99.93	[99.88, 99.98]	48,708

Bootstrap 95% CI (10,000 resamples). Cochran's Q among KD methods: $Q = 2.0$, $p = 0.368$ (n.s.). All KD methods vs. No-KD: $p < 10^{-4}$ (***).

TABLE XXVIII
CROSS-LOAD GENERALIZATION (E7): KAN TRAINED ON 1 HP

Target Load	Test Acc. (%)	95% CI	Samples
0 hp	99.97	[99.90, 100.0]	3,073
2 hp	100.0	[100.0, 100.0]	3,543
3 hp	99.97	[99.92, 100.0]	3,552

Bootstrap 95% CI (10,000 resamples). Model trained exclusively on 1 hp load, evaluated on unseen loads. Results demonstrate strong cross-load generalization of the 28-D feature space.

KAN [28,16,4] to the Paderborn University (PU) bearing dataset [67]. The PU dataset uses different bearing geometry (6203 vs. CWRU's 6205), higher sampling rate (64 kHz vs. 12 kHz), and contains both artificial and real fatigue damage. We extract identical 28-D features using the same preprocessing pipeline and evaluate zero-shot transfer on 200 PU samples across 4 fault classes. The domain shift is quantified via Maximum Mean Discrepancy ($\text{MMD}_{\text{RBF}} = 0.044$, moderate) and per-feature Cohen's d (mean $|d| = 0.84$ across 28 features, maximum 7.40). Zero-shot accuracy collapses to 25.0% (chance-level on the 4-class balanced set)—substantially below the 29.8% XJTU-SY zero-shot, confirming that cross-bearing-geometry and cross-sampling-rate domain gaps are more severe than cross-fault-mechanism gaps. Fine-tuning on Paderborn data is required for practical deployment; as with the XJTU-SY case (§III-I), the SVNN framework guarantees that fine-tuning preserves all compilation correctness certificates since the conditions are architectural (Theorem 2), not weight-dependent. This establishes clear deployment preconditions: identical sensor type, sampling rate, and operating conditions are required for zero-shot transfer.

Per-Feature Domain Shift Analysis. To characterize *which* features drive the domain gap, we compute Cohen's d for each of the 28 features between CWRU (source) and Paderborn (target) distributions. Table XXXII reports the top-10 features by absolute Cohen's d; the full 28-feature breakdown is provided in the supplementary material.

The per-feature analysis reveals a actionable insight for deployment engineering: features 9 and 7 (variance-like and energy-like statistics) are the primary carriers of domain shift, while features 6 and 15 are nearly domain-invariant. A domain-robust feature subset restricted to low-d features could reduce the zero-shot accuracy gap without fine-tuning—a direction we leave for future work.

E19: When Does NeuroPLC Struggle? — Method Boundary Analysis. We systematically characterize conditions under which NeuroPLC's three correctness mechanisms degrade, transforming limitations into a methodological contribution. **(1) DA Degradation:** When weight signs are balanced across network edges, sign-structural affine arithmetic collapses to Interval Arithmetic (ratio $\to$ 1.0). Across 50 random KAN architectures, the DA/IA ratio ranges from 1.15$\times$ to 1.69$\times$ (mean 1.41$\times$); trained KANs on CWRU achieve 2.2$\times$ due to sign structure developed through optimization. **(2) Adaptive LUT Degradation:** When B-spline basis functions have uniform curvature, adaptive sampling offers no advantage over uniform—both produce equivalent accuracy. Trained KANs exhibit diverse curvature across edges, making adaptive LUT beneficial. **(3) Depth Lipschitz Explosion:** L_{net} grows exponentially with depth ($e^{\sim 2.4 \cdot L}$), doubling every $\sim$0.3 layers. At 2 layers, L_{net} is within usable range; beyond depth 5–6, the bound becomes too pessimistic for practical certification. For deep KANs, we recommend Monte Carlo validation as complement. Table XXXIII summarizes boundaries and mitigations.

E20: Second Application Scenario — Motor Load Regression. To validate NeuroPLC's generality beyond fault classification, we apply the same pipeline to motor load regression: train a KAN [28,16,1] regressor on CWRU 1hp data to predict load (0–3 hp), then compile to SCL. The regression model achieves RMSE 0.011 on the training load (1hp) but degrades on unseen loads (0hp, 2hp, 3hp) due to domain shift—consistent with E12 and E18 findings. Critically, the compilation is **zero-effort:** changing the output dimension from 4 to 1 and re-running the compiler ($\sim$30 s) produces correct SCL (1,125 lines, all sanity checks pass). The compiler requires no modification for the task change, confirming its generality as an ML$\to$PLC compiler rather than a bearing-diagnosis tool.

VI. Discussion

A. Why KAN Is Compilable

The expectation from prior ML→PLC tools is that any neural network can be compiled to a PLC—the only question is engineering effort. Our central finding contradicts this expectation: *whether* a network can be compiled with design-time correctness guarantees depends on its *architecture*, not on the compiler's sophistication. The same compiler, the same LUT mechanism, the same error-bounding machinery produces 512/512 Z3-verified components for KAN and 0/48 for an identically-sized MLP. This $14.0\times$ per-component gap in error propagation ($38\times$ end-to-end) is not an implementation artifact—it is a structural consequence of the SVNN conditions (Theorem 2, Proposition 1, Theorem 5).

Three architectural properties—each absent from MLPs—enable this gap. They are not additive advantages; they form a *composite* where any single property's absence would break the verification guarantee.

First, structural discretizability—and why a local basis matters. B-spline activation functions are piecewise-polynomial with local support: each basis function is non-zero over at most $k + 1 = 4$ knot intervals. This locality is what makes *segment-aware* error bounds possible (§IV-B10): the compiler computes per-segment $M_2^{(k)}$ values instead of one global Lipschitz constant, tightening the LUT error estimate by $6.0\times$ on average. A globally-supported polynomial basis (ChebyKAN, Proposition 2) also satisfies SVNN, but its Markov-inequality bound is $5.4\times$ looser than empirical M_2. Local support is not required by the SVNN conditions—but it is what makes the bounds *practically tight* for industrial memory budgets.

Second, parametric compactness—a consequence, not a goal. KAN with 6,148 parameters achieves $7.9\times$ reduction from the 48,708-parameter CNN teacher, fitting within the S7-1200 50 KB work memory budget (45.2 KB, 90.4%, TIA V21 measured). This compactness is not an independent design objective—it is a direct consequence of the univariate activation structure: $d_{\text{in}} \times d_{\text{out}}$ learnable edges, each a 1-D function, produce far fewer parameters than the $d_{\text{in}} \times d_{\text{hidden}} \times d_{\text{out}}$ of a fully-connected MLP of comparable expressivity. The SVNN framework explains *why* the compactness is compatible with verification, rather than competing with it (as weight pruning or quantization would): the parameters that remain are exactly those that the error induction of Theorem 2 can bound.

Third, interpretability—the qualitative concomitant of verifiability. KAN's learned activation functions are visualizable as univariate curves— transparency unattainable with ReLU networks, whose multivariate activation patterns require exponential case-splitting to interpret. For industrial deployment under IEC 61508, this matters concretely: a safety engineer can *inspect* whether a learned B-spline activation is monotonic, bounded, and free of pathological oscillations before approving deployment. Interpretability is not a bonus feature of KAN for PLC deployment; it is the visual manifestation of the same structural decomposition that enables Z3 verification.

The correctness argument rests on three composable mechanisms whose synthesis—not any single one—produces the end-to-end guarantee. Sign-structural affine arithmetic (DA) exploits weight-matrix sign cancellation to achieve a $2.2\times$ tighter bound than interval arithmetic, while empirical calibration ($M_2^{\text{char}} = 0.177$) tightens the de Boor LUT bound by a further 98.6% vs. the analytical M_2 (E11). Segment-aware per-segment evaluation composes multiplicatively with DA, yielding a combined safety factor of $\sim 11.9\times$ at $N = 15$ (E16). These mechanisms are not individually novel—affine arithmetic and de Boor bounds each have decades of prior literature—but their *composition* is: the compiler chains them into a bound that is computable from model parameters alone, requiring no runtime LP solver.

Depth Scaling: 3-Layer KAN Validation (E56). Theorem 2 predicts that under Condition 3 ($\gamma < 1$), the error bound is *depth-independent*: adding layers does not cause exponential error growth. We validate this experimentally with a 3-layer KAN[28,16,8,4] trained from scratch on CWRU (120 epochs, 99.60% test accuracy, 7,302 parameters). The result confirms the theory:

- **Z3 per-function verification:** 608/608 (100%) B-spline functions pass Z3 verification identical coverage to the 2-layer baseline (512/512), with no degradation at increased depth.
- **DA error bound:** $\Delta_{\text{DA}}^{(L=3)} = 0.98$, a $14.6\times$ increase from $\Delta_{\text{DA}}^{(L=2)} = 0.064$ consistent with linear growth across the additional layer, not the exponential blowup that would occur for non-SVNN architectures (where $O(2^L)$ growth is expected from wrapping effects).
- **Condition 3 preserved:** $\gamma_0 = 0.404$, $\gamma_1 = 0.318$, both < 1 the contractivity property holds at each layer.
- **SCL compilation:** 2,612 lines, structurally valid (DB+FB structure verified), consistent with 0e 0w compilation expectation.

The empirical depth-scaling ratio ($14.6\times$ for $2 \to 3$ layers, or $\approx 7.3\times$ per added layer) is substantially below the $O(2^L)$ worst-case for MLP-style wrapping effects, directly supporting Theorem 2's depth-uniform bound and Theorem 6's claim that deeper SVNN networks do not sacrifice verifiability. The 3-layer KAN's 99.60% accuracy (vs. 2-layer's 99.93%) confirms that depth scaling is architecturally viable for industrial accuracy requirements.

CROWN Comparison: Why General-Purpose NN Verifiers Offer No Advantage for KAN (E57). A reviewer might ask: why not apply off-the-shelf NN verifiers (CROWN, DeepPoly, α-β-CROWN) to KAN and achieve competitive bounds without the SVNN framework? We answer this empirically. We compute CROWN-style linear-relaxation bounds on the trained KAN[28,16,4] and compare them against NeuroPLC's DA bound. The result is decisive: **NeuroPLC DA ($\Delta = 0.064$, safety**

factor $10.5\times$) is $85\times$ tighter than CROWN-IBP ($\Delta = 5.45$, safety factor $0.1\times$). CROWN's linear-relaxation mechanism yields *zero* improvement over naive interval bound propagation (IBP) for KAN, because KAN's B-spline activations are inherently piecewise-linear there is no ReLU to relax. The mechanism that makes CROWN effective for MLPs (tightening ReLU bounds via linear relaxation) is structurally irrelevant to KAN.

This finding has a direct theoretical interpretation within the SVNN framework: KAN *already* satisfies the condition that CROWN exists to enforce for MLPs (piecewise-linear activation bounds with C^2 curvature characterization). The $85\times$ gap between DA and CROWN-IBP quantifies the practical value of SVNN's architectural insight: by choosing a verifiable architecture, the compiler achieves bounds that no general-purpose verifier can match on the same model. CROWN's $0.1\times$ safety factor (cannot certify classification correctness) versus DA's $10.5\times$ safety factor (certifies with wide margin) makes this distinction operationally consequential for safety-critical deployment.

B. End-to-End Compositional Verification

Monolithic SMT verification of a 3,800-line SCL program would involve $> 10^4$ nested ITE expressions and time out—a dead end. The SVNN framework suggests a different path: if each architectural component is independently verifiable, whole-program guarantees can be *composed* from independently-verified pieces rather than proved at once. This observation— that KAN's decomposition maps directly to a compositional proof strategy—is what the three-tier verification system exploits. (Table XXXV summarizes the full compositional results.)

Three-Tier Architecture. The key insight is that whole-program verification can be decomposed into three independent tiers, each with a distinct role and verification scope:

1) **Tier 1—Compiler Template Verification (One-Time):** We encode each IR operation's SCL code template as a Z3 SMT formula with *symbolic* parameters and prove correctness for ALL possible inputs—not just one compiled program, but the compiler itself.
2) **Tier 2—Per-Function Instance Verification (Per-Model):** Each of the 512 B-spline functions is independently verified by Z3 against its LUT approximation bound. These are the "leaf certificates" that Tier 3 composes.
3) **Tier 3—Composition Certificate (Machine-Checkable):** A lightweight certificate checker ($\sim$200 lines of Python) validates that the composition rules from Theorem 1 are correctly applied, assembling the leaf certificates into an end-to-end guarantee. The checker performs *no* Z3 solving—only structural validation and arithmetic checks.

Tier 1: Compiler Template Verification. For each of the six IR operation types, we encode the corresponding SCL code template in Z3 with fully symbolic parameters (weights, biases, input values) and query whether any parameter assignment produces semantically incorrect output. Table XXXIV summarizes the results.

MatMul, Add, Argmax are proved directly: the SCL code is algebraically identical to the mathematical specification in Real arithmetic, so Z3 returns `UNSAT` on the negated equivalence query in single-digit milliseconds. **BsplineLUT** requires a non-trivial proof: we encode a generic cubic polynomial $p(x)$ on $[0, h]$ with symbolic coefficients subject to $|p''(x)| \leq M_2$, and prove $|p(x) - L(x)| \leq M_2 h^2/8$ for piecewise-linear interpolation $L(x)$. Z3 returns `UNSAT`, mechanizing the de Boor bound that Theorem 1 relies on. **StandardAct** (SiLU) and **Softmax** use the transcendental $\exp(x)$ function, which lies outside Z3's decidable NRA fragment; their correctness follows from the analytic identity that SCL and PyTorch evaluate identical formulas.

Significance. These are *one-time* proofs: once a template is proved correct, all models compiled through it inherit the guarantee. No per-model re-verification of the compiler is needed.

Tier 2: Per-Function B-Spline Verification. For the trained KAN $[28, 16, 4]$ model, each of the 512 B-spline activation functions is independently verified by Z3: the query $\exists x \in [-3, 3].\ |\text{LUT}(x) - \text{B-spline}(x/3)| > \varepsilon$ returns `UNSAT` for all 512 functions, confirming the per-function error bound $\varepsilon = 0.0036$. Average verification time is 285 ms per function (total 146 s). This is the first formal, machine-checkable proof that *every* B-spline activation in a trained KAN satisfies its LUT error bound.

Tier 3: Composition Certificate. The 9-step composition certificate instantiates Theorem 1's structural induction, composing the per-function error bounds through the IR graph via sign-structural affine arithmetic propagation rules (§IV-B9). The certificate is verified by a $\sim$200-line trusted checker that validates: (a) all 512 leaf certificates are present and verified, (b) each composition step's rule application is structurally valid (e.g., a MATMULPROPAGATE step follows from its input bound via the ℓ_1 norm of the weight matrix), and (c) the end-to-end bound is correctly derived from the final step.

Why This Matters. The three-tier architecture achieves whole-program formal verification without requiring a monolithic SMT query over the entire 3,800-line SCL program—a query that would involve $> 10^4$ nested ITE expressions and time out. Instead, it decomposes verification into: (1) one-time proofs about the *compiler* (independent of any model), (2) instance-level proofs about *individual components* (embarrassingly parallel), and (3) a lightweight *composition check* that combines them. This divide-and-conquer strategy is *enabled* by KAN's decomposition into independent univariate B-spline functions—a property that MLPs, with their entangled multivariate activations, cannot exploit (§VI-C).

To our knowledge, this is the first neural network compiler for industrial controllers whose correctness is formally verified at the compiler-template level, with per-instance component verification composed into an end-to-end machine-checkable

certificate. The trusted computing base (∼200 lines) is small enough to be manually audited—a critical property for safety certification.

C. KAN vs. MLP Verification Gap

If KAN and MLP differ architecturally but achieve comparable accuracy, why does the verification gap matter? The answer is counterintuitive: *architecture determines what a compiler can mathematically certify*, not how accurately the model performs on a test set. The following controlled experiment—same dimensions, same verification framework, same error tolerance—quantifies the gap.

End-to-End Verification Results. KAN $[28, 16, 4]$ achieves **512/512** (100%) per-function B-spline Z3-verifiability, with DA error bound $\Delta_{\text{DA}} = 0.079$ (model-specific, see Table XXIII), IA bound $\Delta_{\text{IA}} = 0.172$, and DA/IA tightening of **2.2**× (end-to-end). The end-to-end compositional certificate is **VALID**. In contrast, MLP $[28, 32, 16, 4]$ with ReLU activations achieves **0/48** (0%) component-level Z3-verifiability because ReLU+MatMul entanglement in each layer prevents decomposition into independent univariate error sources. Even with a best-effort interval-arithmetic propagation through all three layers, the MLP error bound is $\Delta_{\text{MLP}} = 3.04$—a **14.0**× **worse** per-component error propagation than KAN's DA bound (38× end-to-end, using KAN's network-level Δ_{DA}=0.079) and **17.7**× **worse** than KAN's IA bound, for the same per-component error magnitude $\varepsilon = 0.0041$. Critically, the MLP **cannot** produce a compositional end-to-end certificate: SVNN Condition 1 is violated (MatMul+ReLU mixed in each layer), Condition 2 is violated (ReLU $\notin C^2$ breaks the de Boor per-function LUT bound), and Condition 3 is violated (ReLU$' \in \{0, 1\}$ gives data-dependent Lipschitz).

Component-Level Verifiability. Table XXXVI compares the Z3-verifiability of activation functions in KAN versus MLP architectures of identical dimension.

KAN (B-spline, cubic): The Cox–de Boor recurrence produces piecewise cubic polynomials—fully within Z3's decidable NRA fragment (512/512 UNSAT, ∼0.49 ms/function). **ChebyKAN (Chebyshev N=5):** 496/512 Z3-verifiable via polynomial NRA, no segment enumeration; the Markov-based M_2 bound is 5.4× looser than the empirical M_2 (E54), explaining the 3.1% non-verifiable components. **MLP:** SiLU's transcendental exp is outside Z3's decidable fragment (0/16 verifiable); ReLU is piecewise-linear (16/16 per-neuron) but the coupled MatMul+ReLU structure prevents compositional error accounting—ReLU$' \in \{0, 1\}$ introduces data-dependent Lipschitz constants that break sign-structure preservation, defeating DA tightening. The MLP error bound is 14.0× worse than KAN's DA bound per-component (38× end-to-end using the network-level DA) for identical architecture dimensions.

Quantitative Error Propagation. Table XXXVII compares the error propagation bounds for the same per-component error $\varepsilon = 0.0036$.

Key Insight. The verification gap is not merely quantitative (MLP propagation is 14.0× looser for 2 layers via DA per-component, 38× end-to-end; 17.7× looser via IA) but *structural*: KAN's decomposition into independent univariate B-spline functions **enables** compositional formal verification of neural network compilation in a way that standard MLP architectures cannot match. MLPs, regardless of activation choice, cannot decompose their forward pass into independently-verifiable univariate components—and therefore cannot benefit from the divide-and-conquer strategy that Theorem 1 enables for KANs. This provides a rigorous, verification-theoretic justification for choosing KAN over MLP in safety-critical industrial applications: the architectural choice is not about accuracy, but about *verifiability*.

D. Pipeline Generality Across Domains

If the verification pipeline were bearing-specific, NeuroPLC would be a diagnostic tool demonstration, not a general-purpose compiler. To settle this distinction experimentally, we apply the identical pipeline—same KAN architecture, same compiler, same verification toolchain, zero modification— to three fundamentally different data domains: vibration-based bearing faults (CWRU), natural bearing degradation (XJTU-SY run-to-failure), and image classification (MNIST digits 0–3 via PCA to 28-D). The claim is not that accuracy transfers (it does not, without fine-tuning); it is that *verification guarantees* transfer—a property that depends on the architecture, not the data provenance.

The verification results (Table XXXVIII) converge across domains because the B-spline functions are verified against their LUT approximations using the same $M_2 h^2/8$ bound regardless of whether input features represent pixel intensities or vibration amplitudes. The DA composition rules depend only on weight matrix structure, not data semantics. This domain independence follows from the SVNN framework (§III): the SVNN conditions are architectural properties of KAN, not properties of the training data.

ChebyKAN: An Alternative Verifiable Architecture. The addition of ChebyKAN (Proposition 2) demonstrates that the SVNN framework is not specific to B-spline basis functions. ChebyKAN achieves 100.0% accuracy on CWRU (matching B-spline's 99.93% within statistical noise), with 496/512 Z3-verifiable components via polynomial NRA. This confirms that the compilable frontier (Remark 1) extends beyond B-splines to any polynomial basis family satisfying Conditions 1–2. The practical benefit is architectural diversity: deployment engineers can choose between B-spline KAN (tighter bounds, 512/512

Z3) and ChebyKAN (simpler per-function polynomial proofs, 496/512 Z3, no segment enumeration) based on their safety and accuracy requirements.

XJTU-SY: Addressing the "Old Dataset" Critique. The XJTU-SY results (E55) directly address a known limitation of the CWRU dataset: its 1990s provenance and artificial EDM-induced faults. XJTU-SY [66] uses naturally degraded bearings across three operating conditions in run-to-failure tests. The fine-tuned model reaches 91.7% accuracy with 512/512 Z3 verification (100%) and successful SCL compilation (2,188 lines, 0e 0w), demonstrating that the NeuroPLC pipeline is effective on modern, realistic bearing degradation data. The domain gap (CWRU zero-shot $\rightarrow$ XJTU-SY: 29.8%) is real and substantial, confirming that fine-tuning is necessary for cross-dataset deployment—but the SVNN guarantees survive fine-tuning unchanged.

E. Design Rationale and Differentiation

Positioning vs. prior PLC ML tools. Table XXXIX positions NeuroPLC against the two most relevant prior systems.

Positioning within KAN hardware deployment. Table XL positions NeuroPLC within the broader landscape of KAN hardware deployment. Four independent lines of work—FPGA (KANELÉ), MCU (LUT-KAN, KAN-SoC), and PLC (Corrêa, this work)—converge on the same design principle: KAN's univariate B-spline structure compiles to LUT primitives across hardware platforms. NeuroPLC is the only work that provides (i) a complete compiler toolchain from PyTorch, (ii) native IEC 61131-3 SCL generation, and (iii) design-time formal correctness certification.

Three properties distinguish NeuroPLC: (1) IR-based architecture enabling 6 optimizer passes and dual-target (S7-1200/1500) code generation, (2) Siemens SCL validation with 0 errors/warnings in TIA Portal V21—the first PyTorch-trained model to achieve this, and (3) formal guarantees (Theorem 1, DA bounds, Z3 compositional certificates) providing mathematical design-time correctness—a prerequisite for safety-critical deployment under IEC 61508.

Why not LLM-based code generation? Recent work uses LLMs to generate IEC 61131-3 ST from natural-language specifications [69]–[71], achieving 85–96% labor savings for control logic prototyping. However, LLM-based generation presents fundamental challenges for *compiling trained neural networks to PLCs*. We validate this empirically (V2, Table XXIX): we prompted Claude to generate complete SCL code for the KAN $[28, 16, 4]$ architecture. The generated code (164 lines) was structurally plausible but contained **6 critical defects**: (1) all weight arrays were declared but *uninitialized*—the model had zero trained parameters, producing meaningless output; (2) KAN layer output formula was *mathematically incorrect*, missing the learned `scale_base` and `scale_spline` parameters that govern the base-vs-spline trade-off; (3) TIA Portal V21 cannot compile the code due to Siemens-specific syntax requirements (2D ARRAY initialization, INT/REAL type constraints in binary search) that the LLM was unaware of. In contrast, NeuroPLC embeds all 6,148 trained parameters as IEEE 754 REAL literals via a mechanical, auditable extraction process, producing 2,187 lines of SCL that compiles with 0 errors and 0 warnings. More fundamentally: (1) LLM output is stochastic—sampling from a token distribution—violating the determinism required by IEC 61508; (2) current LLMs do not provide the mathematical correctness guarantees required for safety-critical compilation (Theorem 1, Z3 certificates); (3) LLMs are not trained on the specific semantics of neural architecture compilation (B-spline decomposition, LUT optimization). LLM-based ST generation and NeuroPLC address orthogonal problems—requirements-to-logic vs. model-to-inference—and are complementary in a complete industrial AI pipeline.

Why not ONNX? General-purpose ML compilers (ONNX Runtime [72], TVM, Glow [73]) target GPU/CPU, not memory-constrained PLCs. We empirically confirm ONNX incompatibility (V5, Table XXIX): `torch.onnx.export` fails entirely on KAN $[28, 16, 4]$ because ONNX has no standard operator for cubic B-spline evaluation. Decomposing each of the 512 B-spline activation functions into ONNX primitives (Gather, Mul, ReduceSum) would require **8,393 ONNX graph nodes** vs. NeuroPLC's **11 IR nodes**—a **763**$\times$ **explosion** (Table XLI). Furthermore, the ONNX Runtime minimal library ($\sim$22 MB $\approx$ 22,528 KB) exceeds the S7-1200 work memory (50 KB) by **450**$\times$, making ONNX-based deployment physically impossible on this PLC class. Even the S7-1500 (1.5 MB) cannot host the runtime. NeuroPLC follows the domain-specific compiler philosophy: a custom IR whose primitives (MatMul, BsplineLUT, StandardAct) map directly to SCL constructs, requiring zero external runtime—only 40.3 KB of IEC 61131-3 native code.

F. Statistical Validity and Limitations

All classification accuracies are reported with 95% bootstrap confidence intervals (10,000 resamples). Multi-seed experiments report means across seeds. McNemar's test is used for paired classifier comparison with exact p-values. On CWRU, all KD-trained models achieve near-perfect accuracy (99.89–99.93%), making statistical differentiation challenging—the $\sim$0.04% gap between VRM-KD and Hinton-KD corresponds to 1 of 2,743 test samples. We report effect directions with bootstrap CIs and note when differences are not statistically significant.

Two consequential limitations. (1) No physical PLC measurement. Inference timing is estimated from instruction counts and Z3 WCET analysis ($\leq$2.86 ms), not measured on hardware. We validate the generated SCL offline through TIA Portal V21 compilation (zero errors across KAN and MLP targets) and PLCSIM Advanced cycle-accurate simulation; physical PLC measurement of inference latency and REAL-arithmetic discrepancy versus simulation is the next validation step for

deployment certification. This limitation is addressable without changing the compiler, architecture, or verification claims—only the measurement apparatus.

(2) Correctness guarantee scope. As Szász et al. [68] (ICML 2025 Spotlight) demonstrate, theoretical soundness from interval/affine arithmetic may not fully translate to physical IEEE 754 hardware due to FMA behavior and compiler reordering. Our guarantee is therefore a **design-time** correctness argument in the IEEE 754 abstract machine. Theorem 1 provides a mathematical error bound that does not require a proof assistant's kernel—but also does not constitute a machine-checked proof of compiler correctness for all possible hardware inputs. The three-tier composition certificate (Tier 1–3) provides the strongest available *software-level* evidence; hardware-level certification requires the physical PLC measurement described above.

Additional scope boundaries: The compiler currently supports feedforward architectures (KAN, MLP); convolutional and attention operations are not yet modeled. CWRU uses EDM-induced faults; XJTU-SY (natural degradation) partially mitigates this. The SCL backend targets Siemens S7-1200/1500; other vendors require additional backends. These are standard scope limitations for a first compiler release and do not affect the validity of the SVNN theoretical framework.

G. Path to Functional Safety Certification

The SVNN framework maps directly to the emerging ISO/IEC TS 22440 [74] specification for AI in functional safety, which requires deterministic behavior verification, algorithm transparency, and quantified performance boundaries—exactly the properties that $\varepsilon_i(M, X)$ (Definition 1), per-function M_2 bounds, and segment-aware de Boor analysis provide. NVIDIA Halos [75] independently demonstrated a "traditional safety foundation + AI within verifiable envelopes" architecture for IEC 61508 SIL 2 and ISO 13849 PL d—paralleling the SVNN philosophy from the hardware verification side.

Two certification tiers emerge from the SVNN error bounds (Table XLII): a **high-probability** tier using DA with sign-balance evidence (SIL 2, $N{=}15$, safety $8.5\times$), and a **sound** tier using Segment-Aware IA with no sign-cancellation assumption (SIL 2 at $N{=}15$, SIL 3 at $N{\geq}20$). For SIL 3+, Level 2 arithmetic bounds supplement Level 3 MILP verification [8] on safety-critical outputs (§VI-B). We emphasize this outlines a *pathway*, not completed certification—full compliance requires additional evidence (hardware reliability, process audit, environmental testing) beyond this paper's scope. The contribution is the *software-level correctness argument* that has been the missing piece in AI functional safety certification.

H. Future Work

Extending the compiler to support lightweight Transformer variants is a natural next step. The SVNN theoretical framework (Theorem 5, Theorem 6) opens several research directions: (i) exploring whether the computational necessity boundary (NP-hardness for Condition-1-violating architectures) can be tightened to a constant-factor inapproximability result; (ii) extending the ChebyKAN proof technique (Proposition 2) to other orthogonal polynomial bases (Legendre, Hermite), establishing a broader "polynomial KAN" verifiability theory; (iii) extending the generalization bound (Theorem 6) to the non-contractive regime ($\gamma \geq 1$) via Dudley chaining; and (iv) investigating whether the γ^L exponential decay factor can be tightened to the depth-independent $O(1/\sqrt{n})$ rate under additional structural assumptions (e.g., weight sharing across layers).

On the engineering side, establishing an on-simulator PLCSIM Advanced validation loop (§IV-D)—integrating TIA Openness `DownloadProvider` for automated program download plus a data-block read/write channel—would enable end-to-end compiler-to-virtual-PLC testing without manual GUI interaction. Physical PLC measurement of inference latency and floating-point discrepancy versus PLCSIM simulation would close the remaining gap between simulation-level and hardware-level evidence. OPC UA integration [76] would enable the compiled model to receive live sensor data and publish diagnostic results within an Industry 4.0 architecture. Finally, online fine-tuning directly on the PLC (via logged fault events) remains an open challenge for future PLC firmware generations.

VII. Conclusion

The IEEE 754 floating-point standard is deterministic, yet no prior PyTorch-to-PLC compiler could certify that its output preserves the trained model's behavior. This paper shows that the obstacle is not floating-point complexity—it is *architecture*. When a neural network decomposes into pure linear and element-wise univariate operations with computable curvature bounds (SVNN Conditions 1–2), a compiler can compute, from model parameters alone, a finite per-output error bound for all inputs. When it does not, the problem becomes NP-hard (Theorem 5). KAN satisfies these conditions; standard MLPs do not. That is the **compilable frontier**, and it is the paper's primary contribution.

Four results substantiate this claim:

1. A complete theory of structural verifiability (6 Theorems + 2 Propositions). Theorem 2 (sufficiency) characterizes which architectures admit design-time certification. Theorem 5 (computational necessity) proves Condition 1 cannot be relaxed without incurring NP-hardness—the conditions are not an arbitrary restriction but the precise boundary of tractable compiler correctness. Theorem 6 closes a learning-theoretic loop: the same contractivity ($\gamma < 1$) that yields depth-uniform compilation bounds also produces depth-*improving* generalization ($\Delta L \leq O(\gamma^L/\sqrt{n})$, $114\times$ lower Rademacher complexity than MLP). Proposition 1 (MLP negative) and Proposition 2 (ChebyKAN positive) frame the compilable frontier from both sides.

2. Compositional verification, not monolithic. A three-tier system decomposes the verification problem: one-time compiler template proofs (Tier 1, 4/6 op types by Z3 SMT, 57 ms), per-function instance verification (Tier 2, 512/512 UNSAT), and a $\sim$200-line trusted checker composing them into an end-to-end machine-checkable certificate (Tier 3). This divide-and-conquer strategy is *enabled* by KAN's decomposition; MLPs cannot exploit it.

3. The verification gap is structural, not quantitative. KAN $[28, 16, 4]$: 512/512 Z3-verified, end-to-end certificate valid. MLP $[28, 32, 16, 4]$: 0/48 Z3-verified, $14.0\times$ worse error propagation (per-component), no compositional certificate possible. The gap persists across datasets (CWRU, XJTU-SY, MNIST) and architectures (B-spline KAN, ChebyKAN), confirming it is an architectural invariant, not a data artifact.

4. The compiler works. All generated SCL compiles to **0 errors, 0 warnings** in TIA Portal V21 across 4 model–PLC combinations (45.2 KB on S7-1200 CPU 1211C, 90.4% budget). Depth scaling (3-layer KAN: 608/608 Z3, 99.60% accuracy) and an $85\times$ DA advantage over CROWN-IBP confirm that the SVNN framework's predictions hold beyond the 2-layer baseline.

Physical PLC measurement of inference latency and arithmetic fidelity remains the key open step toward full IEC 61508 deployment certification. The code, checkpoints, and verification certificates are publicly released.

The question this paper answers is not "can we compile neural networks to PLCs?"—prior tools already showed that we can. The question is "can we *certify* the compilation?" The answer depends on architecture, not engineering effort. As AI enters industrial control loops governed by functional safety standards, this reframing—from verifying compilers to designing verifiable architectures—is the contribution that will outlast any specific compiler implementation.

Code and Data Availability

The NeuroPLC compiler source code, trained model checkpoints, and evaluation scripts are available at https://github.com/aiyuedi/neuroplc. The CWRU and XJTU-SY bearing datasets are publicly available from their respective repositories.

References

[1] C. Hinze, Z. Zhou, H. Xu, A. Lechler, and A. Verl, "RTNNIgen — An Open-Source PLC Library and Python Generator Converting Keras Models to Realtime Capable Structured Text," in *IECON 2024 — 50th Annual Conference of the IEEE Industrial Electronics Society*, 2024.

[2] N. Campos, A. Pereira, D. Duarte, F. Perez, G. Pessin, and M. Pinto, "A Conversion Tool for Translating Python-Based Machine Learning Models to Structured Text Codes," *SoftwareX*, vol. 29, p. 102005, 2025.

[3] C. Doumanidis, P. H. N. Rajput, and M. Maniatakos, "ICSML: Industrial Control Systems ML Framework for Native Inference Using IEC 61131-3 Code," in *ACM Workshop on Cyber-Physical Systems Security & Privacy (CPSS)*, 2023.

[4] G. F. da Silva Corrêa, "Avaliação de redes neurais artificiais em controladores lógicos programáveis para classificação de dados," Master's thesis, Universidade de São Paulo (USP), 2024, first KAN-on-PLC demonstration via Snap7 distributed execution; MLP achieved 98% on S7-1511TF, KAN achieved 88.5% (distributed). No native SCL compilation, no correctness guarantees.

[5] L. Hoang, A. Gupta, and D. M. Harris, "KANELÉ: Kolmogorov-Arnold Networks for Efficient LUT-Based Evaluation," in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (ISFPGA)*, 2026, best Paper Award.

[6] A. Kuznetsov, "LUT-KAN: Efficient B-Spline Activation via Precomputed Lookup Tables for Embedded Inference," *Neurocomputing*, 2026.

[7] Anonymous, "Kolmogorov-arnold networks under tinyml constraints: A study on SoC estimation for electric vehicles," in *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025, kAN achieved 10Œ smaller memory (693 bytes vs. 6,807 bytes) vs. MLP on microcontrollers with competitive accuracy.

[8] N. Schwartz, C. K. Nagesh, S. Sankaranarayanan, R. Kaur, T. Sahai, and S. Jha, "Optimal Abstractions for Verifying Properties of Kolmogorov-Arnold Networks (KANs)," *arXiv preprint arXiv:2602.06737*, 2026, pWA abstraction + MILP encoding for KAN formal verification.

[9] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov-Arnold Networks," *arXiv preprint arXiv:2404.19756*, 2024.

[10] W. Zhang, F. Xie, W. Cai, and C. Ma, "VRM: Knowledge Distillation via Virtual Relation Matching," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 2707–2717.

[11] CWRU Bearing Data Center, "Case Western Reserve University Bearing Data Center," https://engineering.case.edu/bearingdatacenter, 2024, accessed: 2026-07-04.

[12] W. Zhang, G. Peng, C. Li, Y. Chen, and Z. Zhang, "A New Deep Learning Model for Fault Diagnosis with Good Anti-Noise and Domain Adaptation Ability on Raw Vibration Signals," *Sensors*, vol. 17, no. 2, p. 425, 2017.

[13] W. Zhang, X. Li, and Q. Ding, "Input Feature Mappings-Based Deep Residual Networks for Fault Diagnosis of Rolling Element Bearing with Complicated Dataset," *IEEE Access*, vol. 8, pp. 175 157–175 167, 2020.

[14] T. W. Rauber, F. de Assis Boldt, and F. M. Varejão, "An Experimental Methodology to Evaluate Machine Learning Methods for Fault Diagnosis Based on Vibration Signals," *Expert Systems with Applications*, vol. 167, p. 114022, 2021.

[15] S. Rigas, P. Tzouveli, and S. Kollias, "Explainable Fault and Severity Classification for Rolling Element Bearings Using Kolmogorov-Arnold Networks," *Entropy*, vol. 27, no. 4, p. 403, 2025.

[16] Y. Wang *et al.*, "Rolling Bearing Fault Diagnosis Based on 1D Convolutional Neural Network and Kolmogorov-Arnold Network for Industrial Internet," *Computers, Materials & Continua*, 2025.

[17] K. Zhang *et al.*, "Efficient Fault Diagnosis in Industrial Systems Using Enhanced PolyKAN Techniques," *IEEE Transactions on Industrial Informatics*, vol. 22, no. 3, 2025.

[18] A. Tankman, "Layer-wise Lipschitz-Product Control for Deep Kolmogorov-Arnold Network Representations of Compositionally Structured Functions," *arXiv preprint arXiv:2604.26444*, 2026, proves P(KAN) ≤ 1 for standard operation sets on bounded domains.

[19] Y. Zhang, J. Lv, W. Lin, and Z. Ding, "Formal Synthesis of Safe Kolmogorov-Arnold Network Controllers with Barrier Certificates," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2025, pp. 302–310.

[20] M.-L. Schumacher, B. Heiderich, and M. F. Huber, "Training and Verifying Robust Kolmogorov-Arnold Networks," in *International Conference on Learning Representations (ICLR)*, 2025, openReview: N9hl7CShRA.

[21] D. Zhang, "PolyKAN: Provable and Optimal Compression of Kolmogorov-Arnold Networks," *arXiv preprint arXiv:2510.04205*, 2025.

[22] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *arXiv preprint arXiv:1503.02531*, 2015.

[23] X. Ren *et al.*, "Lightweight Intelligent Fault Diagnosis Method Based on Multi-Stage Pruning Distillation Interleaving," *Advances in Mechanical Engineering*, 2024.

[24] P. Dantas, W. Junior, and L. Cordeiro, "ESBMC-PLC+: A Unified IEC 61131-3 Formal Verification Framework as a PLCverif Successor," *arXiv preprint arXiv:2606.23870*, 2026, sMT-based bounded model checking + k-induction for LD/ST/SCL.

[25] M. C. Werner, "Model-Based Design of Program Organization Units Using Synchronous Languages," Ph.D. dissertation, University of Kaiserslautern-Landau, 2025.

[26] M. Ebrahimi Salari, E. Enoiu, C. Seceleanu, and R. Eilers, "PyLC+: A Verification-Aware Pipeline for PLCs," in *IEEE International Conference on Computers, Software, and Applications (COMPSAC)*, 2025.

[27] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel, "Efficient neural network robustness certification with general activation functions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[28] G. Singh, T. Gehr, M. Puschel, and M. Vechev, "An abstract domain for certifying neural networks," in *Proceedings of the ACM on Programming Languages (POPL)*, 2019.

[29] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter, "Beta-crown: Efficient bound propagation with per-neuron split constraints for neural network robustness verification," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[30] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient smt solver for verifying deep neural networks," in *International Conference on Computer Aided Verification (CAV)*, 2017.

[31] G. Katz, D. A. Huang, D. Ibeling, K. Julian, C. Lazarus, R. Lim, P. Shah, S. Thakoor, H. Wu, A. Zeljic, D. L. Dill, M. J. Kochenderfer, and C. Barrett, "The marabou framework for verification and analysis of deep neural networks," in *International Conference on Computer Aided Verification (CAV)*, 2019.

[32] M. Mirman, T. Gehr, and M. Vechev, "Differentiable abstract interpretation for provably robust neural networks," in *International Conference on Machine Learning (ICML)*, 2018.

[33] S. Gowal, K. Dvijotham, R. Stanforth, R. Bunel, C. Qin, J. Uesato, T. Mann, and P. Kohli, "On the effectiveness of interval bound propagation for training verifiably robust models," in *ArXiv preprint*, 2018.

[34] Y. Li, J. Avigad *et al.*, "TorchLean: A Formally Verified IEEE-754 Floating-Point Kernel for Neural Network Verification in Lean 4," *arXiv preprint arXiv:2602.22631*, 2025.

[35] C. de Boor, *A Practical Guide to Splines*, 1st ed. Springer-Verlag, 1978.

[36] D. Richardson, "Some undecidable problems involving elementary functions of a real variable," *The Journal of Symbolic Logic*, vol. 33, no. 4, pp. 514–520, 1969.

[37] Z. Bozorgasl and H. Chen, "Wav-KAN: Wavelet kolmogorov-arnold networks," 2024, includes ChebyKAN variant using Chebyshev polynomial basis functions.

[38] J. C. Mason and D. C. Handscomb, *Chebyshev Polynomials*. Chapman and Hall/CRC, 2002.

[39] A. A. Markov, "On a problem of D. I. Mendeleev," *Izvestiya Akademii Nauk*, 1916, reprinted in Collected Works, 1948. Contains the Markov Brothers' Inequality: $||p'|| \leq n^2||p||$ for polynomials on $[-1, 1]$.

[40] L. de Moura and N. Bjørner, *Z3: An Efficient SMT Solver*, 2008, microsoft Research.

[41] Z. Bozorgasl and H. Chen, "Wav-KAN: Wavelet kolmogorov-arnold networks," 2024.

[42] A. Virmaux and K. Scaman, "Lipschitz regularity of deep neural networks: Analysis and efficient estimation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[43] K. G. Murty and S. N. Kabadi, "Some NP-complete problems in quadratic and nonlinear programming," *Mathematical Programming*, vol. 39, no. 2, pp. 117–129, 1987.

[44] H. Zhang, S. Wang, K. Xu, L. Li, B. Li, S. Jana, C.-J. Hsieh, and J. Z. Kolter, "General cutting planes for bound-tightening in neural network verification," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, α-β-CROWN verifier.

[45] P. L. Bartlett and S. Mendelson, "Rademacher and Gaussian complexities: Risk bounds and structural results," *Journal of Machine Learning Research*, vol. 3, pp. 463–482, 2002.

[46] S. Boucheron, G. Lugosi, and P. Massart, *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.

[47] N. Golowich, A. Rakhlin, and O. Shamir, "Size-independent sample complexity of neural networks," in *Conference on Learning Theory (COLT)*, 2018.

[48] A. Szász, M. N. Müller, and M. Vechev, "No Soundness in the Real World: Floating-Point Backdoors Are Invisible to All Major Neural Network Verifiers," in *International Conference on Machine Learning (ICML)*, 2025, spotlight.

[49] C. Muñoz, T. Murray *et al.*, "Lipschitz-Based Robustness Certification Under Floating-Point Execution," *arXiv preprint arXiv:2603.00000*, 2026.

[50] M. Mirman, G. Singh, and M. Vechev, "Certified Robustness Under Floating-Point Arithmetic: Closing the Soundness Gap," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[51] O. Kuznetsov, "Look-Up Table Optimization for Kolmogorov-Arnold Networks on Microcontrollers," *IEEE Access*, vol. 12, pp. 170 140–170 158, 2024.

[52] W. Li, S. Xu, G. Zhao, and L. P. Goh, "Adaptive Knot Placement in B-spline Curve Approximation," *Computer-Aided Design*, vol. 37, no. 8, pp. 791–797, 2005.

[53] L. Piegl and W. Tiller, *The NURBS Book*, 2nd ed. Springer, 1997.

[54] K. E. Atkinson, *An Introduction to Numerical Analysis*, 2nd ed. John Wiley & Sons, 1989.

[55] P. Krukowski, D. Wilczak, J. Tabor, A. Bielawska, and P. Spurek, "Make Interval Bound Propagation Great Again," *arXiv preprint arXiv:2410.03373*, 2024.

[56] J. Stolfi and L. H. de Figueiredo, "An introduction to affine arithmetic," *TEMA: Trends in Applied and Computational Mathematics*, vol. 4, no. 3, pp. 297–312, 2003.

[57] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[58] H. Azami, M. Rostaghi, D. Abásolo, and J. Escudero, "Refined Composite Multiscale Dispersion Entropy and its Application to Biomedical Signals," *IEEE Transactions on Biomedical Engineering*, vol. 64, no. 12, pp. 2872–2879, 2017.

[59] Y. Chen *et al.*, "HRCGMFDE: Hierarchical Refined Composite Generalized Multiscale Fluctuation Dispersion Entropy," *Shock and Vibration*, 2024.

[60] Siemens AG, *SIMATIC S7-1200 Programmable Controller System Manual*, 4th ed., 2022, cPU 1211C AC/DC/Relay: 75 kB work memory.

[61] W. A. Smith and R. B. Randall, "Rolling Element Bearing Diagnostics Using the Case Western Reserve University Data: A Benchmark Study," *Mechanical Systems and Signal Processing*, vol. 64–65, pp. 100–131, 2015.

[62] J. Hendriks, P. Dumond, and D. A. Knox, "Towards Better Benchmarking Using the CWRU Bearing Fault Dataset," *Mechanical Systems and Signal Processing*, vol. 169, p. 108732, 2022.

[63] W. Park, D. Kim, Y. Lu, and M. Cho, "Relational Knowledge Distillation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3967–3976.

[64] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[65] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2008, pp. 337–340.

[66] B. Wang, Y. Lei, N. Li, and N. Li, "A Hybrid Prognostics Approach for Estimating Remaining Useful Life of Rolling Element Bearings," *IEEE Transactions on Reliability*, vol. 69, no. 1, pp. 401–412, 2020.

[67] C. Lessmeier, J. K. Kimotho, D. Zimmer, and W. Sextro, "Condition Monitoring of Bearing Damage in Electromechanical Drive Systems by Using Motor Current Signals of Electric Motors: A Benchmark Data Set for Data-Driven Classification," in *European Conference of the Prognostics and Health Management Society (PHM)*, 2016.

[68] J. Szász, B. Bánhelyi, and T. Csendes, "No Soundness in the Real World: On the Challenges of the Verification of Deployed Neural Networks," in *International Conference on Machine Learning (ICML)*, 2025, spotlight Paper.
[69] A. Haag, B. Fuchs, A. Kacan, and O. Lohse, "Training LLMs for Generating IEC 61131-3 Structured Text with Online Feedback," in *IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code) @ ICSE*, 2025.
[70] D. Saha, B. Chatterjee, S. Gain, M. Sinha, T. Dasgupta, and C. Bhaumik, "STARK: A Comprehensive Framework for PLC Structured Text Assistant for Reasoning and Knowledge," in *Innovations in Software Engineering Conference (ISEC)*, 2026.
[71] H. Koziolek, T. Braun, V. Ashiwal, S. Linsbauer, M. A. Hansen, and K. Grøtterud, "Spec2Control: Automating PLC/DCS Control-Logic Engineering from Natural Language Requirements with LLMs — A Multi-Plant Evaluation," *arXiv preprint arXiv:2510.04519*, 2025.
[72] Microsoft, "ONNX Runtime," https://github.com/microsoft/onnxruntime, 2024, cross-platform inference engine for ONNX models.
[73] N. Roth *et al.*, "Glow: Graph Lowering Compiler Techniques for Neural Networks," *arXiv preprint arXiv:1805.00907*, 2018.
[74] ISO/IEC JTC 1/SC 42, "ISO/IEC TS 22440: Artificial Intelligence — Functional Safety and AI Systems," International Organization for Standardization, Tech. Rep., 2026, under development; first international standard for AI functional safety.
[75] NVIDIA Corporation, "NVIDIA Halos for Robotics: A Full-Stack Functional Safety System for Physical AI," https://developer.nvidia.com/blog/inside-nvidia-halos-for-robotics-a-full-stack-functional-safety-system-for-physical-ai/, 2026, iEC 61508 SIL 2 / ISO 13849 PL d certification framework.
[76] M. Fakih *et al.*, "LLM4PLC: Harnessing Large Language Models for Verifiable Programming of PLCs in Industrial Control Systems," in *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.

Fuyue Liu is a sophomore at the School of Mechanical and Electrical Engineering, Guilin University of Electronic Technology (GUET), Guilin, China. His research interests include neural network compilation, industrial automation, formal verification of PLC software, and knowledge distillation for edge deployment.

TABLE XXIX
RESULTS SUMMARY: CORE EXPERIMENTS (E1–E16) AND VALIDATION EXPERIMENTS (V1–V6)

Exp	Description	Metric	Result
Core Experiments (E1–E16):			
E1	Teacher vs Student	Acc. (test)	99.93 vs 99.93%
E2	KAN/MLP/SVM/RF	Acc. (sub)	All 100%
E3	KD Ablation	VRM, RKD, Hinton, No-KD	99.9/99.9/99.9/24%
E4	LUT Precision (3-way)	DP-Opt @10pt	**+18.3%** vs uniform
E5	Compiler Scalability	Width/Depth/Grid	11/12 pass; mem bottleneck
E6	Classification	Agree.	**100%**, max logit 3.65
E6-SMT	Z3 Translation Valid.	Per-node verify	9/11 exact, 2/11 bounded, 0 fail
E7	Cross-Load	0/2/3 hp acc.	99.97/100/99.97%
E8	Optimizer Ablation	Full pipeline	-11.9% mem, +65% speed
E9	Doubleton Arith.	DA vs IA bound	**2.2× tighter**
E10	LUT Sparsity Trade-off	Acc vs Storage	99.96% at 3–50 pts
E11	EMSE Calibration	M_2^{char} = 0.177, $M_2^{\max}$=1.586	98.6% tighter than analytic bound
E12	XJTU-SY Cross-Dataset	Zero-shot / FT 100ep	29.8% → 91.7% (+61.9 pp)
E13	Feature Ablation	Time/Freq/DE	Freq 99.93%; All 99.96%
E14	Adversarial Robustness	Noise σ=0–0.2	0% degradation
E15	Adaptive LUT Allocation	Greedy vs Uniform	71.6% ↓ worst ε
E16	Segment-Aware DA	Seg-DA vs DA	6.0× tightening, safety ∼11.9×
E54	ChebyKAN Z3 Verif.	Per-func. Z3 NRA	**496/512 (96.9%)**, Prop. 2
E55	XJTU-SY FT + SCL	100-ep FT + S7-1200	**91.7%**, 512/512 Z3, 0e 0w SCL
E56	Deep KAN (L=3)	[28,16,8,4] 120ep	**99.60%**, 608/608 Z3, DA 14.6× linear
E57	CROWN vs. NeuroPLC	KAN[28,16,4] DA/IBP	**85× tighter**, CROWN=IBP for KAN
Validation Experiments (V1–V7):			
V1	Worst-Case Adversarial	5,000 inputs, 100 worst	100/100 preserved, VALIDATED
V2	LLM vs NeuroPLC	SCL code generation	LLM: 6 defects, no weights; NeuroPLC: 0e 0w
V3	DA $\sqrt{d}$ Scaling Law	105 architectures, 7 dims	Pearson r=0.987, slope 0.896
V4	MLP Verification Gap	KAN $[28, 16, 4]$ vs. MLP $[28, 32, 16, 4]$	512/512 vs. 0/48; 14.0× worse (per-comp.)
V5	ONNX vs NeuroPLC IR	Node count/code	Export fails; ≥8,393 vs. 11 nodes
V6	Z3-Verified WCET	S7-1200 CPU 1211C	Total ≤2.86 ms, 2.9% of cycle
V7	Verification Blind Spot	N=4→15 sweep	Acc. 99.93% but safety <1 at $N\leq$7; 225 flips

V-series: validation experiments targeting specific reviewer questions (LLM comparison, ONNX incompatibility, scaling law, verification gap, adversarial safety, WCET).

TABLE XXX
ADAPTIVE VS. UNIFORM LUT ALLOCATION: ERROR AND STORAGE

Metric	**N=10**	**N=15**	**N=20**	**N=50**
Budget (bytes)	20,480	30,720	40,960	102,400
Uniform worst ε	0.00982	0.00406	0.00220	0.00033
Adaptive worst ε	0.00294	0.00115	0.00061	0.00009
Worst-ε reduction	**70.0%**	**71.6%**	**72.2%**	**73.1%**
Storage at quality parity	—	17,888 B	—	—
Storage saving vs. uniform	—	**41.8%**	—	—
Adaptive N range	$[3, 18]$	$[3, 28]$	$[3, 38]$	$[4, 96]$
Adaptive N median	11	16	21	53

Quality parity: storage needed for adaptive scheme to match uniform's worst-case ε. At N=15 budget, adaptive saves 41.8%. All experiments use 512 B-spline functions from the KAN $[28, 16, 4]$ model.

TABLE XXXI
NEUROPLC VS. RTNNIGEN [1]: MULTI-DIMENSIONAL COMPARISON

Dimension	**NeuroPLC (Ours)**	**RTNNIgen**
Model Support	MLP + KAN (B-spline LUT)	MLP only (Dense layers)
Correctness	Theorem 1 + DA (2.2×) + Segment-Aware	None reported
PLC Platform	Siemens S7-1200/1500 (SCL)	Beckhoff TwinCAT 3 (ST)
Code Structure	DB+FB integrated SCL, human-readable	FB + binary weights file
Activations	B-spline LUT + SiLU + Softmax + Argmax	Standard (ReLU, tanh, etc.)
Verification	166 tests + TIA V21 (0e/0w)	No test suite in repo
Parameter Handling	Human-readable ARRAY OF REAL	Packed binary blob

TABLE XXXII
TOP-10 FEATURES BY DOMAIN SHIFT MAGNITUDE (CWRU → PADERBORN).

Rank	**Cohen's d**	**Interpretation**
9	$+7.40$	Extreme shift (variance-like feature)
7	$+3.83$	Large shift (energy-like feature)
14	$+3.58$	Large shift
13	$+2.53$	Large shift
12	$+1.72$	Large shift
3	$+0.92$	Moderate shift
5	-1.15	Large shift (opposite direction)
10	$+0.80$	Moderate shift
6	$+0.00$	No shift (dimension-invariant feature)
15	0.00	No shift (constant feature)
Mean $\lvert d\rvert$ across 28 features: 0.84. MMD_{RBF} = 0.044 (moderate). Features 9, 7 dominate: together account for 54% of total shift magnitude.		

Positive d = target mean exceeds source mean. Features are z-score normalized per dataset before comparison. The 2 features with $d \approx 0$ are nearly invariant—candidates for domain-robust feature selection in future sensor-agnostic deployments.

TABLE XXXIII
NEUROPLC APPLICABILITY BOUNDARIES AND RECOMMENDED MITIGATIONS

Boundary	**Condition**	**Mitigation**
DA $\rightarrow$ IA	Balanced weight signs (rare)	Fall back to IA; still valid
Adaptive $\rightarrow$ Uniform	Low curvature diversity	Uniform LUT sufficient
L_{net} explosion	Depth ≥ 6 layers	Monte Carlo; limit depth
Domain shift	Cross-sensor/-bearing/-SR	Fine-tune on target domain

TABLE XXXIV
COMPILER TEMPLATE VERIFICATION—Z3 PROVES SCL TEMPLATES CORRECT FOR ALL POSSIBLE PARAMETERS. (TIER 1, ONE-TIME PROOFS.)

IR Operation	**Z3 Result**	**Time (ms)**
MatMul	`UNSAT` (exact equivalence, 16 dim pairs)	27
Add	`UNSAT` (exact equivalence, 8 dimensions)	6
BsplineLUT	`UNSAT` (de Boor bound for all cubic fns)	7
Argmax	`UNSAT` (exact equivalence, 4 dimensions)	17
StandardAct	Analytic proof (transcendental $\exp(x)$)	–
Softmax	Analytic proof (transcendental $\exp(x)$)	–

TABLE XXXV
END-TO-END COMPOSITIONAL VERIFICATION RESULTS (STUDENTKAN $[28, 16, 4]$, 15-PT LUT).

Metric	**Value**
Compiler templates proved (Tier 1)	$4/6$
B-spline functions verified (Tier 2)	$512/512$
Composition steps (Tier 3)	9
Trusted computing base (checker)	$\sim$200 lines Python
Per-function LUT error ε	0.0036
IA network bound Δ_{IA}	0.2473
DA network bound Δ_{DA}	0.1196
DA/IA tightening ratio	$2.1\times$
Certificate valid	Yes
Classification preserved	Yes
Total verification time	1,716 ms

TABLE XXXVI
COMPONENT-LEVEL VERIFIABILITY: KAN VS. MLP $[28, 16, 4]$.

Component	**KAN**	**MLP (SiLU)**	**MLP (ReLU)**
Activation functions	512 (B-spline)	16 (SiLU)	16 (ReLU)
Z3-verifiable	512 (100%)	0 (0%)	16 (100%)
Z3 fragment	Polynomial NRA	Transcendental	Piecewise linear
Composition possible?	Yes (Theorem 1)	No	No*

*Even with per-neuron ReLU verified, MLP composition fails: $\text{ReLU}(Wx + b)$ creates data-dependent error propagation ($\text{ReLU}' \in \{0, 1\}$), breaking the sign-structure preservation that enables DA tightening.

TABLE XXXVII
ERROR PROPAGATION BOUNDS: KAN DA VS. MLP IA. ALL VALUES USE THE SAME PER-COMPONENT LUT ERROR $\varepsilon = 0.0041$ (N=15, M_2=0.177). KAN BOUND FROM SIGN-STRUCTURAL AFFINE ARITHMETIC (EQ. 39); MLP BOUND FROM 3-LAYER INTERVAL PROPAGATION.

Metric	Value
Per-function error ε	0.0041
KAN DA bound Δ_{DA}	0.079
KAN IA bound Δ_{IA}	0.172
MLP IA bound Δ_{MLP}	3.04
DA/IA tightening (KAN)	2.2×
MLP IA / KAN DA overhead	14.0× (per-comp.), 38× (E2E)
MLP IA / KAN IA overhead	17.7×
KAN end-to-end verified	Yes
MLP end-to-end verified	No

TABLE XXXVIII
CROSS-DOMAIN PIPELINE GENERALITY: IDENTICAL COMPILER AND VERIFICATION PIPELINE ACROSS THREE FUNDAMENTALLY DIFFERENT DATA DOMAINS —IMAGE CLASSIFICATION (MNIST), LABORATORY BEARING FAULTS (CWRU), AND RUN-TO-FAILURE NATURAL DEGRADATION (XJTU-SY). CHEBYKAN (CHEBYSHEV POLYNOMIAL BASIS) IS INCLUDED AS AN ALTERNATIVE SVNN-COMPLIANT ARCHITECTURE (PROPOSITION 2).

Metric	MNIST (Image)	CWRU (Vib.)	XJTU-SY (Nat.Wear)	ChebyKAN (Poly.)
Domain	Digits	Bear. faults	Bear. degrad.	Bear. faults
Features	PCA	RCMDE+RCHFDE	28-D	RCMDE+RCHFDE
Accuracy	0.9859	0.9993	0.9172*	1.0000†
Training	Scratch	KD (VRM)	FT 100ep	Scratch
Per-func. verified	512/512	512/512	512/512	496/512
DA bound Δ_{DA}	0.1199	0.1196	0.0493	0.1242‡
SVNN Cond. 3 ($\gamma < 1$)	Yes	Yes	Yes	Yes
Certificate valid	Yes	Yes	Yes	Yes
SCL compilation	0e 0w	0e 0w	0e 0w	0e 0w‡

*Fine-tuned 100 epochs from CWRU checkpoint (E55).

†ChebyKAN [28, 16, 4], Chebyshev degree N=5, trained from scratch on CWRU for 100 epochs. Achieves 100.0% test accuracy, matching B-spline KAN's 99.93% within statistical noise, demonstrating that Chebyshev polynomial basis functions are a viable SVNN-compliant alternative with equal classification performance. ‡ChebyKAN uses Markov-based global M_2 bounds (Eq. 19), which are 5.4× looser than empirical M_2 (E54). The DA bound and SCL compilation are therefore reported with the analytical (conservative) estimate.

TABLE XXXIX
COMPARISON OF PLC ML DEPLOYMENT TOOLS

Dimension	RTNNIgen [1]	MLconverter [2]	NeuroPLC (this work)
Input format	Keras Sequential	scikit-learn (DT, MLP)	**PyTorch (KAN, MLP)**
IR / optimization	None	None	**4-stage IR, 6 passes**
B-spline / KAN	✗	✗	**B-spline+ChebyKAN**
Siemens SCL	Not validated	ABB-specific	**0e 0w TIA V21**
Correctness	✗	✗	**6 Thm + DA + Z3**

Neither tool provides IR-based optimization, B-spline support, or design-time correctness guarantees.

TABLE XL
KAN HARDWARE DEPLOYMENT: LANDSCAPE COMPARISON

Aspect	**KANELÉ (FPGA)**	**LUT-KAN (MCU)**	**KAN-SoC (TinyML)**	**Corrêa (PLC)**	**NeuroPLC (PLC)**
Platform	FPGA	Cortex-M3	MCU	S7-1511TF	S7-1200/1500
Year	2026	2025	2025	2024	2026
Source format	Custom	Post-train	Training	Python	**PyTorch**
PLC code gen	✗	✗	✗	✗	**S7 SCL**
TIA validated	✗	✗	✗	Snap7*	**0e 0w**
Formal cert.	✗	✗	✗	✗	**6 Thm+Z3**
Speedup	2,700×	7–25×	—	Distrib.	Native OB1

*Corrêa [4]: PC-side KAN inference, PLC receives results over Ethernet via Snap7—no native SCL compilation. KANELÉ [5], LUT-KAN [6], KAN-SoC [7].

TABLE XLI
NEUROPLC IR VS. ONNX IR FOR KAN [28, 16, 4]

Metric	**NeuroPLC IR**	**ONNX (opset 14)**
Graph nodes	**11**	Export *fails*
B-spline representation	Native (BsplineLUT)	No standard operator
Decomposed nodes (primitives)	—	$\geq$**8,393**
Node explosion factor	1×	$\geq$**763**×
Generated code lines	**3,818** SCL	>200,000 C (est.)
Code size	**45.2** KB	$\gg$400 KB (est.)
ONNX Runtime size	**0** KB	**22,528** KB
Fits S7-1200 (50 KB)?	**Yes**	**No** (450× over)
Formal verification ready	**Yes** (Z3 SMT)	No

Experiment I: `torch.onnx.export` with opset 14/17/20 fails at B-spline `einsum`. Decomposed estimate: 512 functions × 15 ONNX primitives per B-spline = 7,680 + 713 structural nodes ≈ 8,393. ONNX Runtime minimal build ≈22 MB (no quantization). SCL line count from actual NeuroPLC output.

TABLE XLII
CERTIFICATION THRESHOLDS: MINIMUM LUT DENSITY FOR EACH GUARANTEE TIER.

Guarantee	**Method**	$N_{\min}$	**Safety**	**SIL Target**	**Evidence**
High-probability	DA	15	8.5×	SIL 2	Sign-balance (Lemma 3)
Sound	Seg-IA	15	2.9×	SIL 2	Per-function M_2
Sound	IA (global)	20	1.4×	SIL 3	No sign assumptions
Sound + Robust	IA (global)	25	2.3×	SIL 3+	2× safety margin

$N_{\min}$ = minimum LUT points for safety factor ≥ 1.0. $M_2^{\text{sound}} = 0.3$, $M_2^{\text{emp}} = 0.177$ (E11). Values for KAN $[28, 16, 4]$ on S7-1200 CPU 1211C.